

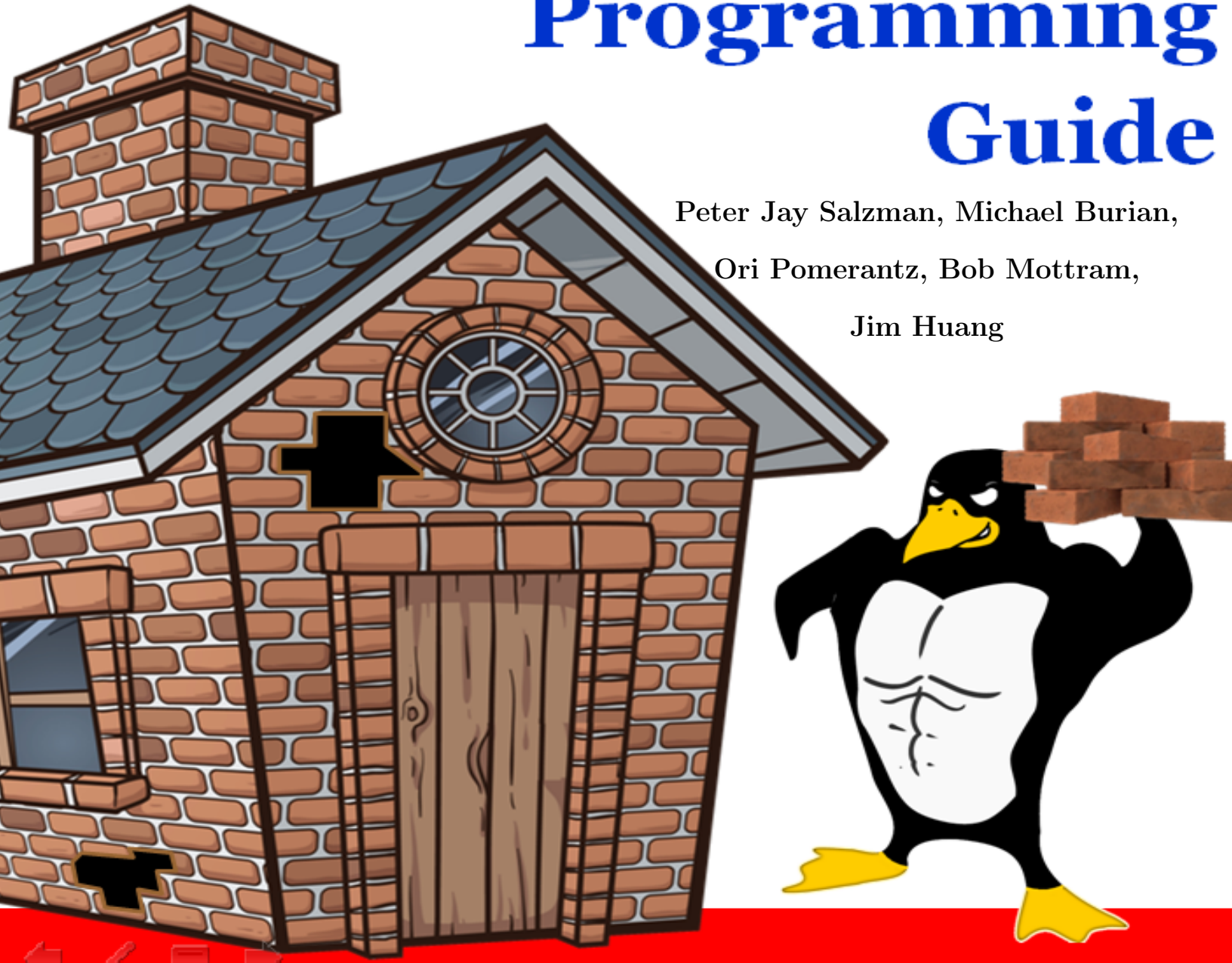
The Linux Kernel Module Programming Guide

Peter Jay Salzman, Michael Burian, Ori Pomerantz, Bob Mottram, Jim Huang

May 1, 2023

The Linux Kernel Module Programming Guide

Peter Jay Salzman, Michael Burian,
Ori Pomerantz, Bob Mottram,
Jim Huang



Contents

1	Introduction	4
1.1	Authorship	4
1.2	Acknowledgements	4
1.3	What Is A Kernel Module?	5
1.4	Kernel module package	5
1.5	What Modules are in my Kernel?	5
1.6	Do I need to download and compile the kernel?	6
1.7	Before We Begin	6
2	Headers	7
3	Examples	7
4	Hello World	8
4.1	The Simplest Module	8
4.2	Hello and Goodbye	12
4.3	The <code>__init</code> and <code>__exit</code> Macros	13
4.4	Licensing and Module Documentation	14
4.5	Passing Command Line Arguments to a Module	15
4.6	Modules Spanning Multiple Files	18
4.7	Building modules for a precompiled kernel	19
5	Preliminaries	21
5.1	How modules begin and end	21
5.2	Functions available to modules	22
5.3	User Space vs Kernel Space	23
5.4	Name Space	23
5.5	Code space	24
5.6	Device Drivers	24
6	Character Device drivers	26
6.1	The <code>file_operations</code> Structure	26
6.2	The file structure	28

6.3	Registering A Device	28
6.4	Unregistering A Device	30
6.5	chardev.c	31
6.6	Writing Modules for Multiple Kernel Versions	34
7	The /proc File System	34
7.1	The proc_ops Structure	37
7.2	Read and Write a /proc File	37
7.3	Manage /proc file with standard filesystem	39
7.4	Manage /proc file with seq_file	42
8	sysfs: Interacting with your module	44
9	Talking To Device Files	47
10	System Calls	58
11	Blocking Processes and threads	67
11.1	Sleep	67
11.2	Completions	74
12	Avoiding Collisions and Deadlocks	76
12.1	Mutex	76
12.2	Spinlocks	77
12.3	Read and write locks	78
12.4	Atomic operations	80
13	Replacing Print Macros	82
13.1	Replacement	82
13.2	Flashing keyboard LEDs	83
14	Scheduling Tasks	86
14.1	Tasklets	86
14.2	Work queues	88
15	Interrupt Handlers	89
15.1	Interrupt Handlers	89
15.2	Detecting button presses	90
15.3	Bottom Half	93
16	Crypto	96
16.1	Hash functions	96
16.2	Symmetric key encryption	98
17	Virtual Input Device Driver	101
18	Standardizing the interfaces: The Device Model	113
19	Optimizations	115
19.1	Likely and Unlikely conditions	115
20	Common Pitfalls	116
20.1	Using standard libraries	116
20.2	Disabling interrupts	116
21	Where To Go From Here?	116

1 Introduction

The Linux Kernel Module Programming Guide is a free book; you may reproduce and/or modify it under the terms of the [Open Software License](#), version 3.0.

This book is distributed in the hope that it would be useful, but without any warranty, without even the implied warranty of merchantability or fitness for a particular purpose.

The author encourages wide distribution of this book for personal or commercial use, provided the above copyright notice remains intact and the method adheres to the provisions of the [Open Software License](#). In summary, you may copy and distribute this book free of charge or for a profit. No explicit permission is required from the author for reproduction of this book in any medium, physical or electronic.

Derivative works and translations of this document must be placed under the Open Software License, and the original copyright notice must remain intact. If you have contributed new material to this book, you must make the material and source code available for your revisions. Please make revisions and updates available directly to the document maintainer, Jim Huang <jserv@ccns.ncku.edu.tw>. This will allow for the merging of updates and provide consistent revisions to the Linux community.

If you publish or distribute this book commercially, donations, royalties, and/or printed copies are greatly appreciated by the author and the [Linux Documentation Project](#) (LDP). Contributing in this way shows your support for free software and the LDP. If you have questions or comments, please contact the address above.

1.1 Authorship

The Linux Kernel Module Programming Guide was originally written for the 2.2 kernels by Ori Pomerantz. Eventually, Ori no longer had time to maintain the document. After all, the Linux kernel is a fast moving target. Peter Jay Salzman took over maintenance and updated it for the 2.4 kernels. Eventually, Peter no longer had time to follow developments with the 2.6 kernel, so Michael Burian became a co-maintainer to update the document for the 2.6 kernels. Bob Mottram updated the examples for 3.8+ kernels. Jim Huang upgraded to recent kernel versions (v5.x) and revised the $\text{\LaTeX}$ document.

1.2 Acknowledgements

The following people have contributed corrections or good suggestions:

2011eric, 25077667, Arush Sharma, asas1asas200, Benno Bielmeyer, Bob Lee, Brad Baker, ccs100203, Chih-Yu Chen, Ching-Hua (Vivian) Lin, ChinYikMing, Cyril Brulebois, Daniele Paolo Scarpazza, David Porter, demonsome, Dimo Velez, Ekang Monyet, fennecJ, Francois Audeon,

gagachang, Gilad Reti, Horst Schirmeier, Hsin-Hsiang Peng, Ignacio Martin, JianXing Wu, linD026, lyctw, manbing, Marconi Jiang, mengxinayan, RinHizakura, Roman Lakeev, Stacy Prowell, Steven Lung, Tristan Lelong, Tucker Polomik, VxTeemo, Wei-Lun Tsai, xatier, Ylowy.

1.3 What Is A Kernel Module?

So, you want to write a kernel module. You know C, you have written a few normal programs to run as processes, and now you want to get to where the real action is, to where a single wild pointer can wipe out your file system and a core dump means a reboot.

What exactly is a kernel module? Modules are pieces of code that can be loaded and unloaded into the kernel upon demand. They extend the functionality of the kernel without the need to reboot the system. For example, one type of module is the device driver, which allows the kernel to access hardware connected to the system. Without modules, we would have to build monolithic kernels and add new functionality directly into the kernel image. Besides having larger kernels, this has the disadvantage of requiring us to rebuild and reboot the kernel every time we want new functionality.

1.4 Kernel module package

Linux distributions provide the commands `modprobe`, `insmod` and `depmod` within a package.

On Ubuntu/Debian:

```
1 sudo apt-get install build-essential kmod
```

On Arch Linux:

```
1 sudo pacman -S gcc kmod
```

1.5 What Modules are in my Kernel?

To discover what modules are already loaded within your current kernel use the command `lsmod`.

```
1 sudo lsmod
```

Modules are stored within the file `/proc/modules`, so you can also see them with:

```
1 sudo cat /proc/modules
```

This can be a long list, and you might prefer to search for something particular. To search for the `fat` module:

```
1 sudo lsmod | grep fat
```

1.6 Do I need to download and compile the kernel?

For the purposes of following this guide you don't necessarily need to do that. However, it would be wise to run the examples within a test distribution running on a virtual machine in order to avoid any possibility of messing up your system.

1.7 Before We Begin

Before we delve into code, there are a few issues we need to cover. Everyone's system is different and everyone has their own groove. Getting your first "hello world" program to compile and load correctly can sometimes be a trick. Rest assured, after you get over the initial hurdle of doing it for the first time, it will be smooth sailing thereafter.

1. Modversioning. A module compiled for one kernel will not load if you boot a different kernel unless you enable `CONFIG_MODVERSIONS` in the kernel. We will not go into module versioning until later in this guide. Until we cover modversions, the examples in the guide may not work if you are running a kernel with modversioning turned on. However, most stock Linux distribution kernels come with it turned on. If you are having trouble loading the modules because of versioning errors, compile a kernel with modversioning turned off.
2. Using X Window System. It is highly recommended that you extract, compile and load all the examples this guide discusses from a console. You should not be working on this stuff in X Window System.

Modules can not print to the screen like `printf()` can, but they can log information and warnings, which ends up being printed on your screen, but only on a console. If you `insmod` a module from an xterm, the information and warnings will be logged, but only to your systemd journal. You will not see it unless you look through your `journalctl`. See [4](#) for details. To have immediate access to this information, do all your work from the console.

3. SecureBoot. Many contemporary computers are pre-configured with UEFI SecureBoot enabled. It is a security standard that can make sure the device boots using only software that is trusted by original equipment manufacturer. The default Linux kernel from some distributions have also enabled the SecureBoot. For such distributions, the kernel module has to be signed with the security key or you would get the "*ERROR: could not insert module*" when you insert your first hello world module:

```
1 insmod ./hello-1.ko
```

And then you can check further with `dmesg` and see the following text:

Lockdown: insmod: unsigned module loading is restricted; see man kernel lockdown.7

If you got this message, the simplest way is to disable the UEFI SecureBoot from the PC/laptop boot menu to have your "hello-1" to be inserted. Of course you can go through complicated steps to generate keys, install keys to your system, and finally sign your module to make it work. However, this is not suitable for beginners. You could read and follow the steps in [SecureBoot](#) if you are interested.

2 Headers

Before you can build anything you'll need to install the header files for your kernel.

On Ubuntu/Debian:

```
1 sudo apt-get update
2 apt-cache search linux-headers-`uname -r`
```

This will tell you what kernel header files are available. Then for example:

```
1 sudo apt-get install kmod linux-headers-5.4.0-80-generic
```

On Arch Linux:

```
1 sudo pacman -S linux-headers
```

3 Examples

All the examples from this document are available within the `examples` subdirectory.

If there are any compile errors then you might have a more recent kernel version or need to install the corresponding kernel header files.

4 Hello World

4.1 The Simplest Module

Most people learning programming start out with some sort of "*hello world*" example. I don't know what happens to people who break with this tradition, but I think it is safer not to find out. We will start with a series of hello world programs that demonstrate the different aspects of the basics of writing a kernel module.

Here is the simplest module possible.

Make a test directory:

```
1 mkdir -p ~/develop/kernel/hello-1
2 cd ~/develop/kernel/hello-1
```

Paste this into your favorite editor and save it as `hello-1.c`:

```
1  /*
2   * hello-1.c - The simplest kernel module.
3   */
4  #include <linux/module.h> /* Needed by all modules */
5  #include <linux/printk.h> /* Needed for pr_info() */
6
7  int init_module(void)
8  {
9      pr_info("Hello world 1.\n");
10
11      /* A non 0 return means init_module failed; module can't be loaded. */
12      return 0;
13  }
14
15  void cleanup_module(void)
16  {
17      pr_info("Goodbye world 1.\n");
18  }
19
20  MODULE_LICENSE("GPL");
```

Now you will need a `Makefile`. If you copy and paste this, change the indentation to use *tabs*, not spaces.

```
1  obj-m += hello-1.o
2
3  PWD := $(CURDIR)
4
5  all:
6      make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
7
8  clean:
9      make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean
```

In **Makefile**, `$(CURDIR)` can be set to the absolute pathname of the current working directory (after all `-C` options are processed, if any). See more about `CURDIR` in [GNU make manual](#).

And finally, just run **make** directly.

```
1 make
```

If there is no `PWD := $(CURDIR)` statement in **Makefile**, then it may not compile correctly with **sudo make**. Because some environment variables are specified by the security policy, they can't be inherited. The default security policy is **sudoers**. In the **sudoers** security policy, `env_reset` is enabled by default, which restricts environment variables. Specifically, path variables are not retained from the user environment, they are set to default values (For more information see: [sudoers manual](#)). You can see the environment variable settings by:

```
$ sudo -s
# sudo -V
```

Here is a simple **Makefile** as an example to demonstrate the problem mentioned above.

```
1 all:
2     echo $(PWD)
```

Then, we can use `-p` flag to print out the environment variable values from the **Makefile**.

```
$ make -p | grep PWD
PWD = /home/ubuntu/temp
OLDPWD = /home/ubuntu
echo $(PWD)
```

The `PWD` variable won't be inherited with **sudo**.

```
$ sudo make -p | grep PWD
echo $(PWD)
```

However, there are three ways to solve this problem.

1. You can use the `-E` flag to temporarily preserve them.

```
1 $ sudo -E make -p | grep PWD
2 PWD = /home/ubuntu/temp
3 OLDPWD = /home/ubuntu
4 echo $(PWD)
```

2. You can set the `env_reset` disabled by editing the `/etc/sudoers` with `root` and `visudo`.

```
1  ## sudoers file.
2  ##
3  ...
4  Defaults env_reset
5  ## Change env_reset to !env_reset in previous line to keep all
   ↪ environment variables
```

Then execute `env` and `sudo env` individually.

```
1  # disable the env_reset
2  echo "user:" > non-env_reset.log; env >>
   ↪ non-env_reset.log
3  echo "root:" >> non-env_reset.log; sudo env >>
   ↪ non-env_reset.log
4  # enable the env_reset
5  echo "user:" > env_reset.log; env >> env_reset.log
6  echo "root:" >> env_reset.log; sudo env >>
   ↪ env_reset.log
```

You can view and compare these logs to find differences between `env_reset` and `!env_reset`.

3. You can preserve environment variables by appending them to `env_keep` in `/etc/sudoers`.

```
1  Defaults env_keep += "PWD"
```

After applying the above change, you can check the environment variable settings by:

```
$ sudo -s
# sudo -V
```

If all goes smoothly you should then find that you have a compiled `hello-1.ko` module. You can find info on it with the command:

```
1  modinfo hello-1.ko
```

At this point the command:

```
1 sudo lsmod | grep hello
```

should return nothing. You can try loading your shiny new module with:

```
1 sudo insmod hello-1.ko
```

The dash character will get converted to an underscore, so when you again try:

```
1 sudo lsmod | grep hello
```

you should now see your loaded module. It can be removed again with:

```
1 sudo rmmod hello_1
```

Notice that the dash was replaced by an underscore. To see what just happened in the logs:

```
1 sudo journalctl --since "1 hour ago" | grep kernel
```

You now know the basics of creating, compiling, installing and removing modules. Now for more of a description of how this module works.

Kernel modules must have at least two functions: a "start" (initialization) function called `init_module()` which is called when the module is `insmod`ed into the kernel, and an "end" (cleanup) function called `cleanup_module()` which is called just before it is removed from the kernel. Actually, things have changed starting with kernel 2.3.13. You can now use whatever name you like for the start and end functions of a module, and you will learn how to do this in Section 4.2. In fact, the new method is the preferred method. However, many people still use `init_module()` and `cleanup_module()` for their start and end functions.

Typically, `init_module()` either registers a handler for something with the kernel, or it replaces one of the kernel functions with its own code (usually code to do something and then call the original function). The `cleanup_module()` function is supposed to undo whatever `init_module()` did, so the module can be unloaded safely.

Lastly, every kernel module needs to include `<linux/module.h>`. We needed to include `<linux/printk.h>` only for the macro expansion for the `pr_alert()` log level, which you'll learn about in Section 2.

1. A point about coding style. Another thing which may not be immediately obvious to anyone getting started with kernel programming is that indentation within your code should be using **tabs** and **not spaces**. It is one of the coding conventions of the kernel. You may not like it, but you'll need to get used to it if you ever submit a patch upstream.

2. Introducing print macros. In the beginning there was `printk`, usually followed by a priority such as `KERN_INFO` or `KERN_DEBUG`. More recently this can also be expressed in abbreviated form using a set of print macros, such as `pr_info` and `pr_debug`. This just saves some mindless keyboard bashing and looks a bit neater. They can be found within [include/linux/printk.h](#). Take time to read through the available priority macros.
3. About Compiling. Kernel modules need to be compiled a bit differently from regular userspace apps. Former kernel versions required us to care much about these settings, which are usually stored in Makefiles. Although hierarchically organized, many redundant settings accumulated in sublevel Makefiles and made them large and rather difficult to maintain. Fortunately, there is a new way of doing these things, called `kbuild`, and the build process for external loadable modules is now fully integrated into the standard kernel build mechanism. To learn more on how to compile modules which are not part of the official kernel (such as all the examples you will find in this guide), see file [Documentation/kbuild/modules.rst](#).

Additional details about Makefiles for kernel modules are available in [Documentation/kbuild/makefiles.rst](#). Be sure to read this and the related files before starting to hack Makefiles. It will probably save you lots of work.

Here is another exercise for the reader. See that comment above the return statement in `init_module()`? Change the return value to something negative, recompile and load the module again. What happens?

4.2 Hello and Goodbye

In early kernel versions you had to use the `init_module` and `cleanup_module` functions, as in the first hello world example, but these days you can name those anything you want by using the `module_init` and `module_exit` macros. These macros are defined in [include/linux/module.h](#). The only requirement is that your init and cleanup functions must be defined before calling the those macros, otherwise you'll get compilation errors. Here is an example of this technique:

```
1  /*
2   * hello-2.c - Demonstrating the module_init() and module_exit() macros.
3   * This is preferred over using init_module() and cleanup_module().
4   */
5  #include <linux/init.h> /* Needed for the macros */
6  #include <linux/module.h> /* Needed by all modules */
7  #include <linux/printk.h> /* Needed for pr_info() */
8
9  static int __init hello_2_init(void)
10 {
11     pr_info("Hello, world 2\n");
12     return 0;
13 }
```

```

14
15 static void __exit hello_2_exit(void)
16 {
17     pr_info("Goodbye, world 2\n");
18 }
19
20 module_init(hello_2_init);
21 module_exit(hello_2_exit);
22
23 MODULE_LICENSE("GPL");

```

So now we have two real kernel modules under our belt. Adding another module is as simple as this:

```

1 obj-m += hello-1.o
2 obj-m += hello-2.o
3
4 PWD := $(CURDIR)
5
6 all:
7     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
8
9 clean:
10    make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

```

Now have a look at [drivers/char/Makefile](#) for a real world example. As you can see, some things got hardwired into the kernel (obj-y) but where have all those obj-m gone? Those familiar with shell scripts will easily be able to spot them. For those who are not, the obj-\$(CONFIG_FOO) entries you see everywhere expand into obj-y or obj-m, depending on whether the CONFIG_FOO variable has been set to y or m. While we are at it, those were exactly the kind of variables that you have set in the .config file in the top-level directory of Linux kernel source tree, the last time when you said `make menuconfig` or something like that.

4.3 The __init and __exit Macros

The `__init` macro causes the init function to be discarded and its memory freed once the init function finishes for built-in drivers, but not loadable modules. If you think about when the init function is invoked, this makes perfect sense.

There is also an `__initdata` which works similarly to `__init` but for init variables rather than functions.

The `__exit` macro causes the omission of the function when the module is built into the kernel, and like `__init`, has no effect for loadable modules. Again, if you consider when the cleanup function runs, this makes complete sense; built-in drivers do not need a cleanup function, while loadable modules do.

These macros are defined in [include/linux/init.h](#) and serve to free up kernel memory. When you boot your kernel and see something like Freeing unused kernel memory: 236k freed, this is precisely what the kernel is freeing.

```
1  /*
2   * hello-3.c - Illustrating the __init, __initdata and __exit macros.
3   */
4  #include <linux/init.h> /* Needed for the macros */
5  #include <linux/module.h> /* Needed by all modules */
6  #include <linux/printk.h> /* Needed for pr_info() */
7
8  static int hello3_data __initdata = 3;
9
10 static int __init hello_3_init(void)
11 {
12     pr_info("Hello, world %d\n", hello3_data);
13     return 0;
14 }
15
16 static void __exit hello_3_exit(void)
17 {
18     pr_info("Goodbye, world %d\n", 3);
19 }
20
21 module_init(hello_3_init);
22 module_exit(hello_3_exit);
23
24 MODULE_LICENSE("GPL");
```

4.4 Licensing and Module Documentation

Honestly, who loads or even cares about proprietary modules? If you do then you might have seen something like this:

```
$ sudo insmod xxxxxx.ko
loading out-of-tree module taints kernel.
module license 'unspecified' taints kernel.
```

You can use a few macros to indicate the license for your module. Some examples are "GPL", "GPL v2", "GPL and additional rights", "Dual BSD/GPL", "Dual MIT/GPL", "Dual MPL/GPL" and "Proprietary". They are defined within [include/linux/module.h](#).

To reference what license you're using a macro is available called `MODULE_LICENSE`. This and a few other macros describing the module are illustrated in the below example.

```
1  /*
2   * hello-4.c - Demonstrates module documentation.
3   */
4  #include <linux/init.h> /* Needed for the macros */
5  #include <linux/module.h> /* Needed by all modules */
6  #include <linux/printk.h> /* Needed for pr_info() */
```

```

7
8 MODULE_LICENSE("GPL");
9 MODULE_AUTHOR("LKMPG");
10 MODULE_DESCRIPTION("A sample driver");
11
12 static int __init init_hello_4(void)
13 {
14     pr_info("Hello, world 4\n");
15     return 0;
16 }
17
18 static void __exit cleanup_hello_4(void)
19 {
20     pr_info("Goodbye, world 4\n");
21 }
22
23 module_init(init_hello_4);
24 module_exit(cleanup_hello_4);

```

4.5 Passing Command Line Arguments to a Module

Modules can take command line arguments, but not with the `argc/argv` you might be used to.

To allow arguments to be passed to your module, declare the variables that will take the values of the command line arguments as global and then use the `module_param()` macro, (defined in [include/linux/moduleparam.h](#)) to set the mechanism up. At runtime, `insmod` will fill the variables with any command line arguments that are given, like `insmod mymodule.ko myvariable=5`. The variable declarations and macros should be placed at the beginning of the module for clarity. The example code should clear up my admittedly lousy explanation.

The `module_param()` macro takes 3 arguments: the name of the variable, its type and permissions for the corresponding file in `sysfs`. Integer types can be signed as usual or unsigned. If you'd like to use arrays of integers or strings see `module_param_array()` and `module_param_string()`.

```

1 int myint = 3;
2 module_param(myint, int, 0);

```

Arrays are supported too, but things are a bit different now than they were in the olden days. To keep track of the number of parameters you need to pass a pointer to a count variable as third parameter. At your option, you could also ignore the count and pass `NULL` instead. We show both possibilities here:

```

1 int myintarray[2];
2 module_param_array(myintarray, int, NULL, 0); /* not interested in count */
3
4 short myshortarray[4];
5 int count;
6 module_param_array(myshortarray, short, &count, 0); /* put count into "count"
   ↳ variable */

```

A good use for this is to have the module variable's default values set, like a port or IO address. If the variables contain the default values, then perform autodetection (explained elsewhere). Otherwise, keep the current value. This will be made clear later on.

Lastly, there is a macro function, `MODULE_PARM_DESC()`, that is used to document arguments that the module can take. It takes two parameters: a variable name and a free form string describing that variable.

```
1  /*
2   * hello-5.c - Demonstrates command line argument passing to a module.
3   */
4  #include <linux/init.h>
5  #include <linux/kernel.h> /* for ARRAY_SIZE() */
6  #include <linux/module.h>
7  #include <linux/moduleparam.h>
8  #include <linux/printk.h>
9  #include <linux/stat.h>
10
11 MODULE_LICENSE("GPL");
12
13 static short int myshort = 1;
14 static int myint = 420;
15 static long int mylong = 9999;
16 static char *mystring = "blah";
17 static int myintarray[2] = { 420, 420 };
18 static int arr_argc = 0;
19
20 /* module_param(foo, int, 0000)
21  * The first param is the parameters name.
22  * The second param is its data type.
23  * The final argument is the permissions bits,
24  * for exposing parameters in sysfs (if non-zero) at a later stage.
25  */
26 module_param(myshort, short, S_IRUSR | S_IWUSR | S_IRGRP | S_IWGRP);
27 MODULE_PARM_DESC(myshort, "A short integer");
28 module_param(myint, int, S_IRUSR | S_IWUSR | S_IRGRP | S_IROTH);
29 MODULE_PARM_DESC(myint, "An integer");
30 module_param(mylong, long, S_IRUSR);
31 MODULE_PARM_DESC(mylong, "A long integer");
32 module_param(mystring, charp, 0000);
33 MODULE_PARM_DESC(mystring, "A character string");
34
35 /* module_param_array(name, type, num, perm);
36  * The first param is the parameter's (in this case the array's) name.
37  * The second param is the data type of the elements of the array.
38  * The third argument is a pointer to the variable that will store the number
39  * of elements of the array initialized by the user at module loading time.
40  * The fourth argument is the permission bits.
41  */
42 module_param_array(myintarray, int, &arr_argc, 0000);
43 MODULE_PARM_DESC(myintarray, "An array of integers");
44
45 static int __init hello_5_init(void)
46 {
47     int i;
48 }
```

```

49     pr_info("Hello, world 5\n=====\\n");
50     pr_info("myshort is a short integer: %hd\\n", myshort);
51     pr_info("myint is an integer: %d\\n", myint);
52     pr_info("mylong is a long integer: %ld\\n", mylong);
53     pr_info("mystring is a string: %s\\n", mystring);
54
55     for (i = 0; i < ARRAY_SIZE(myintarray); i++)
56         pr_info("myintarray[%d] = %d\\n", i, myintarray[i]);
57
58     pr_info("got %d arguments for myintarray.\\n", arr_argc);
59     return 0;
60 }
61
62 static void __exit hello_5_exit(void)
63 {
64     pr_info("Goodbye, world 5\\n");
65 }
66
67 module_init(hello_5_init);
68 module_exit(hello_5_exit);

```

I would recommend playing around with this code:

```

$ sudo insmod hello-5.ko mystring="bebop" myintarray=-1
$ sudo dmesg -t | tail -7
myshort is a short integer: 1
myint is an integer: 420
mylong is a long integer: 9999
mystring is a string: bebop
myintarray[0] = -1
myintarray[1] = 420
got 1 arguments for myintarray.

```

```

$ sudo rmmod hello-5
$ sudo dmesg -t | tail -1
Goodbye, world 5

```

```

$ sudo insmod hello-5.ko mystring="supercalifragilisticexpialidocious" myintarray=-1,-1
$ sudo dmesg -t | tail -7
myshort is a short integer: 1
myint is an integer: 420
mylong is a long integer: 9999
mystring is a string: supercalifragilisticexpialidocious
myintarray[0] = -1
myintarray[1] = -1
got 2 arguments for myintarray.

```

```

$ sudo rmmod hello-5
$ sudo dmesg -t | tail -1
Goodbye, world 5

```

```
$ sudo insmod hello-5.ko mylong=hello
insmod: ERROR: could not insert module hello-5.ko: Invalid parameters
```

4.6 Modules Spanning Multiple Files

Sometimes it makes sense to divide a kernel module between several source files.

Here is an example of such a kernel module.

```
1  /*
2   * start.c - Illustration of multi filed modules
3   */
4
5  #include <linux/kernel.h> /* We are doing kernel work */
6  #include <linux/module.h> /* Specifically, a module */
7
8  int init_module(void)
9  {
10     pr_info("Hello, world - this is the kernel speaking\n");
11     return 0;
12 }
13
14 MODULE_LICENSE("GPL");
```

The next file:

```
1  /*
2   * stop.c - Illustration of multi filed modules
3   */
4
5  #include <linux/kernel.h> /* We are doing kernel work */
6  #include <linux/module.h> /* Specifically, a module */
7
8  void cleanup_module(void)
9  {
10     pr_info("Short is the life of a kernel module\n");
11 }
12
13 MODULE_LICENSE("GPL");
```

And finally, the makefile:

```
1  obj-m += hello-1.o
2  obj-m += hello-2.o
3  obj-m += hello-3.o
4  obj-m += hello-4.o
5  obj-m += hello-5.o
6  obj-m += startstop.o
7  startstop-objs := start.o stop.o
8
9  PWD := $(CURDIR)
10
11 all:
```

```

12         make -C /lib/modules/$(shell uname -r)/build M=$(PWD) modules
13
14 clean:
15     make -C /lib/modules/$(shell uname -r)/build M=$(PWD) clean

```

This is the complete makefile for all the examples we have seen so far. The first five lines are nothing special, but for the last example we will need two lines. First we invent an object name for our combined module, second we tell **make** what object files are part of that module.

4.7 Building modules for a precompiled kernel

Obviously, we strongly suggest you to recompile your kernel, so that you can enable a number of useful debugging features, such as forced module unloading (**MODULE_FORCE_UNLOAD**): when this option is enabled, you can force the kernel to unload a module even when it believes it is unsafe, via a **sudo rmmmod -f module** command. This option can save you a lot of time and a number of reboots during the development of a module. If you do not want to recompile your kernel then you should consider running the examples within a test distribution on a virtual machine. If you mess anything up then you can easily reboot or restore the virtual machine (VM).

There are a number of cases in which you may want to load your module into a precompiled running kernel, such as the ones shipped with common Linux distributions, or a kernel you have compiled in the past. In certain circumstances you could require to compile and insert a module into a running kernel which you are not allowed to recompile, or on a machine that you prefer not to reboot. If you can't think of a case that will force you to use modules for a precompiled kernel you might want to skip this and treat the rest of this chapter as a big footnote.

Now, if you just install a kernel source tree, use it to compile your kernel module and you try to insert your module into the kernel, in most cases you would obtain an error as follows:

```
insmod: ERROR: could not insert module poet.ko: Invalid module format
```

Less cryptic information is logged to the systemd journal:

```
kernel: poet: disagrees about version of symbol module_layout
```

In other words, your kernel refuses to accept your module because version strings (more precisely, *version magic*, see [include/linux/vermagic.h](https://www.kernel.org/doc/Documentation/vermagic.txt)) do not match. Incidentally, version magic strings are stored in the module object in the form of a static string, starting with **vermagic:.** Version data are inserted in your module when it is linked against the **kernel/module.o** file. To inspect version magics and other strings stored in a given module, issue the command **modinfo module.ko**:

```

$ modinfo hello-4.ko
description:    A sample driver
author:        LKMPG
license:       GPL
srcversion:    B2AA7FBFCC2C39AED665382
depends:
retpoline:     Y
name:          hello_4
vermagic:      5.4.0-70-generic SMP mod_unload modversions

```

To overcome this problem we could resort to the `--force-vermagic` option, but this solution is potentially unsafe, and unquestionably unacceptable in production modules. Consequently, we want to compile our module in an environment which was identical to the one in which our precompiled kernel was built. How to do this, is the subject of the remainder of this chapter.

First of all, make sure that a kernel source tree is available, having exactly the same version as your current kernel. Then, find the configuration file which was used to compile your precompiled kernel. Usually, this is available in your current `boot` directory, under a name like `config-5.14.x`. You may just want to copy it to your kernel source tree: `cp /boot/config-`uname -r` .config`.

Let's focus again on the previous error message: a closer look at the version magic strings suggests that, even with two configuration files which are exactly the same, a slight difference in the version magic could be possible, and it is sufficient to prevent insertion of the module into the kernel. That slight difference, namely the custom string which appears in the module's version magic and not in the kernel's one, is due to a modification with respect to the original, in the makefile that some distributions include. Then, examine your `Makefile`, and make sure that the specified version information matches exactly the one used for your current kernel. For example, your makefile could start as follows:

```

VERSION = 5
PATCHLEVEL = 14
SUBLEVEL = 0
EXTRAVERSION = -rc2

```

In this case, you need to restore the value of symbol **EXTRAVERSION** to **-rc2**. We suggest to keep a backup copy of the makefile used to compile your kernel available in `/lib/modules/5.14.0-rc2/build`. A simple command as following should suffice.

```

1 cp /lib/modules/`uname -r`/build/Makefile linux-`uname -r`

```

Here `linux-`uname -r`` is the Linux kernel source you are attempting to build.

Now, please run `make` to update configuration and version headers and objects:

```
$ make
SYNC    include/config/auto.conf.cmd
HOSTCC  scripts/basic/fixdep
HOSTCC  scripts/kconfig/conf.o
HOSTCC  scripts/kconfig/confdata.o
HOSTCC  scripts/kconfig/expr.o
HOSTCC  scripts/kconfig/lexer.lex.c
LEX      scripts/kconfig/lexer.lex.c
YACC     scripts/kconfig/parser.tab.[ch]
HOSTCC  scripts/kconfig/preprocess.o
HOSTCC  scripts/kconfig/symbol.o
HOSTCC  scripts/kconfig/util.o
HOSTCC  scripts/kconfig/lexer.lex.o
HOSTCC  scripts/kconfig/parser.tab.o
HOSTLD  scripts/kconfig/conf
```

If you do not desire to actually compile the kernel, you can interrupt the build process (CTRL-C) just after the SPLIT line, because at that time, the files you need are ready. Now you can turn back to the directory of your module and compile it: It will be built exactly according to your current kernel settings, and it will load into it without any errors.

5 Preliminaries

5.1 How modules begin and end

A program usually begins with a `main()` function, executes a bunch of instructions and terminates upon completion of those instructions. Kernel modules work a bit differently. A module always begin with either the `init_module` or the function you specify with `module_init` call. This is the entry function for modules; it tells the kernel what functionality the module provides and sets up the kernel to run the module's functions when they are needed. Once it does this, entry function returns and the module does nothing until the kernel wants to do something with the code that the module provides.

All modules end by calling either `cleanup_module` or the function you specify with the `module_exit` call. This is the exit function for modules; it undoes whatever entry function did. It unregisters the functionality that the entry function registered.

Every module must have an entry function and an exit function. Since there's more than one way to specify entry and exit functions, I will try my best to use the terms "entry function" and "exit function", but if I slip and simply refer to them as `init_module` and `cleanup_module`, I think you will know what I mean.

5.2 Functions available to modules

Programmers use functions they do not define all the time. A prime example of this is `printf()`. You use these library functions which are provided by the standard C library, `libc`. The definitions for these functions do not actually enter your program until the linking stage, which insures that the code (for `printf()` for example) is available, and fixes the call instruction to point to that code.

Kernel modules are different here, too. In the hello world example, you might have noticed that we used a function, `pr_info()` but did not include a standard I/O library. That is because modules are object files whose symbols get resolved upon running `insmod` or `modprobe`. The definition for the symbols comes from the kernel itself; the only external functions you can use are the ones provided by the kernel. If you're curious about what symbols have been exported by your kernel, take a look at `/proc/kallsyms`.

One point to keep in mind is the difference between library functions and system calls. Library functions are higher level, run completely in user space and provide a more convenient interface for the programmer to the functions that do the real work — system calls. System calls run in kernel mode on the user's behalf and are provided by the kernel itself. The library function `printf()` may look like a very general printing function, but all it really does is format the data into strings and write the string data using the low-level system call `write()`, which then sends the data to standard output.

Would you like to see what system calls are made by `printf()`? It is easy! Compile the following program:

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      printf("hello");
6      return 0;
7  }
```

with `gcc -Wall -o hello hello.c`. Run the executable with `strace ./hello`. Are you impressed? Every line you see corresponds to a system call. `strace` is a handy program that gives you details about what system calls a program is making, including which call is made, what its arguments are and what it returns. It is an invaluable tool for figuring out things like what files a program is trying to access. Towards the end, you will see a line which looks like `write(1, "hello", 5hello)`. There it is. The face behind the `printf()` mask. You may not be familiar with `write`, since most people use library functions for file I/O (like `fopen`, `fputs`, `fclose`). If that is the case, try looking at `man 2 write`. The 2nd man section is devoted to system calls (like `kill()` and `read()`). The 3rd man section is devoted to library calls, which you would probably be more familiar with (like `cosh()` and `random()`).

You can even write modules to replace the kernel's system calls, which we will do shortly. Crackers often make use of this sort of thing for backdoors or

trojans, but you can write your own modules to do more benign things, like have the kernel write Tee hee, that tickles! every time someone tries to delete a file on your system.

5.3 User Space vs Kernel Space

A kernel is all about access to resources, whether the resource in question happens to be a video card, a hard drive or even memory. Programs often compete for the same resource. As I just saved this document, updatedb started updating the locate database. My vim session and updatedb are both using the hard drive concurrently. The kernel needs to keep things orderly, and not give users access to resources whenever they feel like it. To this end, a CPU can run in different modes. Each mode gives a different level of freedom to do what you want on the system. The Intel 80386 architecture had 4 of these modes, which were called rings. Unix uses only two rings; the highest ring (ring 0, also known as “supervisor mode” where everything is allowed to happen) and the lowest ring, which is called “user mode”.

Recall the discussion about library functions vs system calls. Typically, you use a library function in user mode. The library function calls one or more system calls, and these system calls execute on the library function’s behalf, but do so in supervisor mode since they are part of the kernel itself. Once the system call completes its task, it returns and execution gets transferred back to user mode.

5.4 Name Space

When you write a small C program, you use variables which are convenient and make sense to the reader. If, on the other hand, you are writing routines which will be part of a bigger problem, any global variables you have are part of a community of other peoples’ global variables; some of the variable names can clash. When a program has lots of global variables which aren’t meaningful enough to be distinguished, you get namespace pollution. In large projects, effort must be made to remember reserved names, and to find ways to develop a scheme for naming unique variable names and symbols.

When writing kernel code, even the smallest module will be linked against the entire kernel, so this is definitely an issue. The best way to deal with this is to declare all your variables as static and to use a well-defined prefix for your symbols. By convention, all kernel prefixes are lowercase. If you do not want to declare everything as static, another option is to declare a symbol table and register it with the kernel. We will get to this later.

The file `/proc/kallsyms` holds all the symbols that the kernel knows about and which are therefore accessible to your modules since they share the kernel’s codespace.

5.5 Code space

Memory management is a very complicated subject and the majority of O'Reilly's [Understanding The Linux Kernel](#) exclusively covers memory management! We are not setting out to be experts on memory managements, but we do need to know a couple of facts to even begin worrying about writing real modules.

If you have not thought about what a segfault really means, you may be surprised to hear that pointers do not actually point to memory locations. Not real ones, anyway. When a process is created, the kernel sets aside a portion of real physical memory and hands it to the process to use for its executing code, variables, stack, heap and other things which a computer scientist would know about. This memory begins with 0x00000000 and extends up to whatever it needs to be. Since the memory space for any two processes do not overlap, every process that can access a memory address, say 0xbffff978, would be accessing a different location in real physical memory! The processes would be accessing an index named 0xbffff978 which points to some kind of offset into the region of memory set aside for that particular process. For the most part, a process like our Hello, World program can't access the space of another process, although there are ways which we will talk about later.

The kernel has its own space of memory as well. Since a module is code which can be dynamically inserted and removed in the kernel (as opposed to a semi-autonomous object), it shares the kernel's codespace rather than having its own. Therefore, if your module segfaults, the kernel segfaults. And if you start writing over data because of an off-by-one error, then you're trampling on kernel data (or code). This is even worse than it sounds, so try your best to be careful.

By the way, I would like to point out that the above discussion is true for any operating system which uses a monolithic kernel. This is not quite the same thing as *"building all your modules into the kernel"*, although the idea is the same. There are things called microkernels which have modules which get their own codespace. The [GNU Hurd](#) and the [Zircon kernel](#) of Google Fuchsia are two examples of a microkernel.

5.6 Device Drivers

One class of module is the device driver, which provides functionality for hardware like a serial port. On Unix, each piece of hardware is represented by a file located in `/dev` named a device file which provides the means to communicate with the hardware. The device driver provides the communication on behalf of a user program. So the `es1370.ko` sound card device driver might connect the `/dev/sound` device file to the Ensoniq IS1370 sound card. A userspace program like `mp3blaster` can use `/dev/sound` without ever knowing what kind of sound card is installed.

Let's look at some device files. Here are device files which represent the first three partitions on the primary master IDE hard drive:

```
$ ls -l /dev/hda[1-3]
```

```
brw-rw---- 1 root disk 3, 1 Jul 5 2000 /dev/hda1
brw-rw---- 1 root disk 3, 2 Jul 5 2000 /dev/hda2
brw-rw---- 1 root disk 3, 3 Jul 5 2000 /dev/hda3
```

Notice the column of numbers separated by a comma. The first number is called the device's major number. The second number is the minor number. The major number tells you which driver is used to access the hardware. Each driver is assigned a unique major number; all device files with the same major number are controlled by the same driver. All the above major numbers are 3, because they're all controlled by the same driver.

The minor number is used by the driver to distinguish between the various hardware it controls. Returning to the example above, although all three devices are handled by the same driver they have unique minor numbers because the driver sees them as being different pieces of hardware.

Devices are divided into two types: character devices and block devices. The difference is that block devices have a buffer for requests, so they can choose the best order in which to respond to the requests. This is important in the case of storage devices, where it is faster to read or write sectors which are close to each other, rather than those which are further apart. Another difference is that block devices can only accept input and return output in blocks (whose size can vary according to the device), whereas character devices are allowed to use as many or as few bytes as they like. Most devices in the world are character, because they don't need this type of buffering, and they don't operate with a fixed block size. You can tell whether a device file is for a block device or a character device by looking at the first character in the output of `ls -l`. If it is 'b' then it is a block device, and if it is 'c' then it is a character device. The devices you see above are block devices. Here are some character devices (the serial ports):

```
crw-rw---- 1 root dial 4, 64 Feb 18 23:34 /dev/ttyS0
crw-r----- 1 root dial 4, 65 Nov 17 10:26 /dev/ttyS1
crw-rw---- 1 root dial 4, 66 Jul 5 2000 /dev/ttyS2
crw-rw---- 1 root dial 4, 67 Jul 5 2000 /dev/ttyS3
```

If you want to see which major numbers have been assigned, you can look at [Documentation/admin-guide/devices.txt](#).

When the system was installed, all of those device files were created by the `mknod` command. To create a new char device named `coffee` with major/minor number 12 and 2, simply do `mknod /dev/coffee c 12 2`. You do not have to put your device files into `/dev`, but it is done by convention. Linus put his device files in `/dev`, and so should you. However, when creating a device file for testing purposes, it is probably OK to place it in your working directory where you compile the kernel module. Just be sure to put it in the right place when you're done writing the device driver.

I would like to make a few last points which are implicit from the above discussion, but I would like to make them explicit just in case. When a device

file is accessed, the kernel uses the major number of the file to determine which driver should be used to handle the access. This means that the kernel doesn't really need to use or even know about the minor number. The driver itself is the only thing that cares about the minor number. It uses the minor number to distinguish between different pieces of hardware.

By the way, when I say "*hardware*", I mean something a bit more abstract than a PCI card that you can hold in your hand. Look at these two device files:

```
$ ls -l /dev/sda /dev/sdb
brw-rw---- 1 root disk 8,  0 Jan  3 09:02 /dev/sda
brw-rw---- 1 root disk 8, 16 Jan  3 09:02 /dev/sdb
```

By now you can look at these two device files and know instantly that they are block devices and are handled by same driver (block major 8). Sometimes two device files with the same major but different minor number can actually represent the same piece of physical hardware. So just be aware that the word "hardware" in our discussion can mean something very abstract.

6 Character Device drivers

6.1 The file_operations Structure

The `file_operations` structure is defined in `include/linux/fs.h`, and holds pointers to functions defined by the driver that perform various operations on the device. Each field of the structure corresponds to the address of some function defined by the driver to handle a requested operation.

For example, every character driver needs to define a function that reads from the device. The `file_operations` structure holds the address of the module's function that performs that operation. Here is what the definition looks like for kernel 5.4:

```
1  struct file_operations {
2      struct module *owner;
3      loff_t (*llseek) (struct file *, loff_t, int);
4      ssize_t (*read) (struct file *, char __user *, size_t, loff_t *);
5      ssize_t (*write) (struct file *, const char __user *, size_t, loff_t *);
6      ssize_t (*read_iter) (struct kiocb *, struct iov_iter *);
7      ssize_t (*write_iter) (struct kiocb *, struct iov_iter *);
8      int (*iopoll)(struct kiocb *kiocb, bool spin);
9      int (*iterate) (struct file *, struct dir_context *);
10     int (*iterate_shared) (struct file *, struct dir_context *);
11     __poll_t (*poll) (struct file *, struct poll_table_struct *);
12     long (*unlocked_ioctl) (struct file *, unsigned int, unsigned long);
13     long (*compat_ioctl) (struct file *, unsigned int, unsigned long);
14     int (*mmap) (struct file *, struct vm_area_struct *);
15     unsigned long mmap_supported_flags;
16     int (*open) (struct inode *, struct file *);
17     int (*flush) (struct file *, fl_owner_t id);
18     int (*release) (struct inode *, struct file *);
19     int (*fsync) (struct file *, loff_t, loff_t, int datasync);
```

```

20     int (*fasync) (int, struct file *, int);
21     int (*lock) (struct file *, int, struct file_lock *);
22     ssize_t (*sendpage) (struct file *, struct page *, int, size_t, loff_t *,
    ↪ int);
23     unsigned long (*get_unmapped_area)(struct file *, unsigned long, unsigned
    ↪ long, unsigned long, unsigned long);
24     int (*check_flags)(int);
25     int (*flock) (struct file *, int, struct file_lock *);
26     ssize_t (*splice_write)(struct pipe_inode_info *, struct file *, loff_t *,
    ↪ size_t, unsigned int);
27     ssize_t (*splice_read)(struct file *, loff_t *, struct pipe_inode_info *,
    ↪ size_t, unsigned int);
28     int (*setlease)(struct file *, long, struct file_lock **, void **);
29     long (*fallocate)(struct file *file, int mode, loff_t offset,
30         loff_t len);
31     void (*show_fdinfo)(struct seq_file *m, struct file *f);
32     ssize_t (*copy_file_range)(struct file *, loff_t, struct file *,
33         loff_t, size_t, unsigned int);
34     loff_t (*remap_file_range)(struct file *file_in, loff_t pos_in,
35         struct file *file_out, loff_t pos_out,
36         loff_t len, unsigned int remap_flags);
37     int (*fadvise)(struct file *, loff_t, loff_t, int);
38 } __randomize_layout;

```

Some operations are not implemented by a driver. For example, a driver that handles a video card will not need to read from a directory structure. The corresponding entries in the `file_operations` structure should be set to `NULL`.

There is a gcc extension that makes assigning to this structure more convenient. You will see it in modern drivers, and may catch you by surprise. This is what the new way of assigning to the structure looks like:

```

1     struct file_operations fops = {
2         .read = device_read,
3         .write = device_write,
4         .open = device_open,
5         .release = device_release
6     };

```

However, there is also a C99 way of assigning to elements of a structure, [designated initializers](#), and this is definitely preferred over using the GNU extension. You should use this syntax in case someone wants to port your driver. It will help with compatibility:

```

1     struct file_operations fops = {
2         .read = device_read,
3         .write = device_write,
4         .open = device_open,
5         .release = device_release
6     };

```

The meaning is clear, and you should be aware that any member of the structure which you do not explicitly assign will be initialized to `NULL` by gcc.

An instance of `struct file_operations` containing pointers to functions that are used to implement `read`, `write`, `open`, ... system calls is commonly named `fops`.

Since Linux v3.14, the `read`, `write` and `seek` operations are guaranteed for thread-safe by using the `f_pos` specific lock, which makes the file position update to become the mutual exclusion. So, we can safely implement those operations without unnecessary locking.

Additionally, since Linux v5.6, the `proc_ops` structure was introduced to replace the use of the `file_operations` structure when registering proc handlers. See more information in the 7.1 section.

6.2 The file structure

Each device is represented in the kernel by a file structure, which is defined in `include/linux/fs.h`. Be aware that a file is a kernel level structure and never appears in a user space program. It is not the same thing as a `FILE`, which is defined by glibc and would never appear in a kernel space function. Also, its name is a bit misleading; it represents an abstract open ‘file’, not a file on a disk, which is represented by a structure named `inode`.

An instance of struct file is commonly named `filp`. You’ll also see it referred to as a struct file object. Resist the temptation.

Go ahead and look at the definition of file. Most of the entries you see, like struct dentry are not used by device drivers, and you can ignore them. This is because drivers do not fill file directly; they only use structures contained in file which are created elsewhere.

6.3 Registering A Device

As discussed earlier, char devices are accessed through device files, usually located in `/dev`. This is by convention. When writing a driver, it is OK to put the device file in your current directory. Just make sure you place it in `/dev` for a production driver. The major number tells you which driver handles which device file. The minor number is used only by the driver itself to differentiate which device it is operating on, just in case the driver handles more than one device.

Adding a driver to your system means registering it with the kernel. This is synonymous with assigning it a major number during the module’s initialization. You do this by using the `register_chrdev` function, defined by `include/linux/fs.h`.

```
1 int register_chrdev(unsigned int major, const char *name, struct
   ↪ file_operations *fops);
```

Where unsigned int major is the major number you want to request, `const char *name` is the name of the device as it will appear in `/proc/devices` and `struct file_operations *fops` is a pointer to the `file_operations` table for your driver. A negative return

value means the registration failed. Note that we didn't pass the minor number to `register_chrdev`. That is because the kernel doesn't care about the minor number; only our driver uses it.

Now the question is, how do you get a major number without hijacking one that's already in use? The easiest way would be to look through [Documentation/admin-guide/devices.txt](#) and pick an unused one. That is a bad way of doing things because you will never be sure if the number you picked will be assigned later. The answer is that you can ask the kernel to assign you a dynamic major number.

If you pass a major number of 0 to `register_chrdev`, the return value will be the dynamically allocated major number. The downside is that you can not make a device file in advance, since you do not know what the major number will be. There are a couple of ways to do this. First, the driver itself can print the newly assigned number and we can make the device file by hand. Second, the newly registered device will have an entry in `/proc/devices`, and we can either make the device file by hand or write a shell script to read the file in and make the device file. The third method is that we can have our driver make the device file using the `device_create` function after a successful registration and `device_destroy` during the call to `cleanup_module`.

However, `register_chrdev()` would occupy a range of minor numbers associated with the given major. The recommended way to reduce waste for char device registration is using cdev interface.

The newer interface completes the char device registration in two distinct steps. First, we should register a range of device numbers, which can be completed with `register_chrdev_region` or `alloc_chrdev_region`.

```
1 int register_chrdev_region(dev_t from, unsigned count, const char *name);
2 int alloc_chrdev_region(dev_t *dev, unsigned baseminor, unsigned count, const
  ↪ char *name);
```

The choice between two different functions depends on whether you know the major numbers for your device. Using `register_chrdev_region` if you know the device major number and `alloc_chrdev_region` if you would like to allocate a dynamically-allocated major number.

Second, we should initialize the data structure `struct cdev` for our char device and associate it with the device numbers. To initialize the `struct cdev`, we can achieve by the similar sequence of the following codes.

```
1 struct cdev *my_dev = cdev_alloc();
2 my_cdev->ops = &my_fops;
```

However, the common usage pattern will embed the `struct cdev` within a device-specific structure of your own. In this case, we'll need `cdev_init` for the initialization.

```
1 void cdev_init(struct cdev *cdev, const struct file_operations *fops);
```

Once we finish the initialization, we can add the char device to the system by using the `cdev_add`.

```
1 int cdev_add(struct cdev *p, dev_t dev, unsigned count);
```

To find an example using the interface, you can see `ioctl1.c` described in section 9.

6.4 Unregistering A Device

We can not allow the kernel module to be `rmmod`'ed whenever root feels like it. If the device file is opened by a process and then we remove the kernel module, using the file would cause a call to the memory location where the appropriate function (read/write) used to be. If we are lucky, no other code was loaded there, and we'll get an ugly error message. If we are unlucky, another kernel module was loaded into the same location, which means a jump into the middle of another function within the kernel. The results of this would be impossible to predict, but they can not be very positive.

Normally, when you do not want to allow something, you return an error code (a negative number) from the function which is supposed to do it. With `cleanup_module` that's impossible because it is a void function. However, there is a counter which keeps track of how many processes are using your module. You can see what its value is by looking at the 3rd field with the command `cat /proc/modules` or `sudo lsmod`. If this number isn't zero, `rmmod` will fail. Note that you do not have to check the counter within `cleanup_module` because the check will be performed for you by the system call `sys_delete_module`, defined in `include/linux/syscalls.h`. You should not use this counter directly, but there are functions defined in `include/linux/module.h` which let you increase, decrease and display this counter:

- `try_module_get(THIS_MODULE)`: Increment the reference count of current module.
- `module_put(THIS_MODULE)`: Decrement the reference count of current module.
- `module_refcount(THIS_MODULE)`: Return the value of reference count of current module.

It is important to keep the counter accurate; if you ever do lose track of the correct usage count, you will never be able to unload the module; it's now reboot time, boys and girls. This is bound to happen to you sooner or later during a module's development.

6.5 chardev.c

The next code sample creates a char driver named `chardev`. You can dump its device file.

```
1 cat /proc/devices
```

(or open the file with a program) and the driver will put the number of times the device file has been read from into the file. We do not support writing to the file (like `echo "hi" > /dev/hello`), but catch these attempts and tell the user that the operation is not supported. Don't worry if you don't see what we do with the data we read into the buffer; we don't do much with it. We simply read in the data and print a message acknowledging that we received it.

In the multiple-threaded environment, without any protection, concurrent access to the same memory may lead to the race condition, and will not preserve the performance. In the kernel module, this problem may happen due to multiple instances accessing the shared resources. Therefore, a solution is to enforce the exclusive access. We use atomic Compare-And-Swap (CAS) to maintain the states, `CDEV_NOT_USED` and `CDEV_EXCLUSIVE_OPEN`, to determine whether the file is currently opened by someone or not. CAS compares the contents of a memory location with the expected value and, only if they are the same, modifies the contents of that memory location to the desired value. See more concurrency details in the [12](#) section.

[illegible]

```

28 #define DEVICE_NAME "chardev" /* Dev name as it appears in /proc/devices */
29 #define BUF_LEN 80 /* Max length of the message from the device */
30
31 /* Global variables are declared as static, so are global within the file. */
32
33 static int major; /* major number assigned to our device driver */
34
35 enum {
36     CDEV_NOT_USED = 0,
37     CDEV_EXCLUSIVE_OPEN = 1,
38 };
39
40 /* Is device open? Used to prevent multiple access to device */
41 static atomic_t already_open = ATOMIC_INIT(CDEV_NOT_USED);
42
43 static char msg[BUF_LEN + 1]; /* The msg the device will give when asked */
44
45 static struct class *cls;
46
47 static struct file_operations chardev_fops = {
48     .read = device_read,
49     .write = device_write,
50     .open = device_open,
51     .release = device_release,
52 };
53
54 static int __init chardev_init(void)
55 {
56     major = register_chrdev(0, DEVICE_NAME, &chardev_fops);
57
58     if (major < 0) {
59         pr_alert("Registering char device failed with %d\n", major);
60         return major;
61     }
62
63     pr_info("I was assigned major number %d.\n", major);
64
65     cls = class_create(THIS_MODULE, DEVICE_NAME);
66     device_create(cls, NULL, MKDEV(major, 0), NULL, DEVICE_NAME);
67
68     pr_info("Device created on /dev/%s\n", DEVICE_NAME);
69
70     return SUCCESS;
71 }
72
73 static void __exit chardev_exit(void)
74 {
75     device_destroy(cls, MKDEV(major, 0));
76     class_destroy(cls);
77
78     /* Unregister the device */
79     unregister_chrdev(major, DEVICE_NAME);
80 }
81
82 /* Methods */
83
84 /* Called when a process tries to open the device file, like

```

```

85  * "sudo cat /dev/chardev"
86  */
87  static int device_open(struct inode *inode, struct file *file)
88  {
89      static int counter = 0;
90
91      if (atomic_cmpxchg(&already_open, CDEV_NOT_USED, CDEV_EXCLUSIVE_OPEN))
92          return -EBUSY;
93
94      sprintf(msg, "I already told you %d times Hello world!\n", counter++);
95      try_module_get(THIS_MODULE);
96
97      return SUCCESS;
98  }
99
100 /* Called when a process closes the device file. */
101 static int device_release(struct inode *inode, struct file *file)
102 {
103     /* We're now ready for our next caller */
104     atomic_set(&already_open, CDEV_NOT_USED);
105
106     /* Decrement the usage count, or else once you opened the file, you will
107      * never get rid of the module.
108      */
109     module_put(THIS_MODULE);
110
111     return SUCCESS;
112 }
113
114 /* Called when a process, which already opened the dev file, attempts to
115  * read from it.
116  */
117 static ssize_t device_read(struct file *filp, /* see include/linux/fs.h */
118                          char __user *buffer, /* buffer to fill with data */
119                          size_t length, /* length of the buffer */
120                          loff_t *offset)
121 {
122     /* Number of bytes actually written to the buffer */
123     int bytes_read = 0;
124     const char *msg_ptr = msg;
125
126     if (!*(msg_ptr + *offset)) { /* we are at the end of message */
127         *offset = 0; /* reset the offset */
128         return 0; /* signify end of file */
129     }
130
131     msg_ptr += *offset;
132
133     /* Actually put the data into the buffer */
134     while (length && *msg_ptr) {
135         /* The buffer is in the user data segment, not the kernel
136          * segment so "*" assignment won't work. We have to use
137          * put_user which copies data from the kernel data segment to
138          * the user data segment.
139          */
140         put_user(*(msg_ptr++), buffer++);
141         length--;

```

```

142         bytes_read++;
143     }
144
145     *offset += bytes_read;
146
147     /* Most read functions return the number of bytes put into the buffer. */
148     return bytes_read;
149 }
150
151 /* Called when a process writes to dev file: echo "hi" > /dev/hello */
152 static ssize_t device_write(struct file *filp, const char __user *buff,
153                             size_t len, loff_t *off)
154 {
155     pr_alert("Sorry, this operation is not supported.\n");
156     return -EINVAL;
157 }
158
159 module_init(chardev_init);
160 module_exit(chardev_exit);
161
162 MODULE_LICENSE("GPL");

```

6.6 Writing Modules for Multiple Kernel Versions

The system calls, which are the major interface the kernel shows to the processes, generally stay the same across versions. A new system call may be added, but usually the old ones will behave exactly like they used to. This is necessary for backward compatibility – a new kernel version is not supposed to break regular processes. In most cases, the device files will also remain the same. On the other hand, the internal interfaces within the kernel can and do change between versions.

There are differences between different kernel versions, and if you want to support multiple kernel versions, you will find yourself having to code conditional compilation directives. The way to do this is to compare the macro `LINUX_VERSION_CODE` to the macro `KERNEL_VERSION`. In version `a.b.c` of the kernel, the value of this macro would be $2^{16}a + 2^8b + c$.

7 The /proc File System

In Linux, there is an additional mechanism for the kernel and kernel modules to send information to processes — the `/proc` file system. Originally designed to allow easy access to information about processes (hence the name), it is now used by every bit of the kernel which has something interesting to report, such as `/proc/modules` which provides the list of modules and `/proc/meminfo` which gathers memory usage statistics.

The method to use the `proc` file system is very similar to the one used with device drivers — a structure is created with all the information needed for the `/proc` file, including pointers to any handler functions (in our case there is only one, the one called when somebody attempts to read from the `/proc` file).

Then, `init_module` registers the structure with the kernel and `cleanup_module` unregisters it.

Normal file systems are located on a disk, rather than just in memory (which is where `/proc` is), and in that case the index-node (inode for short) number is a pointer to a disk location where the file's inode is located. The inode contains information about the file, for example the file's permissions, together with a pointer to the disk location or locations where the file's data can be found.

Because we don't get called when the file is opened or closed, there's nowhere for us to put `try_module_get` and `module_put` in this module, and if the file is opened and then the module is removed, there's no way to avoid the consequences.

Here a simple example showing how to use a `/proc` file. This is the HelloWorld for the `/proc` filesystem. There are three parts: create the file `/proc/helloworld` in the function `init_module`, return a value (and a buffer) when the file `/proc/helloworld` is read in the callback function `procfile_read`, and delete the file `/proc/helloworld` in the function `cleanup_module`.

The `/proc/helloworld` is created when the module is loaded with the function `proc_create`. The return value is a `struct proc_dir_entry`, and it will be used to configure the file `/proc/helloworld` (for example, the owner of this file). A null return value means that the creation has failed.

Every time the file `/proc/helloworld` is read, the function `procfile_read` is called. Two parameters of this function are very important: the buffer (the second parameter) and the offset (the fourth one). The content of the buffer will be returned to the application which read it (for example the `cat` command). The offset is the current position in the file. If the return value of the function is not null, then this function is called again. So be careful with this function, if it never returns zero, the read function is called endlessly.

```
$ cat /proc/helloworld
HelloWorld!
```

```
1  /*
2   * procfs1.c
3   */
4
5  #include <linux/kernel.h>
6  #include <linux/module.h>
7  #include <linux/proc_fs.h>
8  #include <linux/uaccess.h>
9  #include <linux/version.h>
10
11 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
12 #define HAVE_PROC_OPS
13 #endif
14
15 #define procfs_name "helloworld"
16
17 static struct proc_dir_entry *our_proc_file;
18
19 static ssize_t procfile_read(struct file *file_pointer, char __user *buffer,
```

```

20         size_t buffer_length, loff_t *offset)
21     {
22         char s[13] = "HelloWorld!\n";
23         int len = sizeof(s);
24         ssize_t ret = len;
25
26         if (*offset >= len || copy_to_user(buffer, s, len)) {
27             pr_info("copy_to_user failed\n");
28             ret = 0;
29         } else {
30             pr_info("procfile read %s\n",
31                 ↪ file_pointer->f_path.dentry->d_name.name);
32             *offset += len;
33         }
34
35         return ret;
36     }
37
38     #ifdef HAVE_PROC_OPS
39     static const struct proc_ops proc_file_fops = {
40         .proc_read = procfile_read,
41     };
42     #else
43     static const struct file_operations proc_file_fops = {
44         .read = procfile_read,
45     };
46     #endif
47
48     static int __init procfs1_init(void)
49     {
50         our_proc_file = proc_create(procfs_name, 0644, NULL, &proc_file_fops);
51         if (NULL == our_proc_file) {
52             proc_remove(our_proc_file);
53             pr_alert("Error:Could not initialize /proc/%s\n", procfs_name);
54             return -ENOMEM;
55         }
56
57         pr_info("/proc/%s created\n", procfs_name);
58         return 0;
59     }
60
61     static void __exit procfs1_exit(void)
62     {
63         proc_remove(our_proc_file);
64         pr_info("/proc/%s removed\n", procfs_name);
65     }
66
67     module_init(procfs1_init);
68     module_exit(procfs1_exit);
69
70     MODULE_LICENSE("GPL");

```

7.1 The `proc_ops` Structure

The `proc_ops` structure is defined in [include/linux/proc_fs.h](#) in Linux v5.6+. In older kernels, it used `file_operations` for custom hooks in `/proc` file system, but it contains some members that are unnecessary in VFS, and every time VFS expands `file_operations` set, `/proc` code comes bloated. On the other hand, not only the space, but also some operations were saved by this structure to improve its performance. For example, the file which never disappears in `/proc` can set the `proc_flag` as `PROC_ENTRY_PERMANENT` to save 2 atomic ops, 1 allocation, 1 free in per open/read/close sequence.

7.2 Read and Write a `/proc` File

We have seen a very simple example for a `/proc` file where we only read the file `/proc/helloworld`. It is also possible to write in a `/proc` file. It works the same way as read, a function is called when the `/proc` file is written. But there is a little difference with read, data comes from user, so you have to import data from user space to kernel space (with `copy_from_user` or `get_user`)

The reason for `copy_from_user` or `get_user` is that Linux memory (on Intel architecture, it may be different under some other processors) is segmented. This means that a pointer, by itself, does not reference a unique location in memory, only a location in a memory segment, and you need to know which memory segment it is to be able to use it. There is one memory segment for the kernel, and one for each of the processes.

The only memory segment accessible to a process is its own, so when writing regular programs to run as processes, there is no need to worry about segments. When you write a kernel module, normally you want to access the kernel memory segment, which is handled automatically by the system. However, when the content of a memory buffer needs to be passed between the currently running process and the kernel, the kernel function receives a pointer to the memory buffer which is in the process segment. The `put_user` and `get_user` macros allow you to access that memory. These functions handle only one character, you can handle several characters with `copy_to_user` and `copy_from_user`. As the buffer (in read or write function) is in kernel space, for write function you need to import data because it comes from user space, but not for the read function because data is already in kernel space.

```
1  /*
2   * procfs2.c - create a "file" in /proc
3   */
4
5  #include <linux/kernel.h> /* We're doing kernel work */
6  #include <linux/module.h> /* Specifically, a module */
7  #include <linux/proc_fs.h> /* Necessary because we use the proc fs */
8  #include <linux/uaccess.h> /* for copy_from_user */
9  #include <linux/version.h>
10
11 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
12 #define HAVE_PROC_OPS
```

```

13 #endif
14
15 #define PROCFS_MAX_SIZE 1024
16 #define PROCFS_NAME "buffer1k"
17
18 /* This structure hold information about the /proc file */
19 static struct proc_dir_entry *our_proc_file;
20
21 /* The buffer used to store character for this module */
22 static char procfs_buffer[PROCFS_MAX_SIZE];
23
24 /* The size of the buffer */
25 static unsigned long procfs_buffer_size = 0;
26
27 /* This function is called then the /proc file is read */
28 static ssize_t procfile_read(struct file *file_pointer, char __user *buffer,
29                             size_t buffer_length, loff_t *offset)
30 {
31     char s[13] = "HelloWorld!\n";
32     int len = sizeof(s);
33     ssize_t ret = len;
34
35     if (*offset >= len || copy_to_user(buffer, s, len)) {
36         pr_info("copy_to_user failed\n");
37         ret = 0;
38     } else {
39         pr_info("procfile read %s\n",
40               ↪ file_pointer->f_path.dentry->d_name.name);
41         *offset += len;
42     }
43
44     return ret;
45 }
46
47 /* This function is called with the /proc file is written. */
48 static ssize_t procfile_write(struct file *file, const char __user *buff,
49                              size_t len, loff_t *off)
50 {
51     procfs_buffer_size = len;
52     if (procfs_buffer_size > PROCFS_MAX_SIZE)
53         procfs_buffer_size = PROCFS_MAX_SIZE;
54
55     if (copy_from_user(procfs_buffer, buff, procfs_buffer_size))
56         return -EFAULT;
57
58     procfs_buffer[procfs_buffer_size & (PROCFS_MAX_SIZE - 1)] = '\0';
59     *off += procfs_buffer_size;
60     pr_info("procfile write %s\n", procfs_buffer);
61
62     return procfs_buffer_size;
63 }
64
65 #ifdef HAVE_PROC_OPS
66 static const struct proc_ops proc_file_fops = {
67     .proc_read = procfile_read,
68     .proc_write = procfile_write,
69 };

```

```

69  #else
70  static const struct file_operations proc_file_fops = {
71      .read = procfile_read,
72      .write = procfile_write,
73  };
74  #endif
75
76  static int __init procfs2_init(void)
77  {
78      our_proc_file = proc_create(PROCFS_NAME, 0644, NULL, &proc_file_fops);
79      if (NULL == our_proc_file) {
80          proc_remove(our_proc_file);
81          pr_alert("Error: Could not initialize /proc/%s\n", PROCFS_NAME);
82          return -ENOMEM;
83      }
84
85      pr_info("/proc/%s created\n", PROCFS_NAME);
86      return 0;
87  }
88
89  static void __exit procfs2_exit(void)
90  {
91      proc_remove(our_proc_file);
92      pr_info("/proc/%s removed\n", PROCFS_NAME);
93  }
94
95  module_init(procfs2_init);
96  module_exit(procfs2_exit);
97
98  MODULE_LICENSE("GPL");

```

7.3 Manage /proc file with standard filesystem

We have seen how to read and write a /proc file with the /proc interface. But it is also possible to manage /proc file with inodes. The main concern is to use advanced functions, like permissions.

In Linux, there is a standard mechanism for file system registration. Since every file system has to have its own functions to handle inode and file operations, there is a special structure to hold pointers to all those functions, **struct inode_operations**, which includes a pointer to **struct proc_ops**.

The difference between file and inode operations is that file operations deal with the file itself whereas inode operations deal with ways of referencing the file, such as creating links to it.

In /proc, whenever we register a new file, we're allowed to specify which **struct inode_operations** will be used to access to it. This is the mechanism we use, a **struct inode_operations** which includes a pointer to a **struct proc_ops** which includes pointers to our **procfs_read** and **procfs_write** functions.

Another interesting point here is the **module_permission** function. This function is called whenever a process tries to do something with the /proc file, and it can decide whether to allow access or not. Right now it is only based on the operation and the uid of the current user (as available in **current**, a pointer

to a structure which includes information on the currently running process), but it could be based on anything we like, such as what other processes are doing with the same file, the time of day, or the last input we received.

It is important to note that the standard roles of read and write are reversed in the kernel. Read functions are used for output, whereas write functions are used for input. The reason for that is that read and write refer to the user's point of view — if a process reads something from the kernel, then the kernel needs to output it, and if a process writes something to the kernel, then the kernel receives it as input.

```
1  /*
2   * procfs3.c
3   */
4
5  #include <linux/kernel.h>
6  #include <linux/module.h>
7  #include <linux/proc_fs.h>
8  #include <linux/sched.h>
9  #include <linux/uaccess.h>
10 #include <linux/version.h>
11 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 10, 0)
12 #include <linux/minmax.h>
13 #endif
14
15 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
16 #define HAVE_PROC_OPS
17 #endif
18
19 #define PROCFS_MAX_SIZE 2048UL
20 #define PROCFS_ENTRY_FILENAME "buffer2k"
21
22 static struct proc_dir_entry *our_proc_file;
23 static char procfs_buffer[PROCFS_MAX_SIZE];
24 static unsigned long procfs_buffer_size = 0;
25
26 static ssize_t procfs_read(struct file *filp, char __user *buffer,
27                           size_t length, loff_t *offset)
28 {
29     if (*offset || procfs_buffer_size == 0) {
30         pr_debug("procfs_read: END\n");
31         *offset = 0;
32         return 0;
33     }
34     procfs_buffer_size = min(procfs_buffer_size, length);
35     if (copy_to_user(buffer, procfs_buffer, procfs_buffer_size))
36         return -EFAULT;
37     *offset += procfs_buffer_size;
38
39     pr_debug("procfs_read: read %lu bytes\n", procfs_buffer_size);
40     return procfs_buffer_size;
41 }
42 static ssize_t procfs_write(struct file *file, const char __user *buffer,
43                            size_t len, loff_t *off)
44 {
45     procfs_buffer_size = min(PROCFS_MAX_SIZE, len);
```

```

46     if (copy_from_user(procfs_buffer, buffer, procfs_buffer_size))
47         return -EFAULT;
48     *off += procfs_buffer_size;
49
50     pr_debug("procfs_write: write %lu bytes\n", procfs_buffer_size);
51     return procfs_buffer_size;
52 }
53 static int procfs_open(struct inode *inode, struct file *file)
54 {
55     try_module_get(THIS_MODULE);
56     return 0;
57 }
58 static int procfs_close(struct inode *inode, struct file *file)
59 {
60     module_put(THIS_MODULE);
61     return 0;
62 }
63
64 #ifdef HAVE_PROC_OPS
65 static struct proc_ops file_ops_4_our_proc_file = {
66     .proc_read = procfs_read,
67     .proc_write = procfs_write,
68     .proc_open = procfs_open,
69     .proc_release = procfs_close,
70 };
71 #else
72 static const struct file_operations file_ops_4_our_proc_file = {
73     .read = procfs_read,
74     .write = procfs_write,
75     .open = procfs_open,
76     .release = procfs_close,
77 };
78 #endif
79
80 static int __init procfs3_init(void)
81 {
82     our_proc_file = proc_create(PROCFS_ENTRY_FILENAME, 0644, NULL,
83                                &file_ops_4_our_proc_file);
84     if (our_proc_file == NULL) {
85         remove_proc_entry(PROCFS_ENTRY_FILENAME, NULL);
86         pr_debug("Error: Could not initialize /proc/%s\n",
87                 PROCFS_ENTRY_FILENAME);
88         return -ENOMEM;
89     }
90     proc_set_size(our_proc_file, 80);
91     proc_set_user(our_proc_file, GLOBAL_ROOT_UID, GLOBAL_ROOT_GID);
92
93     pr_debug("/proc/%s created\n", PROCFS_ENTRY_FILENAME);
94     return 0;
95 }
96
97 static void __exit procfs3_exit(void)
98 {
99     remove_proc_entry(PROCFS_ENTRY_FILENAME, NULL);
100    pr_debug("/proc/%s removed\n", PROCFS_ENTRY_FILENAME);
101 }
102

```

```

103 module_init(procfs3_init);
104 module_exit(procfs3_exit);
105
106 MODULE_LICENSE("GPL");

```

Still hungry for procfs examples? Well, first of all keep in mind, there are rumors around, claiming that procfs is on its way out, consider using **sysfs** instead. Consider using this mechanism, in case you want to document something kernel related yourself.

7.4 Manage /proc file with seq_file

As we have seen, writing a /proc file may be quite “complex”. So to help people writting /proc file, there is an API named **seq_file** that helps forming a /proc file for output. It is based on sequence, which is composed of 3 functions: **start()**, **next()**, and **stop()**. The **seq_file** API starts a sequence when a user read the /proc file.

A sequence begins with the call of the function **start()**. If the return is a non NULL value, the function **next()** is called. This function is an iterator, the goal is to go through all the data. Each time **next()** is called, the function **show()** is also called. It writes data values in the buffer read by the user. The function **next()** is called until it returns NULL. The sequence ends when **next()** returns NULL, then the function **stop()** is called.

BE CAREFUL: when a sequence is finished, another one starts. That means that at the end of function **stop()**, the function **start()** is called again. This loop finishes when the function **start()** returns NULL. You can see a scheme of this in the Figure 1.

The **seq_file** provides basic functions for **proc_ops**, such as **seq_read**, **seq_lseek**, and some others. But nothing to write in the /proc file. Of course, you can still use the same way as in the previous example.

```

1  /*
2   * procfs4.c - create a "file" in /proc
3   * This program uses the seq_file library to manage the /proc file.
4   */
5
6  #include <linux/kernel.h> /* We are doing kernel work */
7  #include <linux/module.h> /* Specifically, a module */
8  #include <linux/proc_fs.h> /* Necessary because we use proc fs */
9  #include <linux/seq_file.h> /* for seq_file */
10 #include <linux/version.h>
11
12 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
13 #define HAVE_PROC_OPS
14 #endif
15
16 #define PROC_NAME "iter"
17
18 /* This function is called at the beginning of a sequence.
19  * ie, when:

```

```

20  * - the /proc file is read (first time)
21  * - after the function stop (end of sequence)
22  */
23  static void my_seq_start(struct seq_file *s, loff_t *pos)
24  {
25      static unsigned long counter = 0;
26
27      /* beginning a new sequence? */
28      if (*pos == 0) {
29          /* yes => return a non null value to begin the sequence */
30          return &counter;
31      }
32
33      /* no => it is the end of the sequence, return end to stop reading */
34      *pos = 0;
35      return NULL;
36  }
37
38  /* This function is called after the beginning of a sequence.
39   * It is called untill the return is NULL (this ends the sequence).
40   */
41  static void my_seq_next(struct seq_file *s, void *v, loff_t *pos)
42  {
43      unsigned long *tmp_v = (unsigned long *)v;
44      (*tmp_v)++;
45      (*pos)++;
46      return NULL;
47  }
48
49  /* This function is called at the end of a sequence. */
50  static void my_seq_stop(struct seq_file *s, void *v)
51  {
52      /* nothing to do, we use a static value in start() */
53  }
54
55  /* This function is called for each "step" of a sequence. */
56  static int my_seq_show(struct seq_file *s, void *v)
57  {
58      loff_t *spos = (loff_t *)v;
59
60      seq_printf(s, "%Ld\n", *spos);
61      return 0;
62  }
63
64  /* This structure gather "function" to manage the sequence */
65  static struct seq_operations my_seq_ops = {
66      .start = my_seq_start,
67      .next = my_seq_next,
68      .stop = my_seq_stop,
69      .show = my_seq_show,
70  };
71
72  /* This function is called when the /proc file is open. */
73  static int my_open(struct inode *inode, struct file *file)
74  {
75      return seq_open(file, &my_seq_ops);
76  };

```

```

77
78  /* This structure gather "function" that manage the /proc file */
79  #ifdef HAVE_PROC_OPS
80  static const struct proc_ops my_file_ops = {
81      .proc_open = my_open,
82      .proc_read = seq_read,
83      .proc_lseek = seq_lseek,
84      .proc_release = seq_release,
85  };
86  #else
87  static const struct file_operations my_file_ops = {
88      .open = my_open,
89      .read = seq_read,
90      .llseek = seq_lseek,
91      .release = seq_release,
92  };
93  #endif
94
95  static int __init procfs4_init(void)
96  {
97      struct proc_dir_entry *entry;
98
99      entry = proc_create(PROC_NAME, 0, NULL, &my_file_ops);
100      if (entry == NULL) {
101          remove_proc_entry(PROC_NAME, NULL);
102          pr_debug("Error: Could not initialize /proc/%s\n", PROC_NAME);
103          return -ENOMEM;
104      }
105
106      return 0;
107  }
108
109  static void __exit procfs4_exit(void)
110  {
111      remove_proc_entry(PROC_NAME, NULL);
112      pr_debug("/proc/%s removed\n", PROC_NAME);
113  }
114
115  module_init(procfs4_init);
116  module_exit(procfs4_exit);
117
118  MODULE_LICENSE("GPL");

```

If you want more information, you can read this web page:

- <https://lwn.net/Articles/22355/>
- <https://kernelnewbies.org/Documents/SeqFileHowTo>

You can also read the code of `fs/seq_file.c` in the linux kernel.

8 sysfs: Interacting with your module

sysfs allows you to interact with the running kernel from userspace by reading or setting variables inside of modules. This can be useful for debugging purposes,

or just as an interface for applications or scripts. You can find sysfs directories and files under the `/sys` directory on your system.

```
1 ls -l /sys
```

Attributes can be exported for kobjects in the form of regular files in the filesystem. Sysfs forwards file I/O operations to methods defined for the attributes, providing a means to read and write kernel attributes.

An attribute definition is simply:

```
1 struct attribute {
2     char *name;
3     struct module *owner;
4     umode_t mode;
5 };
6
7 int sysfs_create_file(struct kobject * kobj, const struct attribute * attr);
8 void sysfs_remove_file(struct kobject * kobj, const struct attribute * attr);
```

For example, the driver model defines `struct device_attribute` like:

```
1 struct device_attribute {
2     struct attribute attr;
3     ssize_t (*show)(struct device *dev, struct device_attribute *attr,
4                     char *buf);
5     ssize_t (*store)(struct device *dev, struct device_attribute *attr,
6                     const char *buf, size_t count);
7 };
8
9 int device_create_file(struct device *, const struct device_attribute *);
10 void device_remove_file(struct device *, const struct device_attribute *);
```

To read or write attributes, `show()` or `store()` method must be specified when declaring the attribute. For the common cases `include/linux/sysfs.h` provides convenience macros (`__ATTR`, `__ATTR_RO`, `__ATTR_WO`, etc.) to make defining attributes easier as well as making code more concise and readable.

An example of a hello world module which includes the creation of a variable accessible via sysfs is given below.

```
1 /*
2  * hello-sysfs.c sysfs example
3  */
4 #include <linux/fs.h>
5 #include <linux/init.h>
6 #include <linux/kobject.h>
7 #include <linux/module.h>
8 #include <linux/string.h>
9 #include <linux/sysfs.h>
10
```

```

11 static struct kobject *mymodule;
12
13 /* the variable you want to be able to change */
14 static int myvariable = 0;
15
16 static ssize_t myvariable_show(struct kobject *kobj,
17                               struct kobj_attribute *attr, char *buf)
18 {
19     return sprintf(buf, "%d\n", myvariable);
20 }
21
22 static ssize_t myvariable_store(struct kobject *kobj,
23                                struct kobj_attribute *attr, char *buf,
24                                size_t count)
25 {
26     sscanf(buf, "%du", &myvariable);
27     return count;
28 }
29
30 static struct kobj_attribute myvariable_attribute =
31     __ATTR(myvariable, 0660, myvariable_show, (void *)myvariable_store);
32
33 static int __init mymodule_init(void)
34 {
35     int error = 0;
36
37     pr_info("mymodule: initialised\n");
38
39     mymodule = kobject_create_and_add("mymodule", kernel_kobj);
40     if (!mymodule)
41         return -ENOMEM;
42
43     error = sysfs_create_file(mymodule, &myvariable_attribute.attr);
44     if (error) {
45         pr_info("failed to create the myvariable file "
46               "in /sys/kernel/mymodule\n");
47     }
48
49     return error;
50 }
51
52 static void __exit mymodule_exit(void)
53 {
54     pr_info("mymodule: Exit success\n");
55     kobject_put(mymodule);
56 }
57
58 module_init(mymodule_init);
59 module_exit(mymodule_exit);
60
61 MODULE_LICENSE("GPL");

```

Make and install the module:

```
1 make
```

```
2 sudo insmod hello-sysfs.ko
```

Check that it exists:

```
1 sudo lsmod | grep hello_sysfs
```

What is the current value of `myvariable` ?

```
1 cat /sys/kernel/mymodule/myvariable
```

Set the value of `myvariable` and check that it changed.

```
1 echo "32" > /sys/kernel/mymodule/myvariable
2 cat /sys/kernel/mymodule/myvariable
```

Finally, remove the test module:

```
1 sudo rmmod hello_sysfs
```

In the above case, we use a simple `kobject` to create a directory under `sysfs`, and communicate with its attributes. Since Linux v2.6.0, the `kobject` structure made its appearance. It was initially meant as a simple way of unifying kernel code which manages reference counted objects. After a bit of mission creep, it is now the glue that holds much of the device model and its `sysfs` interface together. For more information about `kobject` and `sysfs`, see [Documentation/driver-api/driver-model/driver.rst](#) and <https://lwn.net/Articles/51437/>.

9 Talking To Device Files

Device files are supposed to represent physical devices. Most physical devices are used for output as well as input, so there has to be some mechanism for device drivers in the kernel to get the output to send to the device from processes. This is done by opening the device file for output and writing to it, just like writing to a file. In the following example, this is implemented by `device_write`.

This is not always enough. Imagine you had a serial port connected to a modem (even if you have an internal modem, it is still implemented from the CPU's perspective as a serial port connected to a modem, so you don't have to tax your imagination too hard). The natural thing to do would be to use the device file to write things to the modem (either modem commands or data to be sent through the phone line) and read things from the modem (either

responses for commands or the data received through the phone line). However, this leaves open the question of what to do when you need to talk to the serial port itself, for example to configure the rate at which data is sent and received.

The answer in Unix is to use a special function called `ioctl` (short for Input Output ConTroL). Every device can have its own `ioctl` commands, which can be read `ioctl`'s (to send information from a process to the kernel), write `ioctl`'s (to return information to a process), both or neither. Notice here the roles of read and write are reversed again, so in `ioctl`'s read is to send information to the kernel and write is to receive information from the kernel.

The `ioctl` function is called with three parameters: the file descriptor of the appropriate device file, the `ioctl` number, and a parameter, which is of type `long` so you can use a cast to use it to pass anything. You will not be able to pass a structure this way, but you will be able to pass a pointer to the structure. Here is an example:

```
1  /*
2   * ioctl.c
3   */
4  #include <linux/cdev.h>
5  #include <linux/fs.h>
6  #include <linux/init.h>
7  #include <linux/ioctl.h>
8  #include <linux/module.h>
9  #include <linux/slab.h>
10 #include <linux/uaccess.h>
11
12 struct ioctl_arg {
13     unsigned int val;
14 };
15
16 /* Documentation/ioctl/ioctl-number.txt */
17 #define IOC_MAGIC '\x66'
18
19 #define IOCTL_VALSET _IOW(IOC_MAGIC, 0, struct ioctl_arg)
20 #define IOCTL_VALGET _IOR(IOC_MAGIC, 1, struct ioctl_arg)
21 #define IOCTL_VALGET_NUM _IOR(IOC_MAGIC, 2, int)
22 #define IOCTL_VALSET_NUM _IOW(IOC_MAGIC, 3, int)
23
24 #define IOCTL_VAL_MAXNR 3
25 #define DRIVER_NAME "ioctltest"
26
27 static unsigned int test_ioctl_major = 0;
28 static unsigned int num_of_dev = 1;
29 static struct cdev test_ioctl_cdev;
30 static int ioctl_num = 0;
31
32 struct test_ioctl_data {
33     unsigned char val;
34     rwlock_t lock;
35 };
36
37 static long test_ioctl_ioctl(struct file *filp, unsigned int cmd,
38                             unsigned long arg)
39 {
```

```

40     struct test_ioctl_data *ioctl_data = filp->private_data;
41     int retval = 0;
42     unsigned char val;
43     struct ioctl_arg data;
44     memset(&data, 0, sizeof(data));
45
46     switch (cmd) {
47     case IOCTL_VALSET:
48         if (copy_from_user(&data, (int __user *)arg, sizeof(data))) {
49             retval = -EFAULT;
50             goto done;
51         }
52
53         pr_alert("IOCTL set val:%x .\n", data.val);
54         write_lock(&ioctl_data->lock);
55         ioctl_data->val = data.val;
56         write_unlock(&ioctl_data->lock);
57         break;
58
59     case IOCTL_VALGET:
60         read_lock(&ioctl_data->lock);
61         val = ioctl_data->val;
62         read_unlock(&ioctl_data->lock);
63         data.val = val;
64
65         if (copy_to_user((int __user *)arg, &data, sizeof(data))) {
66             retval = -EFAULT;
67             goto done;
68         }
69
70         break;
71
72     case IOCTL_VALGET_NUM:
73         retval = __put_user(ioctl_num, (int __user *)arg);
74         break;
75
76     case IOCTL_VALSET_NUM:
77         ioctl_num = arg;
78         break;
79
80     default:
81         retval = -ENOTTY;
82     }
83
84 done:
85     return retval;
86 }
87
88 static ssize_t test_ioctl_read(struct file *filp, char __user *buf,
89                               size_t count, loff_t *f_pos)
90 {
91     struct test_ioctl_data *ioctl_data = filp->private_data;
92     unsigned char val;
93     int retval;
94     int i = 0;
95
96     read_lock(&ioctl_data->lock);

```

```

97     val = ioctl_data->val;
98     read_unlock(&ioctl_data->lock);
99
100     for (; i < count; i++) {
101         if (copy_to_user(&buf[i], &val, 1)) {
102             retval = -EFAULT;
103             goto out;
104         }
105     }
106
107     retval = count;
108 out:
109     return retval;
110 }
111
112 static int test_ioctl_close(struct inode *inode, struct file *filp)
113 {
114     pr_alert("%s call.\n", __func__);
115
116     if (filp->private_data) {
117         kfree(filp->private_data);
118         filp->private_data = NULL;
119     }
120
121     return 0;
122 }
123
124 static int test_ioctl_open(struct inode *inode, struct file *filp)
125 {
126     struct test_ioctl_data *ioctl_data;
127
128     pr_alert("%s call.\n", __func__);
129     ioctl_data = kmalloc(sizeof(struct test_ioctl_data), GFP_KERNEL);
130
131     if (ioctl_data == NULL)
132         return -ENOMEM;
133
134     rwlock_init(&ioctl_data->lock);
135     ioctl_data->val = 0xFF;
136     filp->private_data = ioctl_data;
137
138     return 0;
139 }
140
141 static struct file_operations fops = {
142     .owner = THIS_MODULE,
143     .open = test_ioctl_open,
144     .release = test_ioctl_close,
145     .read = test_ioctl_read,
146     .unlocked_ioctl = test_ioctl_ioctl,
147 };
148
149 static int ioctl_init(void)
150 {
151     dev_t dev;
152     int alloc_ret = -1;
153     int cdev_ret = -1;

```

```

154     alloc_ret = alloc_chrdev_region(&dev, 0, num_of_dev, DRIVER_NAME);
155
156     if (alloc_ret)
157         goto error;
158
159     test_ioctl_major = MAJOR(dev);
160     cdev_init(&test_ioctl_cdev, &fops);
161     cdev_ret = cdev_add(&test_ioctl_cdev, dev, num_of_dev);
162
163     if (cdev_ret)
164         goto error;
165
166     pr_alert("%s driver(major: %d) installed.\n", DRIVER_NAME,
167             test_ioctl_major);
168     return 0;
169 error:
170     if (cdev_ret == 0)
171         cdev_del(&test_ioctl_cdev);
172     if (alloc_ret == 0)
173         unregister_chrdev_region(dev, num_of_dev);
174     return -1;
175 }
176
177 static void ioctl_exit(void)
178 {
179     dev_t dev = MKDEV(test_ioctl_major, 0);
180
181     cdev_del(&test_ioctl_cdev);
182     unregister_chrdev_region(dev, num_of_dev);
183     pr_alert("%s driver removed.\n", DRIVER_NAME);
184 }
185
186 module_init(ioctl_init);
187 module_exit(ioctl_exit);
188
189 MODULE_LICENSE("GPL");
190 MODULE_DESCRIPTION("This is test_ioctl module");

```

You can see there is an argument called `cmd` in `test_ioctl_ioctl()` function. It is the ioctl number. The ioctl number encodes the major device number, the type of the ioctl, the command, and the type of the parameter. This ioctl number is usually created by a macro call (`_IO`, `_IOR`, `_IOW` or `_IOWR` — depending on the type) in a header file. This header file should then be included both by the programs which will use ioctl (so they can generate the appropriate ioctl's) and by the kernel module (so it can understand it). In the example below, the header file is `chardev.h` and the program which uses it is `userspace_ioctl.c`.

If you want to use ioctls in your own kernel modules, it is best to receive an official ioctl assignment, so if you accidentally get somebody else's ioctls, or if they get yours, you'll know something is wrong. For more information, consult the kernel source tree at [Documentation/userspace-api/ioctl/ioctl-number.rst](#).

Also, we need to be careful that concurrent access to the shared resources will lead to the race condition. The solution is using atomic Compare-And-Swap (CAS), which we mentioned at [6.5](#) section, to enforce the exclusive access.

```

1  /*
2   * chardev2.c - Create an input/output character device
3   */
4
5  #include <linux/atomic.h>
6  #include <linux/cdev.h>
7  #include <linux/delay.h>
8  #include <linux/device.h>
9  #include <linux/fs.h>
10 #include <linux/init.h>
11 #include <linux/module.h> /* Specifically, a module */
12 #include <linux/printk.h>
13 #include <linux/types.h>
14 #include <linux/uaccess.h> /* for get_user and put_user */
15
16 #include <asm/errno.h>
17
18 #include "chardev.h"
19 #define SUCCESS 0
20 #define DEVICE_NAME "char_dev"
21 #define BUF_LEN 80
22
23 enum {
24     CDEV_NOT_USED = 0,
25     CDEV_EXCLUSIVE_OPEN = 1,
26 };
27
28 /* Is the device open right now? Used to prevent concurrent access into
29  * the same device
30  */
31 static atomic_t already_open = ATOMIC_INIT(CDEV_NOT_USED);
32
33 /* The message the device will give when asked */
34 static char message[BUF_LEN + 1];
35
36 static struct class *cls;
37
38 /* This is called whenever a process attempts to open the device file */
39 static int device_open(struct inode *inode, struct file *file)
40 {
41     pr_info("device_open(%p)\n", file);
42
43     try_module_get(THIS_MODULE);
44     return SUCCESS;
45 }
46
47 static int device_release(struct inode *inode, struct file *file)
48 {
49     pr_info("device_release(%p,%p)\n", inode, file);
50
51     module_put(THIS_MODULE);
52     return SUCCESS;
53 }
54
55 /* This function is called whenever a process which has already opened the
56  * device file attempts to read from it.

```

```

57  */
58  static ssize_t device_read(struct file *file, /* see include/linux/fs.h */
59                          char __user *buffer, /* buffer to be filled */
60                          size_t length, /* length of the buffer */
61                          loff_t *offset)
62  {
63      /* Number of bytes actually written to the buffer */
64      int bytes_read = 0;
65      /* How far did the process reading the message get? Useful if the message
66       * is larger than the size of the buffer we get to fill in device_read.
67       */
68      const char *message_ptr = message;
69
70      if (!*(message_ptr + *offset)) { /* we are at the end of message */
71          *offset = 0; /* reset the offset */
72          return 0; /* signify end of file */
73      }
74
75      message_ptr += *offset;
76
77      /* Actually put the data into the buffer */
78      while (length && *message_ptr) {
79          /* Because the buffer is in the user data segment, not the kernel
80           * data segment, assignment would not work. Instead, we have to
81           * use put_user which copies data from the kernel data segment to
82           * the user data segment.
83           */
84          put_user(*(message_ptr++), buffer++);
85          length--;
86          bytes_read++;
87      }
88
89      pr_info("Read %d bytes, %ld left\n", bytes_read, length);
90
91      *offset += bytes_read;
92
93      /* Read functions are supposed to return the number of bytes actually
94       * inserted into the buffer.
95       */
96      return bytes_read;
97  }
98
99  /* called when somebody tries to write into our device file. */
100 static ssize_t device_write(struct file *file, const char __user *buffer,
101                          size_t length, loff_t *offset)
102 {
103     int i;
104
105     pr_info("device_write(%p,%p,%ld)", file, buffer, length);
106
107     for (i = 0; i < length && i < BUF_LEN; i++)
108         get_user(message[i], buffer + i);
109
110     /* Again, return the number of input characters used. */
111     return i;
112 }
113

```

```

114  /* This function is called whenever a process tries to do an ioctl on our
115  * device file. We get two extra parameters (additional to the inode and file
116  * structures, which all device functions get): the number of the ioctl
    called
117  * and the parameter given to the ioctl function.
118  *
119  * If the ioctl is write or read/write (meaning output is returned to the
120  * calling process), the ioctl call returns the output of this function.
121  */
122  static long
123  device_ioctl(struct file *file, /* ditto */
124              unsigned int ioctl_num, /* number and param for ioctl */
125              unsigned long ioctl_param)
126  {
127      int i;
128      long ret = SUCCESS;
129
130      /* We don't want to talk to two processes at the same time. */
131      if (atomic_cmpxchg(&already_open, CDEV_NOT_USED, CDEV_EXCLUSIVE_OPEN))
132          return -EBUSY;
133
134      /* Switch according to the ioctl called */
135      switch (ioctl_num) {
136      case IOCTL_SET_MSG: {
137          /* Receive a pointer to a message (in user space) and set that to
138           * be the device's message. Get the parameter given to ioctl by
139           * the process.
140           */
141          char __user *tmp = (char __user *)ioctl_param;
142          char ch;
143
144          /* Find the length of the message */
145          get_user(ch, tmp);
146          for (i = 0; ch && i < BUF_LEN; i++, tmp++)
147              get_user(ch, tmp);
148
149          device_write(file, (char __user *)ioctl_param, i, NULL);
150          break;
151      }
152      case IOCTL_GET_MSG: {
153          loff_t offset = 0;
154
155          /* Give the current message to the calling process - the parameter
156           * we got is a pointer, fill it.
157           */
158          i = device_read(file, (char __user *)ioctl_param, 99, &offset);
159
160          /* Put a zero at the end of the buffer, so it will be properly
161           * terminated.
162           */
163          put_user('\0', (char __user *)ioctl_param + i);
164          break;
165      }
166      case IOCTL_GET_NTH_BYTE:
167          /* This ioctl is both input (ioctl_param) and output (the return
168           * value of this function).
169           */

```

```

170         ret = (long)message[iocctl_param];
171         break;
172     }
173
174     /* We're now ready for our next caller */
175     atomic_set(&already_open, CDEV_NOT_USED);
176
177     return ret;
178 }
179
180 /* Module Declarations */
181
182 /* This structure will hold the functions to be called when a process does
183  * something to the device we created. Since a pointer to this structure
184  * is kept in the devices table, it can't be local to init_module. NULL is
185  * for unimplemented functions.
186  */
187 static struct file_operations fops = {
188     .read = device_read,
189     .write = device_write,
190     .unlocked_ioctl = device_ioctl,
191     .open = device_open,
192     .release = device_release, /* a.k.a. close */
193 };
194
195 /* Initialize the module - Register the character device */
196 static int __init chardev2_init(void)
197 {
198     /* Register the character device (atleast try) */
199     int ret_val = register_chrdev(MAJOR_NUM, DEVICE_NAME, &fops);
200
201     /* Negative values signify an error */
202     if (ret_val < 0) {
203         pr_alert("%s failed with %d\n",
204                 "Sorry, registering the character device ", ret_val);
205         return ret_val;
206     }
207
208     cls = class_create(THIS_MODULE, DEVICE_FILE_NAME);
209     device_create(cls, NULL, MKDEV(MAJOR_NUM, 0), NULL, DEVICE_FILE_NAME);
210
211     pr_info("Device created on /dev/%s\n", DEVICE_FILE_NAME);
212
213     return 0;
214 }
215
216 /* Cleanup - unregister the appropriate file from /proc */
217 static void __exit chardev2_exit(void)
218 {
219     device_destroy(cls, MKDEV(MAJOR_NUM, 0));
220     class_destroy(cls);
221
222     /* Unregister the device */
223     unregister_chrdev(MAJOR_NUM, DEVICE_NAME);
224 }
225
226 module_init(chardev2_init);

```

```
227 module_exit(chardev2_exit);
228
229 MODULE_LICENSE("GPL");
```

```
1  /*
2   * chardev.h - the header file with the ioctl definitions.
3   *
4   * The declarations here have to be in a header file, because they need
5   * to be known both to the kernel module (in chardev2.c) and the process
6   * calling ioctl() (in userspace_ioctl.c).
7   */
8
9  #ifndef CHARDEV_H
10 #define CHARDEV_H
11
12 #include <linux/ioctl.h>
13
14 /* The major device number. We can not rely on dynamic registration
15  * any more, because ioctls need to know it.
16  */
17 #define MAJOR_NUM 100
18
19 /* Set the message of the device driver */
20 #define IOCTL_SET_MSG _IOW(MAJOR_NUM, 0, char *)
21 /* _IOW means that we are creating an ioctl command number for passing
22  * information from a user process to the kernel module.
23  *
24  * The first arguments, MAJOR_NUM, is the major device number we are using.
25  *
26  * The second argument is the number of the command (there could be several
27  * with different meanings).
28  *
29  * The third argument is the type we want to get from the process to the
30  * kernel.
31  */
32
33 /* Get the message of the device driver */
34 #define IOCTL_GET_MSG _IOR(MAJOR_NUM, 1, char *)
35 /* This IOCTL is used for output, to get the message of the device driver.
36  * However, we still need the buffer to place the message in to be input,
37  * as it is allocated by the process.
38  */
39
40 /* Get the n'th byte of the message */
41 #define IOCTL_GET_NTH_BYTE _IOWR(MAJOR_NUM, 2, int)
42 /* The IOCTL is used for both input and output. It receives from the user
43  * a number, n, and returns message[n].
44  */
45
46 /* The name of the device file */
47 #define DEVICE_FILE_NAME "char_dev"
48 #define DEVICE_PATH "/dev/char_dev"
49
50 #endif
```

```

1  /* userspace_ioctl.c - the process to use ioctl's to control the kernel
   ↪ module
2  *
3  * Until now we could have used cat for input and output. But now
4  * we need to do ioctl's, which require writing our own process.
5  */
6
7  /* device specifics, such as ioctl numbers and the
8  * major device file. */
9  #include "../chardev.h"
10
11 #include <stdio.h> /* standard I/O */
12 #include <fcntl.h> /* open */
13 #include <unistd.h> /* close */
14 #include <stdlib.h> /* exit */
15 #include <sys/ioctl.h> /* ioctl */
16
17 /* Functions for the ioctl calls */
18
19 int ioctl_set_msg(int file_desc, char *message)
20 {
21     int ret_val;
22
23     ret_val = ioctl(file_desc, IOCTL_SET_MSG, message);
24
25     if (ret_val < 0) {
26         printf("ioctl_set_msg failed:%d\n", ret_val);
27     }
28
29     return ret_val;
30 }
31
32 int ioctl_get_msg(int file_desc)
33 {
34     int ret_val;
35     char message[100] = { 0 };
36
37     /* Warning - this is dangerous because we don't tell
38     * the kernel how far it's allowed to write, so it
39     * might overflow the buffer. In a real production
40     * program, we would have used two ioctls - one to tell
41     * the kernel the buffer length and another to give
42     * it the buffer to fill
43     */
44     ret_val = ioctl(file_desc, IOCTL_GET_MSG, message);
45
46     if (ret_val < 0) {
47         printf("ioctl_get_msg failed:%d\n", ret_val);
48     }
49     printf("get_msg message:%s", message);
50
51     return ret_val;
52 }
53
54 int ioctl_get_nth_byte(int file_desc)
55 {

```

```

56     int i, c;
57
58     printf("get_nth_byte message:");
59
60     i = 0;
61     do {
62         c = ioctl(file_desc, IOCTL_GET_NTH_BYTE, i++);
63
64         if (c < 0) {
65             printf("\nioctl_get_nth_byte failed at the %d'th byte:\n", i);
66             return c;
67         }
68
69         putchar(c);
70     } while (c != 0);
71
72     return 0;
73 }
74
75 /* Main - Call the ioctl functions */
76 int main(void)
77 {
78     int file_desc, ret_val;
79     char *msg = "Message passed by ioctl\n";
80
81     file_desc = open(DEVICE_PATH, O_RDWR);
82     if (file_desc < 0) {
83         printf("Can't open device file: %s, error:%d\n", DEVICE_PATH,
84             file_desc);
85         exit(EXIT_FAILURE);
86     }
87
88     ret_val = ioctl_set_msg(file_desc, msg);
89     if (ret_val)
90         goto error;
91     ret_val = ioctl_get_nth_byte(file_desc);
92     if (ret_val)
93         goto error;
94     ret_val = ioctl_get_msg(file_desc);
95     if (ret_val)
96         goto error;
97
98     close(file_desc);
99     return 0;
100 error:
101     close(file_desc);
102     exit(EXIT_FAILURE);
103 }

```

10 System Calls

So far, the only thing we've done was to use well defined kernel mechanisms to register /proc files and device handlers. This is fine if you want to do something the kernel programmers thought you'd want, such as write a device driver. But

what if you want to do something unusual, to change the behavior of the system in some way? Then, you are mostly on your own.

If you are not being sensible and using a virtual machine then this is where kernel programming can become hazardous. While writing the example below, I killed the `open()` system call. This meant I could not open any files, I could not run any programs, and I could not shutdown the system. I had to restart the virtual machine. No important files got annihilated, but if I was doing this on some live mission critical system then that could have been a possible outcome. To ensure you do not lose any files, even within a test environment, please run `sync` right before you do the `insmod` and the `rmmmod`.

Forget about `/proc` files, forget about device files. They are just minor details. Minutiae in the vast expanse of the universe. The real process to kernel communication mechanism, the one used by all processes, is *system calls*. When a process requests a service from the kernel (such as opening a file, forking to a new process, or requesting more memory), this is the mechanism used. If you want to change the behaviour of the kernel in interesting ways, this is the place to do it. By the way, if you want to see which system calls a program uses, run `strace <arguments>`.

In general, a process is not supposed to be able to access the kernel. It can not access kernel memory and it can't call kernel functions. The hardware of the CPU enforces this (that is the reason why it is called "protected mode" or "page protection").

System calls are an exception to this general rule. What happens is that the process fills the registers with the appropriate values and then calls a special instruction which jumps to a previously defined location in the kernel (of course, that location is readable by user processes, it is not writable by them). Under Intel CPUs, this is done by means of interrupt 0x80. The hardware knows that once you jump to this location, you are no longer running in restricted user mode, but as the operating system kernel — and therefore you're allowed to do whatever you want.

The location in the kernel a process can jump to is called `system_call`. The procedure at that location checks the system call number, which tells the kernel what service the process requested. Then, it looks at the table of system calls (`sys_call_table`) to see the address of the kernel function to call. Then it calls the function, and after it returns, does a few system checks and then return back to the process (or to a different process, if the process time ran out). If you want to read this code, it is at the source file `arch/${architecture}/kernel/entry.S`, after the line `ENTRY(system_call)`.

So, if we want to change the way a certain system call works, what we need to do is to write our own function to implement it (usually by adding a bit of our own code, and then calling the original function) and then change the pointer at `sys_call_table` to point to our function. Because we might be removed later and we don't want to leave the system in an unstable state, it's important for `cleanup_module` to restore the table to its original state.

To modify the content of `sys_call_table`, we need to consider the control register. A control register is a processor register that changes or controls the

general behavior of the CPU. For x86 architecture, the `cr0` register has various control flags that modify the basic operation of the processor. The WP flag in `cr0` stands for write protection. Once the WP flag is set, the processor disallows further write attempts to the read-only sections. Therefore, we must disable the WP flag before modifying `sys_call_table`. Since Linux v5.3, the `write_cr0` function cannot be used because of the sensitive `cr0` bits pinned by the security issue, the attacker may write into CPU control registers to disable CPU protections like write protection. As a result, we have to provide the custom assembly routine to bypass it.

However, `sys_call_table` symbol is unexported to prevent misuse. But there have few ways to get the symbol, manual symbol lookup and `kallsyms_lookup_name`. Here we use both depend on the kernel version.

Because of the *control-flow integrity*, which is a technique to prevent the redirect execution code from the attacker, for making sure that the indirect calls go to the expected addresses and the return addresses are not changed. Since Linux v5.7, the kernel patched the series of *control-flow enforcement* (CET) for x86, and some configurations of GCC, like GCC versions 9 and 10 in Ubuntu, will add with CET (the `-fcf-protection` option) in the kernel by default. Using that GCC to compile the kernel with `retpoline` off may result in CET being enabled in the kernel. You can use the following command to check out the `-fcf-protection` option is enabled or not:

```
$ gcc -v -Q -O2 --help=target | grep protection
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/9/lto-wrapper
...
gcc version 9.3.0 (Ubuntu 9.3.0-17ubuntu1~20.04)
COLLECT_GCC_OPTIONS='-v' '-Q' '-O2' '--help=target' '-mtune=generic' '-march=x86-64'
/usr/lib/gcc/x86_64-linux-gnu/9/cc1 -v ... -fcf-protection ...
GNU C17 (Ubuntu 9.3.0-17ubuntu1~20.04) version 9.3.0 (x86_64-linux-gnu)
...
```

But CET should not be enabled in the kernel, it may break the Kprobes and bpf. Consequently, CET is disabled since v5.11. To guarantee the manual symbol lookup worked, we only use up to v5.4.

Unfortunately, since Linux v5.7 `kallsyms_lookup_name` is also unexported, it needs certain trick to get the address of `kallsyms_lookup_name`. If `CONFIG_KPROBES` is enabled, we can facilitate the retrieval of function addresses by means of Kprobes to dynamically break into the specific kernel routine. Kprobes inserts a breakpoint at the entry of function by replacing the first bytes of the probed instruction. When a CPU hits the breakpoint, registers are stored, and the control will pass to Kprobes. It passes the addresses of the saved registers and the Kprobe struct to the handler you defined, then executes it. Kprobes can be registered by symbol name or address. Within the symbol name, the address will be handled by the kernel.

Otherwise, specify the address of `sys_call_table` from `/proc/kallsyms` and `/boot/System.map` into `sym` parameter. Following is the sample usage for `/proc/kallsyms`:

```
$ sudo grep sys_call_table /proc/kallsyms
ffffffff82000280 R x32_sys_call_table
ffffffff820013a0 R sys_call_table
ffffffff820023e0 R ia32_sys_call_table
$ sudo insmod syscall.ko sym=0xffffffff820013a0
```

Using the address from `/boot/System.map`, be careful about KASLR (Kernel Address Space Layout Randomization). KASLR may randomize the address of kernel code and data at every boot time, such as the static address listed in `/boot/System.map` will offset by some entropy. The purpose of KASLR is to protect the kernel space from the attacker. Without KASLR, the attacker may find the target address in the fixed address easily. Then the attacker can use return-oriented programming to insert some malicious codes to execute or receive the target data by a tampered pointer. KASLR mitigates these kinds of attacks because the attacker cannot immediately know the target address, but a brute-force attack can still work. If the address of a symbol in `/proc/kallsyms` is different from the address in `/boot/System.map`, KASLR is enabled with the kernel, which your system running on.

```
$ grep GRUB_CMDLINE_LINUX_DEFAULT /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
$ sudo grep sys_call_table /boot/System.map-$(uname -r)
ffffffff82000300 R sys_call_table
$ sudo grep sys_call_table /proc/kallsyms
ffffffff820013a0 R sys_call_table
# Reboot
$ sudo grep sys_call_table /boot/System.map-$(uname -r)
ffffffff82000300 R sys_call_table
$ sudo grep sys_call_table /proc/kallsyms
ffffffff86400300 R sys_call_table
```

If KASLR is enabled, we have to take care of the address from `/proc/kallsyms` each time we reboot the machine. In order to use the address from `/boot/System.map`, make sure that KASLR is disabled. You can add the `nokaslr` for disabling KASLR in next booting time:

```
$ grep GRUB_CMDLINE_LINUX_DEFAULT /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet splash"
$ sudo perl -i -pe 'm/quiet/ and s//quiet nokaslr/' /etc/default/grub
$ grep quiet /etc/default/grub
GRUB_CMDLINE_LINUX_DEFAULT="quiet nokaslr splash"
$ sudo update-grub
```

For more information, check out the following:

- [Cook: Security things in Linux v5.3](#)
- [Unexporting the system call table](#)
- [Control-flow integrity for the kernel](#)
- [Unexporting kallsyms_lookup_name\(\)](#)
- [Kernel Probes \(Kprobes\)](#)
- [Kernel address space layout randomization](#)

The source code here is an example of such a kernel module. We want to “spy” on a certain user, and to `pr_info()` a message whenever that user opens a file. Towards this end, we replace the system call to open a file with our own function, called `our_sys_openat`. This function checks the uid (user’s id) of the current process, and if it is equal to the uid we spy on, it calls `pr_info()` to display the name of the file to be opened. Then, either way, it calls the original `openat()` function with the same parameters, to actually open the file.

The `init_module` function replaces the appropriate location in `sys_call_table` and keeps the original pointer in a variable. The `cleanup_module` function uses that variable to restore everything back to normal. This approach is dangerous, because of the possibility of two kernel modules changing the same system call. Imagine we have two kernel modules, A and B. A’s `openat` system call will be `A_openat` and B’s will be `B_openat`. Now, when A is inserted into the kernel, the system call is replaced with `A_openat`, which will call the original `sys_openat` when it is done. Next, B is inserted into the kernel, which replaces the system call with `B_openat`, which will call what it thinks is the original system call, `A_openat`, when it’s done.

Now, if B is removed first, everything will be well — it will simply restore the system call to `A_openat`, which calls the original. However, if A is removed and then B is removed, the system will crash. A’s removal will restore the system call to the original, `sys_openat`, cutting B out of the loop. Then, when B is removed, it will restore the system call to what it thinks is the original, `A_openat`, which is no longer in memory. At first glance, it appears we could solve this particular problem by checking if the system call is equal to our open function and if so not changing it at all (so that B won’t change the system call when it is removed), but that will cause an even worse problem. When A is removed, it sees that the system call was changed to `B_openat` so that it is no longer pointing to `A_openat`, so it will not restore it to `sys_openat` before it is removed from memory. Unfortunately, `B_openat` will still try to call `A_openat` which is no longer there, so that even without removing B the system would crash.

Note that all the related problems make syscall stealing unfeasible for production use. In order to keep people from doing potential harmful things `sys_call_table` is no longer exported. This means, if you want to do something more than a mere dry run of this example, you will have to patch your current kernel in order to have `sys_call_table` exported.

```

1  /*
2  * syscall.c
3  *
4  * System call "stealing" sample.
5  *
6  * Disables page protection at a processor level by changing the 16th bit
7  * in the cr0 register (could be Intel specific).
8  *
9  * Based on example by Peter Jay Salzman and
10 * https://bbs.archlinux.org/viewtopic.php?id=139406
11 */
12
13 #include <linux/delay.h>
14 #include <linux/kernel.h>
15 #include <linux/module.h>
16 #include <linux/moduleparam.h> /* which will have params */
17 #include <linux/unistd.h> /* The list of system calls */
18 #include <linux/cred.h> /* For current_uid() */
19 #include <linux/uidgid.h> /* For __kuid_val() */
20 #include <linux/version.h>
21
22 /* For the current (process) structure, we need this to know who the
23 * current user is.
24 */
25 #include <linux/sched.h>
26 #include <linux/uaccess.h>
27
28 /* The way we access "sys_call_table" varies as kernel internal changes.
29 * - Prior to v5.4 : manual symbol lookup
30 * - v5.5 to v5.6 : use kallsyms_lookup_name()
31 * - v5.7+       : Kprobes or specific kernel module parameter
32 */
33
34 /* The in-kernel calls to the ksys_close() syscall were removed in Linux
35 ↪ v5.11+.
36 */
37 #if (LINUX_VERSION_CODE < KERNEL_VERSION(5, 7, 0))
38
39 #if LINUX_VERSION_CODE <= KERNEL_VERSION(5, 4, 0)
40 #define HAVE_KSYS_CLOSE 1
41 #include <linux/syscalls.h> /* For ksys_close() */
42 #else
43 #include <linux/kallsyms.h> /* For kallsyms_lookup_name */
44 #endif
45
46 #else
47
48 #if defined(CONFIG_KPROBES)
49 #define HAVE_KPROBES 1
50 #include <linux/kprobes.h>
51 #else
52 #define HAVE_PARAM 1
53 #include <linux/kallsyms.h> /* For sprint_symbol */
54 /* The address of the sys_call_table, which can be obtained with looking up
55 * "/boot/System.map" or "/proc/kallsyms". When the kernel version is v5.7+,
56 * without CONFIG_KPROBES, you can input the parameter or the module will look

```

```

56  * up all the memory.
57  */
58  static unsigned long sym = 0;
59  module_param(sym, ulong, 0644);
60  #endif /* CONFIG_KPROBES */
61
62  #endif /* Version < v5.7 */
63
64  static unsigned long **sys_call_table;
65
66  /* UID we want to spy on - will be filled from the command line. */
67  static uid_t uid = -1;
68  module_param(uid, int, 0644);
69
70  /* A pointer to the original system call. The reason we keep this, rather
71  * than call the original function (sys_openat), is because somebody else
72  * might have replaced the system call before us. Note that this is not
73  * 100% safe, because if another module replaced sys_openat before us,
74  * then when we are inserted, we will call the function in that module -
75  * and it might be removed before we are.
76  *
77  * Another reason for this is that we can not get sys_openat.
78  * It is a static variable, so it is not exported.
79  */
80  #ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
81  static asmlinkage long (*original_call)(const struct pt_regs *);
82  #else
83  static asmlinkage long (*original_call)(int, const char __user *, int,
84  ↪  umode_t);
85  #endif
86
87  /* The function we will replace sys_openat (the function called when you
88  * call the open system call) with. To find the exact prototype, with
89  * the number and type of arguments, we find the original function first
90  * (it is at fs/open.c).
91  *
92  * In theory, this means that we are tied to the current version of the
93  * kernel. In practice, the system calls almost never change (it would
94  * wreck havoc and require programs to be recompiled, since the system
95  * calls are the interface between the kernel and the processes).
96  */
97  #ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
98  static asmlinkage long our_sys_openat(const struct pt_regs *regs)
99  #else
100  static asmlinkage long our_sys_openat(int dfd, const char __user *filename,
101  ↪  int flags, umode_t mode)
102  #endif
103  {
104      int i = 0;
105      char ch;
106
107      if (__kuid_val(current_uid()) != uid)
108          goto orig_call;
109
110      /* Report the file, if relevant */
111      pr_info("Opened file by %d: ", uid);
112      do {

```

```

112 #ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
113     get_user(ch, (char __user *)regs->si + i);
114 #else
115     get_user(ch, (char __user *)filename + i);
116 #endif
117     i++;
118     pr_info("%c", ch);
119 } while (ch != 0);
120 pr_info("\n");
121
122 orig_call:
123     /* Call the original sys_openat - otherwise, we lose the ability to
124     * open files.
125     */
126 #ifdef CONFIG_ARCH_HAS_SYSCALL_WRAPPER
127     return original_call(regs);
128 #else
129     return original_call(dfd, filename, flags, mode);
130 #endif
131 }
132
133 static unsigned long **acquire_sys_call_table(void)
134 {
135 #ifdef HAVE_KSYS_CLOSE
136     unsigned long int offset = PAGE_OFFSET;
137     unsigned long **sct;
138
139     while (offset < ULLONG_MAX) {
140         sct = (unsigned long **)offset;
141
142         if (sct[_NR_close] == (unsigned long *)ksys_close)
143             return sct;
144
145         offset += sizeof(void *);
146     }
147
148     return NULL;
149 #endif
150
151 #ifdef HAVE_PARAM
152     const char sct_name[15] = "sys_call_table";
153     char symbol[40] = { 0 };
154
155     if (sym == 0) {
156         pr_alert("For Linux v5.7+, Kprobes is the preferable way to get "
157                 "symbol.\n");
158         pr_info("If Kprobes is absent, you have to specify the address of "
159                 "sys_call_table symbol\n");
160         pr_info("by /boot/System.map or /proc/kallsyms, which contains all the
161                 ↪ "
162                 "symbol addresses, into sym parameter.\n");
163         return NULL;
164     }
165     sprintf(symbol, sym);
166     if (!strcmp(sct_name, symbol, sizeof(sct_name) - 1))
167         return (unsigned long **)sym;

```

```

168     return NULL;
169 #endif
170
171 #ifdef HAVE_KPROBES
172     unsigned long (*kallsyms_lookup_name)(const char *name);
173     struct kprobe kp = {
174         .symbol_name = "kallsyms_lookup_name",
175     };
176
177     if (register_kprobe(&kp) < 0)
178         return NULL;
179     kallsyms_lookup_name = (unsigned long (*)(const char *name))kp.addr;
180     unregister_kprobe(&kp);
181 #endif
182
183     return (unsigned long **)kallsyms_lookup_name("sys_call_table");
184 }
185
186 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 3, 0)
187 static inline void __write_cr0(unsigned long cr0)
188 {
189     asm volatile("mov %0,%%cr0" : "+r"(cr0) : : "memory");
190 }
191 #else
192 #define __write_cr0 write_cr0
193 #endif
194
195 static void enable_write_protection(void)
196 {
197     unsigned long cr0 = read_cr0();
198     set_bit(16, &cr0);
199     __write_cr0(cr0);
200 }
201
202 static void disable_write_protection(void)
203 {
204     unsigned long cr0 = read_cr0();
205     clear_bit(16, &cr0);
206     __write_cr0(cr0);
207 }
208
209 static int __init syscall_start(void)
210 {
211     if (!(sys_call_table = acquire_sys_call_table()))
212         return -1;
213
214     disable_write_protection();
215
216     /* keep track of the original open function */
217     original_call = (void *)sys_call_table[__NR_openat];
218
219     /* use our openat function instead */
220     sys_call_table[__NR_openat] = (unsigned long *)our_sys_openat;
221
222     enable_write_protection();
223
224     pr_info("Spying on UID:%d\n", uid);

```

```

225     return 0;
226 }
227
228
229 static void __exit syscall_end(void)
230 {
231     if (!sys_call_table)
232         return;
233
234     /* Return the system call back to normal */
235     if (sys_call_table[__NR_openat] != (unsigned long *)our_sys_openat) {
236         pr_alert("Somebody else also played with the ");
237         pr_alert("open system call\n");
238         pr_alert("The system may be left in ");
239         pr_alert("an unstable state.\n");
240     }
241
242     disable_write_protection();
243     sys_call_table[__NR_openat] = (unsigned long *)original_call;
244     enable_write_protection();
245
246     msleep(2000);
247 }
248
249 module_init(syscall_start);
250 module_exit(syscall_end);
251
252 MODULE_LICENSE("GPL");

```

11 Blocking Processes and threads

11.1 Sleep

What do you do when somebody asks you for something you can not do right away? If you are a human being and you are bothered by a human being, the only thing you can say is: "*Not right now, I'm busy. Go away!*". But if you are a kernel module and you are bothered by a process, you have another possibility. You can put the process to sleep until you can service it. After all, processes are being put to sleep by the kernel and woken up all the time (that is the way multiple processes appear to run on the same time on a single CPU).

This kernel module is an example of this. The file (called `/proc/sleep`) can only be opened by a single process at a time. If the file is already open, the kernel module calls `wait_event_interruptible`. The easiest way to keep a file open is to open it with:

```
1 tail -f
```

This function changes the status of the task (a task is the kernel data structure which holds information about a process and the system call it is in, if

any) to `TASK_INTERRUPTIBLE`, which means that the task will not run until it is woken up somehow, and adds it to `WaitQ`, the queue of tasks waiting to access the file. Then, the function calls the scheduler to context switch to a different process, one which has some use for the CPU.

When a process is done with the file, it closes it, and `module_close` is called. That function wakes up all the processes in the queue (there's no mechanism to only wake up one of them). It then returns and the process which just closed the file can continue to run. In time, the scheduler decides that that process has had enough and gives control of the CPU to another process. Eventually, one of the processes which was in the queue will be given control of the CPU by the scheduler. It starts at the point right after the call to `wait_event_interruptible`.

This means that the process is still in kernel mode - as far as the process is concerned, it issued the open system call and the system call has not returned yet. The process does not know somebody else used the CPU for most of the time between the moment it issued the call and the moment it returned.

It can then proceed to set a global variable to tell all the other processes that the file is still open and go on with its life. When the other processes get a piece of the CPU, they'll see that global variable and go back to sleep.

So we will use `tail -f` to keep the file open in the background, while trying to access it with another process (again in the background, so that we need not switch to a different vt). As soon as the first background process is killed with `kill %1`, the second is woken up, is able to access the file and finally terminates.

To make our life more interesting, `module_close` does not have a monopoly on waking up the processes which wait to access the file. A signal, such as `Ctrl +c` (**SIGINT**) can also wake up a process. This is because we used `wait_event_interruptible`. We could have used `wait_event` instead, but that would have resulted in extremely angry users whose `Ctrl+c`'s are ignored.

In that case, we want to return with `-EINTR` immediately. This is important so users can, for example, kill the process before it receives the file.

There is one more point to remember. Some times processes don't want to sleep, they want either to get what they want immediately, or to be told it cannot be done. Such processes use the `O_NONBLOCK` flag when opening the file. The kernel is supposed to respond by returning with the error code `-EAGAIN` from operations which would otherwise block, such as opening the file in this example. The program `cat_nonblock`, available in the `examples/other` directory, can be used to open a file with `O_NONBLOCK`.

```
$ sudo insmod sleep.ko
$ cat_nonblock /proc/sleep
Last input:
$ tail -f /proc/sleep &
Last input:
Last input:
Last input:
Last input:
Last input:
Last input:
```

```

Last input:
Last input:
tail: /proc/sleep: file truncated
[1] 6540
$ cat_nonblock /proc/sleep
Open would block
$ kill %1
[1]+  Terminated                  tail -f /proc/sleep
$ cat_nonblock /proc/sleep
Last input:
$

```

```

1  /*
2   * sleep.c - create a /proc file, and if several processes try to open it
3   * at the same time, put all but one to sleep.
4   */
5
6  #include <linux/atomic.h>
7  #include <linux/fs.h>
8  #include <linux/kernel.h> /* for sprintf() */
9  #include <linux/module.h> /* Specifically, a module */
10 #include <linux/printk.h>
11 #include <linux/proc_fs.h> /* Necessary because we use proc fs */
12 #include <linux/types.h>
13 #include <linux/uaccess.h> /* for get_user and put_user */
14 #include <linux/version.h>
15 #include <linux/wait.h> /* For putting processes to sleep and
16                        waking them up */
17
18 #include <asm/current.h>
19 #include <asm/errno.h>
20
21 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 6, 0)
22 #define HAVE_PROC_OPS
23 #endif
24
25 /* Here we keep the last message received, to prove that we can process our
26  * input.
27  */
28 #define MESSAGE_LENGTH 80
29 static char message[MESSAGE_LENGTH];
30
31 static struct proc_dir_entry *our_proc_file;
32 #define PROC_ENTRY_FILENAME "sleep"
33
34 /* Since we use the file operations struct, we can't use the special proc
35  * output provisions - we have to use a standard read function, which is this
36  * function.
37  */
38 static ssize_t module_output(struct file *file, /* see include/linux/fs.h */
39                             char __user *buf, /* The buffer to put data to
40                             (in the user segment) */
41                             size_t len, /* The length of the buffer */
42                             loff_t *offset)
43 {

```

```

44     static int finished = 0;
45     int i;
46     char output_msg[MESSAGE_LENGTH + 30];
47
48     /* Return 0 to signify end of file - that we have nothing more to say
49      * at this point.
50      */
51     if (finished) {
52         finished = 0;
53         return 0;
54     }
55
56     sprintf(output_msg, "Last input:%s\n", message);
57     for (i = 0; i < len && output_msg[i]; i++)
58         put_user(output_msg[i], buf + i);
59
60     finished = 1;
61     return i; /* Return the number of bytes "read" */
62 }
63
64 /* This function receives input from the user when the user writes to the
65  * /proc file.
66  */
67 static ssize_t module_input(struct file *file, /* The file itself */
68                             const char __user *buf, /* The buffer with input
69                             ↪ */
69                             size_t length, /* The buffer's length */
70                             loff_t *offset) /* offset to file - ignore */
71 {
72     int i;
73
74     /* Put the input into Message, where module_output will later be able
75      * to use it.
76      */
77     for (i = 0; i < MESSAGE_LENGTH - 1 && i < length; i++)
78         get_user(message[i], buf + i);
79     /* we want a standard, zero terminated string */
80     message[i] = '\0';
81
82     /* We need to return the number of input characters used */
83     return i;
84 }
85
86 /* 1 if the file is currently open by somebody */
87 static atomic_t already_open = ATOMIC_INIT(0);
88
89 /* Queue of processes who want our file */
90 static DECLARE_WAIT_QUEUE_HEAD(waitq);
91
92 /* Called when the /proc file is opened */
93 static int module_open(struct inode *inode, struct file *file)
94 {
95     /* If the file's flags include O_NONBLOCK, it means the process does not
96      * want to wait for the file. In this case, if the file is already open,
97      * we should fail with -EAGAIN, meaning "you will have to try again",
98      * instead of blocking a process which would rather stay awake.
99      */

```

```

100     if ((file->f_flags & O_NONBLOCK) && atomic_read(&already_open))
101         return -EAGAIN;
102
103     /* This is the correct place for try_module_get(THIS_MODULE) because if
104      * a process is in the loop, which is within the kernel module,
105      * the kernel module must not be removed.
106      */
107     try_module_get(THIS_MODULE);
108
109     while (atomic_cmpxchg(&already_open, 0, 1)) {
110         int i, is_sig = 0;
111
112         /* This function puts the current process, including any system
113          * calls, such as us, to sleep. Execution will be resumed right
114          * after the function call, either because somebody called
115          * wake_up(&waitq) (only module_close does that, when the file
116          * is closed) or when a signal, such as Ctrl-C, is sent
117          * to the process
118          */
119         wait_event_interruptible(waitq, !atomic_read(&already_open));
120
121         /* If we woke up because we got a signal we're not blocking,
122          * return -EINTR (fail the system call). This allows processes
123          * to be killed or stopped.
124          */
125         for (i = 0; i < _NSIG_WORDS && !is_sig; i++)
126             is_sig = current->pending.signal.sig[i] &
127                 ~current->blocked.sig[i];
128
129         if (is_sig) {
130             /* It is important to put module_put(THIS_MODULE) here, because
131              * for processes where the open is interrupted there will never
132              * be a corresponding close. If we do not decrement the usage
133              * count here, we will be left with a positive usage count
134              * which we will have no way to bring down to zero, giving us
135              * an immortal module, which can only be killed by rebooting
136              * the machine.
137              */
138             module_put(THIS_MODULE);
139             return -EINTR;
140         }
141     }
142
143     return 0; /* Allow the access */
144 }
145
146 /* Called when the /proc file is closed */
147 static int module_close(struct inode *inode, struct file *file)
148 {
149     /* Set already_open to zero, so one of the processes in the waitq will
150      * be able to set already_open back to one and to open the file. All
151      * the other processes will be called when already_open is back to one,
152      * so they'll go back to sleep.
153      */
154     atomic_set(&already_open, 0);
155
156     /* Wake up all the processes in waitq, so if anybody is waiting for the

```

```

156     * file, they can have it.
157     */
158     wake_up(&waitq);
159
160     module_put(THIS_MODULE);
161
162     return 0; /* success */
163 }
164
165 /* Structures to register as the /proc file, with pointers to all the relevant
166  * functions.
167  */
168
169 /* File operations for our proc file. This is where we place pointers to all
170  * the functions called when somebody tries to do something to our file. NULL
171  * means we don't want to deal with something.
172  */
173 #ifdef HAVE_PROC_OPS
174 static const struct proc_ops file_ops_4_our_proc_file = {
175     .proc_read = module_output, /* "read" from the file */
176     .proc_write = module_input, /* "write" to the file */
177     .proc_open = module_open, /* called when the /proc file is opened */
178     .proc_release = module_close, /* called when it's closed */
179     .proc_lseek = noop_llseek, /* return file->f_pos */
180 };
181 #else
182 static const struct file_operations file_ops_4_our_proc_file = {
183     .read = module_output,
184     .write = module_input,
185     .open = module_open,
186     .release = module_close,
187     .llseek = noop_llseek,
188 };
189 #endif
190
191 /* Initialize the module - register the proc file */
192 static int __init sleep_init(void)
193 {
194     our_proc_file =
195         proc_create(PROC_ENTRY_FILENAME, 0644, NULL,
196             ↪ &file_ops_4_our_proc_file);
197     if (our_proc_file == NULL) {
198         remove_proc_entry(PROC_ENTRY_FILENAME, NULL);
199         pr_debug("Error: Could not initialize /proc/%s\n",
200             ↪ PROC_ENTRY_FILENAME);
201         return -ENOMEM;
202     }
203     proc_set_size(our_proc_file, 80);
204     proc_set_user(our_proc_file, GLOBAL_ROOT_UID, GLOBAL_ROOT_GID);
205
206     pr_info("/proc/%s created\n", PROC_ENTRY_FILENAME);
207
208     return 0;
209 }
210
211 /* Cleanup - unregister our file from /proc. This could get dangerous if
212  * there are still processes waiting in waitq, because they are inside our

```

```

211  * open function, which will get unloaded. I'll explain how to avoid removal
212  * of a kernel module in such a case in chapter 10.
213  */
214  static void __exit sleep_exit(void)
215  {
216      remove_proc_entry(PROC_ENTRY_FILENAME, NULL);
217      pr_debug("/proc/%s removed\n", PROC_ENTRY_FILENAME);
218  }
219
220  module_init(sleep_init);
221  module_exit(sleep_exit);
222
223  MODULE_LICENSE("GPL");

```

```

1  /*
2   * cat_nonblock.c - open a file and display its contents, but exit rather
↳  * than
3   * wait for input.
4   */
5  #include <errno.h> /* for errno */
6  #include <fcntl.h> /* for open */
7  #include <stdio.h> /* standard I/O */
8  #include <stdlib.h> /* for exit */
9  #include <unistd.h> /* for read */
10
11  #define MAX_BYTES 1024 * 4
12
13  int main(int argc, char *argv[])
14  {
15      int fd; /* The file descriptor for the file to read */
16      size_t bytes; /* The number of bytes read */
17      char buffer[MAX_BYTES]; /* The buffer for the bytes */
18
19      /* Usage */
20      if (argc != 2) {
21          printf("Usage: %s <filename>\n", argv[0]);
22          puts("Reads the content of a file, but doesn't wait for input");
23          exit(-1);
24      }
25
26      /* Open the file for reading in non blocking mode */
27      fd = open(argv[1], O_RDONLY | O_NONBLOCK);
28
29      /* If open failed */
30      if (fd == -1) {
31          puts(errno == EAGAIN ? "Open would block" : "Open failed");
32          exit(-1);
33      }
34
35      /* Read the file and output its contents */
36      do {
37          /* Read characters from the file */
38          bytes = read(fd, buffer, MAX_BYTES);
39
40          /* If there's an error, report it and die */
41          if (bytes == -1) {
42              if (errno == EAGAIN)

```

```

43         puts("Normally I'd block, but you told me not to");
44     else
45         puts("Another read error");
46         exit(-1);
47     }
48
49     /* Print the characters */
50     if (bytes > 0) {
51         for (int i = 0; i < bytes; i++)
52             putchar(buffer[i]);
53     }
54
55     /* While there are no errors and the file isn't over */
56 } while (bytes > 0);
57
58 return 0;
59 }

```

11.2 Completions

Sometimes one thing should happen before another within a module having multiple threads. Rather than using `/bin/sleep` commands, the kernel has another way to do this which allows timeouts or interrupts to also happen.

In the following example two threads are started, but one needs to start before another.

```

1  /*
2   * completions.c
3   */
4  #include <linux/completion.h>
5  #include <linux/err.h> /* for IS_ERR() */
6  #include <linux/init.h>
7  #include <linux/kthread.h>
8  #include <linux/module.h>
9  #include <linux/printk.h>
10 #include <linux/version.h>
11
12 static struct {
13     struct completion crank_comp;
14     struct completion flywheel_comp;
15 } machine;
16
17 static int machine_crank_thread(void *arg)
18 {
19     pr_info("Turn the crank\n");
20
21     complete_all(&machine.crank_comp);
22     #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 17, 0)
23     kthread_complete_and_exit(&machine.crank_comp, 0);
24     #else
25     complete_and_exit(&machine.crank_comp, 0);
26     #endif
27 }
28

```

```

29 static int machine_flywheel_spinup_thread(void *arg)
30 {
31     wait_for_completion(&machine.crank_comp);
32
33     pr_info("Flywheel spins up\n");
34
35     complete_all(&machine.flywheel_comp);
36 #if LINUX_VERSION_CODE >= KERNEL_VERSION(5, 17, 0)
37     kthread_complete_and_exit(&machine.flywheel_comp, 0);
38 #else
39     complete_and_exit(&machine.flywheel_comp, 0);
40 #endif
41 }
42
43 static int completions_init(void)
44 {
45     struct task_struct *crank_thread;
46     struct task_struct *flywheel_thread;
47
48     pr_info("completions example\n");
49
50     init_completion(&machine.crank_comp);
51     init_completion(&machine.flywheel_comp);
52
53     crank_thread = kthread_create(machine_crank_thread, NULL, "KThread
↳ Crank");
54     if (IS_ERR(crank_thread))
55         goto ERROR_THREAD_1;
56
57     flywheel_thread = kthread_create(machine_flywheel_spinup_thread, NULL,
58                                     "KThread Flywheel");
59     if (IS_ERR(flywheel_thread))
60         goto ERROR_THREAD_2;
61
62     wake_up_process(flywheel_thread);
63     wake_up_process(crank_thread);
64
65     return 0;
66
67 ERROR_THREAD_2:
68     kthread_stop(crank_thread);
69 ERROR_THREAD_1:
70
71     return -1;
72 }
73
74 static void completions_exit(void)
75 {
76     wait_for_completion(&machine.crank_comp);
77     wait_for_completion(&machine.flywheel_comp);
78
79     pr_info("completions exit\n");
80 }
81
82 module_init(completions_init);
83 module_exit(completions_exit);
84

```

```

85 MODULE_DESCRIPTION("Completions example");
86 MODULE_LICENSE("GPL");

```

The `machine` structure stores the completion states for the two threads. At the exit point of each thread the respective completion state is updated, and `wait_for_completion` is used by the flywheel thread to ensure that it does not begin prematurely.

So even though `flywheel_thread` is started first you should notice if you load this module and run `dmesg` that turning the crank always happens first because the flywheel thread waits for it to complete.

There are other variations upon the `wait_for_completion` function, which include timeouts or being interrupted, but this basic mechanism is enough for many common situations without adding a lot of complexity.

12 Avoiding Collisions and Deadlocks

If processes running on different CPUs or in different threads try to access the same memory, then it is possible that strange things can happen or your system can lock up. To avoid this, various types of mutual exclusion kernel functions are available. These indicate if a section of code is "locked" or "unlocked" so that simultaneous attempts to run it can not happen.

12.1 Mutex

You can use kernel mutexes (mutual exclusions) in much the same manner that you might deploy them in userland. This may be all that is needed to avoid collisions in most cases.

```

1  /*
2   * example_mutex.c
3   */
4  #include <linux/module.h>
5  #include <linux/mutex.h>
6  #include <linux/printk.h>
7
8  static DEFINE_MUTEX(mymutex);
9
10 static int example_mutex_init(void)
11 {
12     int ret;
13
14     pr_info("example_mutex init\n");
15
16     ret = mutex_trylock(&mymutex);
17     if (ret != 0) {
18         pr_info("mutex is locked\n");
19
20         if (mutex_is_locked(&mymutex) == 0)
21             pr_info("The mutex failed to lock!\n");

```

```

22         mutex_unlock(&mymutex);
23         pr_info("mutex is unlocked\n");
24     } else
25         pr_info("Failed to lock\n");
26
27     return 0;
28 }
29
30
31 static void example_mutex_exit(void)
32 {
33     pr_info("example_mutex exit\n");
34 }
35
36 module_init(example_mutex_init);
37 module_exit(example_mutex_exit);
38
39 MODULE_DESCRIPTION("Mutex example");
40 MODULE_LICENSE("GPL");

```

12.2 Spinlocks

As the name suggests, spinlocks lock up the CPU that the code is running on, taking 100% of its resources. Because of this you should only use the spinlock mechanism around code which is likely to take no more than a few milliseconds to run and so will not noticeably slow anything down from the user's point of view.

The example here is "irq safe" in that if interrupts happen during the lock then they will not be forgotten and will activate when the unlock happens, using the `flags` variable to retain their state.

```

1  /*
2   * example_spinlock.c
3   */
4  #include <linux/init.h>
5  #include <linux/module.h>
6  #include <linux/printk.h>
7  #include <linux/spinlock.h>
8
9  static DEFINE_SPINLOCK(sl_static);
10 static spinlock_t sl_dynamic;
11
12 static void example_spinlock_static(void)
13 {
14     unsigned long flags;
15
16     spin_lock_irqsave(&sl_static, flags);
17     pr_info("Locked static spinlock\n");
18
19     /* Do something or other safely. Because this uses 100% CPU time, this
20      * code should take no more than a few milliseconds to run.
21     */
22 }

```

```

23     spin_unlock_irqrestore(&sl_static, flags);
24     pr_info("Unlocked static spinlock\n");
25 }
26
27 static void example_spinlock_dynamic(void)
28 {
29     unsigned long flags;
30
31     spin_lock_init(&sl_dynamic);
32     spin_lock_irqsave(&sl_dynamic, flags);
33     pr_info("Locked dynamic spinlock\n");
34
35     /* Do something or other safely. Because this uses 100% CPU time, this
36      * code should take no more than a few milliseconds to run.
37      */
38
39     spin_unlock_irqrestore(&sl_dynamic, flags);
40     pr_info("Unlocked dynamic spinlock\n");
41 }
42
43 static int example_spinlock_init(void)
44 {
45     pr_info("example spinlock started\n");
46
47     example_spinlock_static();
48     example_spinlock_dynamic();
49
50     return 0;
51 }
52
53 static void example_spinlock_exit(void)
54 {
55     pr_info("example spinlock exit\n");
56 }
57
58 module_init(example_spinlock_init);
59 module_exit(example_spinlock_exit);
60
61 MODULE_DESCRIPTION("Spinlock example");
62 MODULE_LICENSE("GPL");

```

12.3 Read and write locks

Read and write locks are specialised kinds of spinlocks so that you can exclusively read from something or write to something. Like the earlier spinlocks example, the one below shows an "irq safe" situation in which if other functions were triggered from irqs which might also read and write to whatever you are concerned with then they would not disrupt the logic. As before it is a good idea to keep anything done within the lock as short as possible so that it does not hang up the system and cause users to start revolting against the tyranny of your module.

```

1  /*
2  * example_rwlock.c

```

```

3  */
4  #include <linux/module.h>
5  #include <linux/printk.h>
6  #include <linux/rwlock.h>
7
8  static DEFINE_RWLOCK(myrwlock);
9
10 static void example_read_lock(void)
11 {
12     unsigned long flags;
13
14     read_lock_irqsave(&myrwlock, flags);
15     pr_info("Read Locked\n");
16
17     /* Read from something */
18
19     read_unlock_irqrestore(&myrwlock, flags);
20     pr_info("Read Unlocked\n");
21 }
22
23 static void example_write_lock(void)
24 {
25     unsigned long flags;
26
27     write_lock_irqsave(&myrwlock, flags);
28     pr_info("Write Locked\n");
29
30     /* Write to something */
31
32     write_unlock_irqrestore(&myrwlock, flags);
33     pr_info("Write Unlocked\n");
34 }
35
36 static int example_rwlock_init(void)
37 {
38     pr_info("example_rwlock started\n");
39
40     example_read_lock();
41     example_write_lock();
42
43     return 0;
44 }
45
46 static void example_rwlock_exit(void)
47 {
48     pr_info("example_rwlock exit\n");
49 }
50
51 module_init(example_rwlock_init);
52 module_exit(example_rwlock_exit);
53
54 MODULE_DESCRIPTION("Read/Write locks example");
55 MODULE_LICENSE("GPL");

```

Of course, if you know for sure that there are no functions triggered by irqs which could possibly interfere with your logic then you can use the simpler

`read_lock(&myrwlock)` and `read_unlock(&myrwlock)` or the corresponding write functions.

12.4 Atomic operations

If you are doing simple arithmetic: adding, subtracting or bitwise operations, then there is another way in the multi-CPU and multi-hyperthreaded world to stop other parts of the system from messing with your mojo. By using atomic operations you can be confident that your addition, subtraction or bit flip did actually happen and was not overwritten by some other shenanigans. An example is shown below.

```
1  /*
2   * example_atomic.c
3   */
4  #include <linux/atomic.h>
5  #include <linux/bitops.h>
6  #include <linux/module.h>
7  #include <linux/printk.h>
8
9  #define BYTE_TO_BINARY_PATTERN "%c%c%c%c%c%c%c%c"
10 #define BYTE_TO_BINARY(byte)
11     \
12     ((byte & 0x80) ? '1' : '0'), ((byte & 0x40) ? '1' : '0'),
13     \
14     ((byte & 0x20) ? '1' : '0'), ((byte & 0x10) ? '1' : '0'),
15     \
16     ((byte & 0x08) ? '1' : '0'), ((byte & 0x04) ? '1' : '0'),
17     \
18     ((byte & 0x02) ? '1' : '0'), ((byte & 0x01) ? '1' : '0')
19
20 static void atomic_add_subtract(void)
21 {
22     atomic_t debbie;
23     atomic_t chris = ATOMIC_INIT(50);
24
25     atomic_set(&debbie, 45);
26
27     /* subtract one */
28     atomic_dec(&debbie);
29
30     atomic_add(7, &debbie);
31
32     /* add one */
33     atomic_inc(&debbie);
34
35     pr_info("chris: %d, debbie: %d\n", atomic_read(&chris),
36             atomic_read(&debbie));
37 }
38
39 static void atomic_bitwise(void)
40 {
41     unsigned long word = 0;
42
43     pr_info("Bits 0: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
```

```

40     set_bit(3, &word);
41     set_bit(5, &word);
42     pr_info("Bits 1: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
43     clear_bit(5, &word);
44     pr_info("Bits 2: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
45     change_bit(3, &word);
46
47     pr_info("Bits 3: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
48     if (test_and_set_bit(3, &word))
49         pr_info("wrong\n");
50     pr_info("Bits 4: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
51
52     word = 255;
53     pr_info("Bits 5: " BYTE_TO_BINARY_PATTERN, BYTE_TO_BINARY(word));
54 }
55
56 static int example_atomic_init(void)
57 {
58     pr_info("example_atomic started\n");
59
60     atomic_add_subtract();
61     atomic_bitwise();
62
63     return 0;
64 }
65
66 static void example_atomic_exit(void)
67 {
68     pr_info("example_atomic exit\n");
69 }
70
71 module_init(example_atomic_init);
72 module_exit(example_atomic_exit);
73
74 MODULE_DESCRIPTION("Atomic operations example");
75 MODULE_LICENSE("GPL");

```

Before the C11 standard adopts the built-in atomic types, the kernel already provided a small set of atomic types by using a bunch of tricky architecture-specific codes. Implementing the atomic types by C11 atomics may allow the kernel to throw away the architecture-specific codes and letting the kernel code be more friendly to the people who understand the standard. But there are some problems, such as the memory model of the kernel doesn't match the model formed by the C11 atomics. For further details, see:

- [kernel documentation of atomic types](#)
- [Time to move to C11 atomics?](#)
- [Atomic usage patterns in the kernel](#)

13 Replacing Print Macros

13.1 Replacement

In Section 2, I said that X Window System and kernel module programming do not mix. That is true for developing kernel modules. But in actual use, you want to be able to send messages to whichever tty the command to load the module came from.

"tty" is an abbreviation of *teletype*: originally a combination keyboard-printer used to communicate with a Unix system, and today an abstraction for the text stream used for a Unix program, whether it is a physical terminal, an xterm on an X display, a network connection used with ssh, etc.

The way this is done is by using current, a pointer to the currently running task, to get the current task's tty structure. Then, we look inside that tty structure to find a pointer to a string write function, which we use to write a string to the tty.

```
1  /*
2  * print_string.c - Send output to the tty we're running on, regardless if
3  * it is through X11, telnet, etc. We do this by printing the string to the
4  * tty associated with the current task.
5  */
6  #include <linux/init.h>
7  #include <linux/kernel.h>
8  #include <linux/module.h>
9  #include <linux/sched.h> /* For current */
10 #include <linux/tty.h> /* For the tty declarations */
11
12 static void print_string(char *str)
13 {
14     /* The tty for the current task */
15     struct tty_struct *my_tty = get_current_tty();
16
17     /* If my_tty is NULL, the current task has no tty you can print to (i.e.,
18      * if it is a daemon). If so, there is nothing we can do.
19      */
20     if (my_tty) {
21         const struct tty_operations *ttyops = my_tty->driver->ops;
22         /* my_tty->driver is a struct which holds the tty's functions,
23          * one of which (write) is used to write strings to the tty.
24          * It can be used to take a string either from the user's or
25          * kernel's memory segment.
26          *
27          * The function's 1st parameter is the tty to write to, because the
28          * same function would normally be used for all tty's of a certain
29          * type.
30          * The 2nd parameter is a pointer to a string.
31          * The 3rd parameter is the length of the string.
32          *
33          * As you will see below, sometimes it's necessary to use
34          * preprocessor stuff to create code that works for different
35          * kernel versions. The (naive) approach we've taken here does not
36          * scale well. The right way to deal with this is described in
37          * section 2 of
```

```

38         * linux/Documentation/SubmittingPatches
39         */
40         (ttyops->write)(my_tty, /* The tty itself */
41                        str, /* String */
42                        strlen(str)); /* Length */
43
44         /* ttys were originally hardware devices, which (usually) strictly
45         * followed the ASCII standard. In ASCII, to move to a new line you
46         * need two characters, a carriage return and a line feed. On Unix,
47         * the ASCII line feed is used for both purposes - so we can not
48         * just use \n, because it would not have a carriage return and the
49         * next line will start at the column right after the line feed.
50         *
51         * This is why text files are different between Unix and MS Windows.
52         * In CP/M and derivatives, like MS-DOS and MS Windows, the ASCII
53         * standard was strictly adhered to, and therefore a newline requires
54         * both a LF and a CR.
55         */
56         (ttyops->write)(my_tty, "\015\012", 2);
57     }
58 }
59
60 static int __init print_string_init(void)
61 {
62     print_string("The module has been inserted. Hello world!");
63     return 0;
64 }
65
66 static void __exit print_string_exit(void)
67 {
68     print_string("The module has been removed. Farewell world!");
69 }
70
71 module_init(print_string_init);
72 module_exit(print_string_exit);
73
74 MODULE_LICENSE("GPL");

```

13.2 Flashing keyboard LEDs

In certain conditions, you may desire a simpler and more direct way to communicate to the external world. Flashing keyboard LEDs can be such a solution: It is an immediate way to attract attention or to display a status condition. Keyboard LEDs are present on every hardware, they are always visible, they do not need any setup, and their use is rather simple and non-intrusive, compared to writing to a tty or a file.

From v4.14 to v4.15, the timer API made a series of changes to improve memory safety. A buffer overflow in the area of a `timer_list` structure may be able to overwrite the `function` and `data` fields, providing the attacker with a way to use return-object programming (ROP) to call arbitrary functions within the kernel. Also, the function prototype of the callback, containing a `unsigned long` argument, will prevent work from any type checking. Furthermore, the function

prototype with `unsigned long` argument may be an obstacle to the forward-edge protection of *control-flow integrity*. Thus, it is better to use a unique prototype to separate from the cluster that takes an `unsigned long` argument. The timer callback should be passed a pointer to the `timer_list` structure rather than an `unsigned long` argument. Then, it wraps all the information the callback needs, including the `timer_list` structure, into a larger structure, and it can use the `container_of` macro instead of the `unsigned long` value. For more information see: [Improving the kernel timers API](#).

Before Linux v4.14, `setup_timer` was used to initialize the timer and the `timer_list` structure looked like:

```

1 struct timer_list {
2     unsigned long expires;
3     void (*function)(unsigned long);
4     unsigned long data;
5     u32 flags;
6     /* ... */
7 };
8
9 void setup_timer(struct timer_list *timer, void (*callback)(unsigned long),
10                unsigned long data);

```

Since Linux v4.14, `timer_setup` is adopted and the kernel step by step converting to `timer_setup` from `setup_timer`. One of the reasons why API was changed is it need to coexist with the old version interface. Moreover, the `timer_setup` was implemented by `setup_timer` at first.

```

1 void timer_setup(struct timer_list *timer,
2                 void (*callback)(struct timer_list *), unsigned int flags);

```

The `setup_timer` was then removed since v4.15. As a result, the `timer_list` structure had changed to the following.

```

1 struct timer_list {
2     unsigned long expires;
3     void (*function)(struct timer_list *);
4     u32 flags;
5     /* ... */
6 };

```

The following source code illustrates a minimal kernel module which, when loaded, starts blinking the keyboard LEDs until it is unloaded.

```

1 /*
2  * kbleds.c - Blink keyboard leds until the module is unloaded.
3  */
4
5 #include <linux/init.h>
6 #include <linux/kd.h> /* For KDSETLED */

```

```

7  #include <linux/module.h>
8  #include <linux/tty.h> /* For tty_struct */
9  #include <linux/vt.h> /* For MAX_NR_CONSOLES */
10 #include <linux/vt_kern.h> /* for fg_console */
11 #include <linux/console_struct.h> /* For vc_cons */
12
13 MODULE_DESCRIPTION("Example module illustrating the use of Keyboard LEDs.");
14
15 static struct timer_list my_timer;
16 static struct tty_driver *my_driver;
17 static unsigned long kbledstatus = 0;
18
19 #define BLINK_DELAY HZ / 5
20 #define ALL_LEDS_ON 0x07
21 #define RESTORE_LEDS 0xFF
22
23 /* Function my_timer_func blinks the keyboard LEDs periodically by invoking
24  * command KDSETLED of ioctl() on the keyboard driver. To learn more on
25  * ↪ virtual
26  * terminal ioctl operations, please see file:
27  * drivers/tty/vt/vt_ioctl.c, function vt_ioctl().
28  *
29  * The argument to KDSETLED is alternatively set to 7 (thus causing the led
30  * mode to be set to LED_SHOW_IOCTL, and all the leds are lit) and to 0xFF
31  * (any value above 7 switches back the led mode to LED_SHOW_FLAGS, thus
32  * the LEDs reflect the actual keyboard status). To learn more on this,
33  * please see file: drivers/tty/vt/keyboard.c, function settledstate().
34  */
35 static void my_timer_func(struct timer_list *unused)
36 {
37     struct tty_struct *t = vc_cons[fg_console].d->port.tty;
38
39     if (kbledstatus == ALL_LEDS_ON)
40         kbledstatus = RESTORE_LEDS;
41     else
42         kbledstatus = ALL_LEDS_ON;
43
44     (my_driver->ops->ioctl)(t, KDSETLED, kbledstatus);
45
46     my_timer.expires = jiffies + BLINK_DELAY;
47     add_timer(&my_timer);
48 }
49
50 static int __init kbleds_init(void)
51 {
52     int i;
53
54     pr_info("kbleds: loading\n");
55     pr_info("kbleds: fgconsole is %x\n", fg_console);
56     for (i = 0; i < MAX_NR_CONSOLES; i++) {
57         if (!vc_cons[i].d)
58             break;
59         pr_info("poet_atkm: console[%i/%i] #%i, tty %p\n", i, MAX_NR_CONSOLES,
60             vc_cons[i].d->vc_num, (void *)vc_cons[i].d->port.tty);
61     }
62     pr_info("kbleds: finished scanning consoles\n");

```

```

63     my_driver = vc_cons[fg_console].d->port.tty->driver;
64     pr_info("kbleds: tty driver magic %x\n", my_driver->magic);
65
66     /* Set up the LED blink timer the first time. */
67     timer_setup(&my_timer, my_timer_func, 0);
68     my_timer.expires = jiffies + BLINK_DELAY;
69     add_timer(&my_timer);
70
71     return 0;
72 }
73
74 static void __exit kbleds_cleanup(void)
75 {
76     pr_info("kbleds: unloading...\n");
77     del_timer(&my_timer);
78     (my_driver->ops->iocctl)(vc_cons[fg_console].d->port.tty, KDSETLED,
79                             RESTORE_LEDS);
80 }
81
82 module_init(kbleds_init);
83 module_exit(kbleds_cleanup);
84
85 MODULE_LICENSE("GPL");

```

If none of the examples in this chapter fit your debugging needs, there might yet be some other tricks to try. Ever wondered what `CONFIG_LL_DEBUG` in `make menuconfig` is good for? If you activate that you get low level access to the serial port. While this might not sound very powerful by itself, you can patch [kernel/printk.c](#) or any other essential syscall to print ASCII characters, thus making it possible to trace virtually everything what your code does over a serial line. If you find yourself porting the kernel to some new and former unsupported architecture, this is usually amongst the first things that should be implemented. Logging over a netconsole might also be worth a try.

While you have seen lots of stuff that can be used to aid debugging here, there are some things to be aware of. Debugging is almost always intrusive. Adding debug code can change the situation enough to make the bug seem to disappear. Thus, you should keep debug code to a minimum and make sure it does not show up in production code.

14 Scheduling Tasks

There are two main ways of running tasks: tasklets and work queues. Tasklets are a quick and easy way of scheduling a single function to be run. For example, when triggered from an interrupt, whereas work queues are more complicated but also better suited to running multiple things in a sequence.

14.1 Tasklets

Here is an example tasklet module. The `tasklet_fn` function runs for a few seconds. In the meantime, execution of the `example_tasklet_init` function may

continue to the exit point, depending on whether it is interrupted by **softirq**.

```
1  /*
2   * example_tasklet.c
3   */
4  #include <linux/delay.h>
5  #include <linux/interrupt.h>
6  #include <linux/module.h>
7  #include <linux/ printk.h>
8
9  /* Macro DECLARE_TASKLET_OLD exists for compatibility.
10   * See https://lwn.net/Articles/830964/
11   */
12  #ifndef DECLARE_TASKLET_OLD
13  #define DECLARE_TASKLET_OLD(arg1, arg2) DECLARE_TASKLET(arg1, arg2, 0L)
14  #endif
15
16  static void tasklet_fn(unsigned long data)
17  {
18      pr_info("Example tasklet starts\n");
19      mdelay(5000);
20      pr_info("Example tasklet ends\n");
21  }
22
23  static DECLARE_TASKLET_OLD(mytask, tasklet_fn);
24
25  static int example_tasklet_init(void)
26  {
27      pr_info("tasklet example init\n");
28      tasklet_schedule(&mytask);
29      mdelay(200);
30      pr_info("Example tasklet init continues...\n");
31      return 0;
32  }
33
34  static void example_tasklet_exit(void)
35  {
36      pr_info("tasklet example exit\n");
37      tasklet_kill(&mytask);
38  }
39
40  module_init(example_tasklet_init);
41  module_exit(example_tasklet_exit);
42
43  MODULE_DESCRIPTION("Tasklet example");
44  MODULE_LICENSE("GPL");
```

So with this example loaded `dmesg` should show:

```
tasklet example init
Example tasklet starts
Example tasklet init continues...
Example tasklet ends
```

Although tasklet is easy to use, it comes with several defactors, and developers are discussing about getting rid of tasklet in linux kernel. The tasklet callback

runs in atomic context, inside a software interrupt, meaning that it cannot sleep or access user-space data, so not all work can be done in a tasklet handler. Also, the kernel only allows one instance of any given tasklet to be running at any given time; multiple different tasklet callbacks can run in parallel.

In recent kernels, tasklets can be replaced by workqueues, timers, or threaded interrupts.¹ While the removal of tasklets remains a longer-term goal, the current kernel contains more than a hundred uses of tasklets. Now developers are proceeding with the API changes and the macro `DECLARE_TASKLET_OLD` exists for compatibility. For further information, see <https://lwn.net/Articles/830964/>.

14.2 Work queues

To add a task to the scheduler we can use a workqueue. The kernel then uses the Completely Fair Scheduler (CFS) to execute work within the queue.

```
1  /*
2   * sched.c
3   */
4  #include <linux/init.h>
5  #include <linux/module.h>
6  #include <linux/workqueue.h>
7
8  static struct workqueue_struct *queue = NULL;
9  static struct work_struct work;
10
11 static void work_handler(struct work_struct *data)
12 {
13     pr_info("work handler function.\n");
14 }
15
16 static int __init sched_init(void)
17 {
18     queue = alloc_workqueue("HELLOWORLD", WQ_UNBOUND, 1);
19     INIT_WORK(&work, work_handler);
20     schedule_work(&work);
21     return 0;
22 }
23
24 static void __exit sched_exit(void)
25 {
26     destroy_workqueue(queue);
27 }
28
29 module_init(sched_init);
30 module_exit(sched_exit);
31
```

¹The goal of threaded interrupts is to push more of the work to separate threads, so that the minimum needed for acknowledging an interrupt is reduced, and therefore the time spent handling the interrupt (where it can't handle any other interrupts at the same time) is reduced. See <https://lwn.net/Articles/302043/>.

```
32 MODULE_LICENSE("GPL");  
33 MODULE_DESCRIPTION("Workqueue example");
```

15 Interrupt Handlers

15.1 Interrupt Handlers

Except for the last chapter, everything we did in the kernel so far we have done as a response to a process asking for it, either by dealing with a special file, sending an `ioctl()`, or issuing a system call. But the job of the kernel is not just to respond to process requests. Another job, which is every bit as important, is to speak to the hardware connected to the machine.

There are two types of interaction between the CPU and the rest of the computer's hardware. The first type is when the CPU gives orders to the hardware, the other is when the hardware needs to tell the CPU something. The second, called interrupts, is much harder to implement because it has to be dealt with when convenient for the hardware, not the CPU. Hardware devices typically have a very small amount of RAM, and if you do not read their information when available, it is lost.

Under Linux, hardware interrupts are called IRQ's (Interrupt ReQuests). There are two types of IRQ's, short and long. A short IRQ is one which is expected to take a very short period of time, during which the rest of the machine will be blocked and no other interrupts will be handled. A long IRQ is one which can take longer, and during which other interrupts may occur (but not interrupts from the same device). If at all possible, it is better to declare an interrupt handler to be long.

When the CPU receives an interrupt, it stops whatever it is doing (unless it is processing a more important interrupt, in which case it will deal with this one only when the more important one is done), saves certain parameters on the stack and calls the interrupt handler. This means that certain things are not allowed in the interrupt handler itself, because the system is in an unknown state. Linux kernel solves the problem by splitting interrupt handling into two parts. The first part executes right away and masks the interrupt line. Hardware interrupts must be handled quickly, and that is why we need the second part to handle the heavy work deferred from an interrupt handler. Historically, BH (Linux naming for *Bottom Halves*) statistically book-keeps the deferred functions. **Softirq** and its higher level abstraction, **Tasklet**, replace BH since Linux 2.3.

The way to implement this is to call `request_irq()` to get your interrupt handler called when the relevant IRQ is received.

In practice IRQ handling can be a bit more complex. Hardware is often designed in a way that chains two interrupt controllers, so that all the IRQs from interrupt controller B are cascaded to a certain IRQ from interrupt controller A. Of course, that requires that the kernel finds out which IRQ it really was afterwards and that adds overhead. Other architectures offer some special,

very low overhead, so called "fast IRQ" or FIQs. To take advantage of them requires handlers to be written in assembly language, so they do not really fit into the kernel. They can be made to work similar to the others, but after that procedure, they are no longer any faster than "common" IRQs. SMP enabled kernels running on systems with more than one processor need to solve another truckload of problems. It is not enough to know if a certain IRQs has happened, it's also important to know what CPU(s) it was for. People still interested in more details, might want to refer to "APIC" now.

This function receives the IRQ number, the name of the function, flags, a name for `/proc/interrupts` and a parameter to be passed to the interrupt handler. Usually there is a certain number of IRQs available. How many IRQs there are is hardware-dependent. The flags can include `SA_SHIRQ` to indicate you are willing to share the IRQ with other interrupt handlers (usually because a number of hardware devices sit on the same IRQ) and `SA_INTERRUPT` to indicate this is a fast interrupt. This function will only succeed if there is not already a handler on this IRQ, or if you are both willing to share.

15.2 Detecting button presses

Many popular single board computers, such as Raspberry Pi or Beagleboards, have a bunch of GPIO pins. Attaching buttons to those and then having a button press do something is a classic case in which you might need to use interrupts, so that instead of having the CPU waste time and battery power polling for a change in input state, it is better for the input to trigger the CPU to then run a particular handling function.

Here is an example where buttons are connected to GPIO numbers 17 and 18 and an LED is connected to GPIO 4. You can change those numbers to whatever is appropriate for your board.

```
1  /*
2   * intrpt.c - Handling GPIO with interrupts
3   *
4   * Based upon the RPi example by Stefan Wendler (devnull@kaltpost.de)
5   * from:
6   *   https://github.com/wendlers/rpi-kmod-samples
7   *
8   * Press one button to turn on a LED and another to turn it off.
9   */
10
11 #include <linux/gpio.h>
12 #include <linux/interrupt.h>
13 #include <linux/kernel.h> /* for ARRAY_SIZE() */
14 #include <linux/module.h>
15 #include <linux/printk.h>
16
17 static int button_irqs[] = { -1, -1 };
18
19 /* Define GPIOs for LEDs.
20  * TODO: Change the numbers for the GPIO on your board.
21  */
```

```

22 static struct gpio leds[] = { { 4, GPIOF_OUT_INIT_LOW, "LED 1" } };
23
24 /* Define GPIOs for BUTTONS
25  * TODO: Change the numbers for the GPIO on your board.
26  */
27 static struct gpio buttons[] = { { 17, GPIOF_IN, "LED 1 ON BUTTON" },
28                                   { 18, GPIOF_IN, "LED 1 OFF BUTTON" } };
29
30 /* interrupt function triggered when a button is pressed. */
31 static irqreturn_t button_isr(int irq, void *data)
32 {
33     /* first button */
34     if (irq == button_irqs[0] && !gpio_get_value(leds[0].gpio))
35         gpio_set_value(leds[0].gpio, 1);
36     /* second button */
37     else if (irq == button_irqs[1] && gpio_get_value(leds[0].gpio))
38         gpio_set_value(leds[0].gpio, 0);
39
40     return IRQ_HANDLED;
41 }
42
43 static int __init intrpt_init(void)
44 {
45     int ret = 0;
46
47     pr_info("%s\n", __func__);
48
49     /* register LED gpios */
50     ret = gpio_request_array(leds, ARRAY_SIZE(leds));
51
52     if (ret) {
53         pr_err("Unable to request GPIOs for LEDs: %d\n", ret);
54         return ret;
55     }
56
57     /* register BUTTON gpios */
58     ret = gpio_request_array(buttons, ARRAY_SIZE(buttons));
59
60     if (ret) {
61         pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
62         goto fail1;
63     }
64
65     pr_info("Current button1 value: %d\n", gpio_get_value(buttons[0].gpio));
66
67     ret = gpio_to_irq(buttons[0].gpio);
68
69     if (ret < 0) {
70         pr_err("Unable to request IRQ: %d\n", ret);
71         goto fail2;
72     }
73
74     button_irqs[0] = ret;
75
76     pr_info("Successfully requested BUTTON1 IRQ # %d\n", button_irqs[0]);
77
78     ret = request_irq(button_irqs[0], button_isr,

```

```

79             IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
80             "gpiomod#button1", NULL);
81
82     if (ret) {
83         pr_err("Unable to request IRQ: %d\n", ret);
84         goto fail2;
85     }
86
87     ret = gpio_to_irq(buttons[1].gpio);
88
89     if (ret < 0) {
90         pr_err("Unable to request IRQ: %d\n", ret);
91         goto fail2;
92     }
93
94     button_irqs[1] = ret;
95
96     pr_info("Successfully requested BUTTON2 IRQ # %d\n", button_irqs[1]);
97
98     ret = request_irq(button_irqs[1], button_isr,
99                     IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
100                    "gpiomod#button2", NULL);
101
102     if (ret) {
103         pr_err("Unable to request IRQ: %d\n", ret);
104         goto fail3;
105     }
106
107     return 0;
108
109     /* cleanup what has been setup so far */
110 fail3:
111     free_irq(button_irqs[0], NULL);
112
113 fail2:
114     gpio_free_array(buttons, ARRAY_SIZE(leds));
115
116 fail1:
117     gpio_free_array(leds, ARRAY_SIZE(leds));
118
119     return ret;
120 }
121
122 static void __exit intrpt_exit(void)
123 {
124     int i;
125
126     pr_info("%s\n", __func__);
127
128     /* free irqs */
129     free_irq(button_irqs[0], NULL);
130     free_irq(button_irqs[1], NULL);
131
132     /* turn all LEDs off */
133     for (i = 0; i < ARRAY_SIZE(leds); i++)
134         gpio_set_value(leds[i].gpio, 0);
135

```

```

136     /* unregister */
137     gpio_free_array(leds, ARRAY_SIZE(leds));
138     gpio_free_array(buttons, ARRAY_SIZE(buttons));
139 }
140
141 module_init(intrpt_init);
142 module_exit(intrpt_exit);
143
144 MODULE_LICENSE("GPL");
145 MODULE_DESCRIPTION("Handle some GPIO interrupts");

```

15.3 Bottom Half

Suppose you want to do a bunch of stuff inside of an interrupt routine. A common way to do that without rendering the interrupt unavailable for a significant duration is to combine it with a tasklet. This pushes the bulk of the work off into the scheduler.

The example below modifies the previous example to also run an additional task when an interrupt is triggered.

```

1  /*
2   * bottomhalf.c - Top and bottom half interrupt handling
3   *
4   * Based upon the RPi example by Stefan Wendler (devnull@kaltpost.de)
5   * from:
6   *   https://github.com/wendlers/rpi-kmod-samples
7   *
8   * Press one button to turn on an LED and another to turn it off
9   */
10
11 #include <linux/delay.h>
12 #include <linux/gpio.h>
13 #include <linux/interrupt.h>
14 #include <linux/module.h>
15 #include <linux/printk.h>
16
17 /* Macro DECLARE_TASKLET_OLD exists for compatibility.
18  * See https://lwn.net/Articles/830964/
19  */
20 #ifndef DECLARE_TASKLET_OLD
21 #define DECLARE_TASKLET_OLD(arg1, arg2) DECLARE_TASKLET(arg1, arg2, 0L)
22 #endif
23
24 static int button_irqs[] = { -1, -1 };
25
26 /* Define GPIOs for LEDs.
27  * TODO: Change the numbers for the GPIO on your board.
28  */
29 static struct gpio leds[] = { { 4, GPIOF_OUT_INIT_LOW, "LED 1" } };
30
31 /* Define GPIOs for BUTTONS
32  * TODO: Change the numbers for the GPIO on your board.
33  */
34 static struct gpio buttons[] = {

```

```

35     { 17, GPIOF_IN, "LED 1 ON BUTTON" },
36     { 18, GPIOF_IN, "LED 1 OFF BUTTON" },
37 };
38
39 /* Tasklet containing some non-trivial amount of processing */
40 static void bottomhalf_tasklet_fn(unsigned long data)
41 {
42     pr_info("Bottom half tasklet starts\n");
43     /* do something which takes a while */
44     mdelay(500);
45     pr_info("Bottom half tasklet ends\n");
46 }
47
48 static DECLARE_TASKLET_OLD(buttontask, bottomhalf_tasklet_fn);
49
50 /* interrupt function triggered when a button is pressed */
51 static irqreturn_t button_isr(int irq, void *data)
52 {
53     /* Do something quickly right now */
54     if (irq == button_irqs[0] && !gpio_get_value(leds[0].gpio))
55         gpio_set_value(leds[0].gpio, 1);
56     else if (irq == button_irqs[1] && gpio_get_value(leds[0].gpio))
57         gpio_set_value(leds[0].gpio, 0);
58
59     /* Do the rest at leisure via the scheduler */
60     tasklet_schedule(&buttontask);
61
62     return IRQ_HANDLED;
63 }
64
65 static int __init bottomhalf_init(void)
66 {
67     int ret = 0;
68
69     pr_info("%s\n", __func__);
70
71     /* register LED gpios */
72     ret = gpio_request_array(leds, ARRAY_SIZE(leds));
73
74     if (ret) {
75         pr_err("Unable to request GPIOs for LEDs: %d\n", ret);
76         return ret;
77     }
78
79     /* register BUTTON gpios */
80     ret = gpio_request_array(buttons, ARRAY_SIZE(buttons));
81
82     if (ret) {
83         pr_err("Unable to request GPIOs for BUTTONs: %d\n", ret);
84         goto fail1;
85     }
86
87     pr_info("Current button1 value: %d\n", gpio_get_value(buttons[0].gpio));
88
89     ret = gpio_to_irq(buttons[0].gpio);
90
91     if (ret < 0) {

```

```

92         pr_err("Unable to request IRQ: %d\n", ret);
93         goto fail2;
94     }
95
96     button_irqs[0] = ret;
97
98     pr_info("Successfully requested BUTTON1 IRQ # %d\n", button_irqs[0]);
99
100    ret = request_irq(button_irqs[0], button_isr,
101                      IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
102                      "gpiomod#button1", NULL);
103
104    if (ret) {
105        pr_err("Unable to request IRQ: %d\n", ret);
106        goto fail2;
107    }
108
109    ret = gpio_to_irq(buttons[1].gpio);
110
111    if (ret < 0) {
112        pr_err("Unable to request IRQ: %d\n", ret);
113        goto fail2;
114    }
115
116    button_irqs[1] = ret;
117
118    pr_info("Successfully requested BUTTON2 IRQ # %d\n", button_irqs[1]);
119
120    ret = request_irq(button_irqs[1], button_isr,
121                      IRQF_TRIGGER_RISING | IRQF_TRIGGER_FALLING,
122                      "gpiomod#button2", NULL);
123
124    if (ret) {
125        pr_err("Unable to request IRQ: %d\n", ret);
126        goto fail3;
127    }
128
129    return 0;
130
131    /* cleanup what has been setup so far */
132 fail3:
133     free_irq(button_irqs[0], NULL);
134
135 fail2:
136     gpio_free_array(buttons, ARRAY_SIZE(leds));
137
138 fail1:
139     gpio_free_array(leds, ARRAY_SIZE(leds));
140
141     return ret;
142 }
143
144 static void __exit bottomhalf_exit(void)
145 {
146     int i;
147
148     pr_info("%s\n", __func__);

```

```

149
150     /* free irqs */
151     free_irq(button_irqs[0], NULL);
152     free_irq(button_irqs[1], NULL);
153
154     /* turn all LEDs off */
155     for (i = 0; i < ARRAY_SIZE(leds); i++)
156         gpio_set_value(leds[i].gpio, 0);
157
158     /* unregister */
159     gpio_free_array(leds, ARRAY_SIZE(leds));
160     gpio_free_array(buttons, ARRAY_SIZE(buttons));
161 }
162
163 module_init(bottomhalf_init);
164 module_exit(bottomhalf_exit);
165
166 MODULE_LICENSE("GPL");
167 MODULE_DESCRIPTION("Interrupt with top and bottom half");

```

16 Crypto

At the dawn of the internet, everybody trusted everybody completely...but that did not work out so well. When this guide was originally written, it was a more innocent era in which almost nobody actually gave a damn about crypto - least of all kernel developers. That is certainly no longer the case now. To handle crypto stuff, the kernel has its own API enabling common methods of encryption, decryption and your favourite hash functions.

16.1 Hash functions

Calculating and checking the hashes of things is a common operation. Here is a demonstration of how to calculate a sha256 hash within a kernel module. To provide the sha256 algorithm support, make sure CONFIG_CRYPTO_SHA256 is enabled in kernel.

```

1  /*
2   * cryptosha256.c
3   */
4  #include <crypto/internal/hash.h>
5  #include <linux/module.h>
6
7  #define SHA256_LENGTH 32
8
9  static void show_hash_result(char *plaintext, char *hash_sha256)
10 {
11     int i;
12     char str[SHA256_LENGTH * 2 + 1];
13
14     pr_info("sha256 test for string: \"%s\"\n", plaintext);
15     for (i = 0; i < SHA256_LENGTH; i++)

```

```

16         sprintf(&str[i * 2], "%02x", (unsigned char)hash_sha256[i]);
17         str[i * 2] = 0;
18         pr_info("%s\n", str);
19     }
20
21     static int cryptosha256_init(void)
22     {
23         char *plaintext = "This is a test";
24         char hash_sha256[SHA256_LENGTH];
25         struct crypto_shash *sha256;
26         struct shash_desc *shash;
27
28         sha256 = crypto_alloc_shash("sha256", 0, 0);
29         if (IS_ERR(sha256)) {
30             pr_err(
31                 "%s(): Failed to allocate sha256 algorithm, enable
                 ↳ CONFIG_CRYPTOA_SHA256 and try again.\n",
32                 __func__);
33             return -1;
34         }
35
36         shash = kmalloc(sizeof(struct shash_desc) + crypto_shash_descsize(sha256),
37                         GFP_KERNEL);
38         if (!shash)
39             return -ENOMEM;
40
41         shash->tfm = sha256;
42
43         if (crypto_shash_init(shash))
44             return -1;
45
46         if (crypto_shash_update(shash, plaintext, strlen(plaintext)))
47             return -1;
48
49         if (crypto_shash_final(shash, hash_sha256))
50             return -1;
51
52         kfree(shash);
53         crypto_free_shash(sha256);
54
55         show_hash_result(plaintext, hash_sha256);
56
57         return 0;
58     }
59
60     static void cryptosha256_exit(void)
61     {
62     }
63
64     module_init(cryptosha256_init);
65     module_exit(cryptosha256_exit);
66
67     MODULE_DESCRIPTION("sha256 hash test");
68     MODULE_LICENSE("GPL");

```

Install the module:

```
1 sudo insmod cryptosha256.ko
2 sudo dmesg
```

And you should see that the hash was calculated for the test string.
Finally, remove the test module:

```
1 sudo rmmod cryptosha256
```

16.2 Symmetric key encryption

Here is an example of symmetrically encrypting a string using the AES algorithm and a password.

```
1  /*
2   * cryptosk.c
3   */
4  #include <crypto/internal/skcipher.h>
5  #include <linux/crypto.h>
6  #include <linux/module.h>
7  #include <linux/random.h>
8  #include <linux/scatterlist.h>
9
10 #define SYMMETRIC_KEY_LENGTH 32
11 #define CIPHER_BLOCK_SIZE 16
12
13 struct tcrypt_result {
14     struct completion completion;
15     int err;
16 };
17
18 struct skcipher_def {
19     struct scatterlist sg;
20     struct crypto_skcipher *tfm;
21     struct skcipher_request *req;
22     struct tcrypt_result result;
23     char *scratchpad;
24     char *ciphertext;
25     char *ivdata;
26 };
27
28 static struct skcipher_def sk;
29
30 static void test_skcipher_finish(struct skcipher_def *sk)
31 {
32     if (sk->tfm)
33         crypto_free_skcipher(sk->tfm);
34     if (sk->req)
35         skcipher_request_free(sk->req);
36     if (sk->ivdata)
37         kfree(sk->ivdata);
```

```

38     if (sk->scratchpad)
39         kfree(sk->scratchpad);
40     if (sk->ciphertext)
41         kfree(sk->ciphertext);
42 }
43
44 static int test_skcipher_result(struct skcipher_def *sk, int rc)
45 {
46     switch (rc) {
47     case 0:
48         break;
49     case -EINPROGRESS || -EBUSY:
50         rc = wait_for_completion_interruptible(&sk->result.completion);
51         if (!rc && !sk->result.err) {
52             reinit_completion(&sk->result.completion);
53             break;
54         }
55         pr_info("skcipher encrypt returned with %d result %d\n", rc,
56               sk->result.err);
57         break;
58     default:
59         pr_info("skcipher encrypt returned with %d result %d\n", rc,
60               sk->result.err);
61         break;
62     }
63
64     init_completion(&sk->result.completion);
65
66     return rc;
67 }
68
69 static void test_skcipher_callback(struct crypto_async_request *req, int
↳ error)
70 {
71     struct tcrypt_result *result = req->data;
72
73     if (error == -EINPROGRESS)
74         return;
75
76     result->err = error;
77     complete(&result->completion);
78     pr_info("Encryption finished successfully\n");
79
80     /* decrypt data */
81     #if 0
82     memset((void*)sk.scratchpad, '-', CIPHER_BLOCK_SIZE);
83     ret = crypto_skcipher_decrypt(sk.req);
84     ret = test_skcipher_result(&sk, ret);
85     if (ret)
86         return;
87
88     sg_copy_from_buffer(&sk.sg, 1, sk.scratchpad, CIPHER_BLOCK_SIZE);
89     sk.scratchpad[CIPHER_BLOCK_SIZE-1] = 0;
90
91     pr_info("Decryption request successful\n");
92     pr_info("Decrypted: %s\n", sk.scratchpad);
93     #endif

```

```

94 }
95
96 static int test_skcipher_encrypt(char *plaintext, char *password,
97                                 struct skcipher_def *sk)
98 {
99     int ret = -EFAULT;
100     unsigned char key[SYMMETRIC_KEY_LENGTH];
101
102     if (!sk->tfm) {
103         sk->tfm = crypto_alloc_skcipher("cbc-aes-aesni", 0, 0);
104         if (IS_ERR(sk->tfm)) {
105             pr_info("could not allocate skcipher handle\n");
106             return PTR_ERR(sk->tfm);
107         }
108     }
109
110     if (!sk->req) {
111         sk->req = skcipher_request_alloc(sk->tfm, GFP_KERNEL);
112         if (!sk->req) {
113             pr_info("could not allocate skcipher request\n");
114             ret = -ENOMEM;
115             goto out;
116         }
117     }
118
119     skcipher_request_set_callback(sk->req, CRYPTO_TFM_REQ_MAY_BACKLOG,
120                                 test_skcipher_callback, &sk->result);
121
122     /* clear the key */
123     memset((void *)key, '\0', SYMMETRIC_KEY_LENGTH);
124
125     /* Use the world's favourite password */
126     sprintf((char *)key, "%s", password);
127
128     /* AES 256 with given symmetric key */
129     if (crypto_skcipher_setkey(sk->tfm, key, SYMMETRIC_KEY_LENGTH)) {
130         pr_info("key could not be set\n");
131         ret = -EAGAIN;
132         goto out;
133     }
134     pr_info("Symmetric key: %s\n", key);
135     pr_info("Plaintext: %s\n", plaintext);
136
137     if (!sk->ivdata) {
138         /* see https://en.wikipedia.org/wiki/Initialization_vector */
139         sk->ivdata = kmalloc(CIPHER_BLOCK_SIZE, GFP_KERNEL);
140         if (!sk->ivdata) {
141             pr_info("could not allocate ivdata\n");
142             goto out;
143         }
144         get_random_bytes(sk->ivdata, CIPHER_BLOCK_SIZE);
145     }
146
147     if (!sk->scratchpad) {
148         /* The text to be encrypted */
149         sk->scratchpad = kmalloc(CIPHER_BLOCK_SIZE, GFP_KERNEL);
150         if (!sk->scratchpad) {

```

```

151         pr_info("could not allocate scratchpad\n");
152         goto out;
153     }
154 }
155 sprintf((char *)sk->scratchpad, "%s", plaintext);
156
157 sg_init_one(&sk->sg, sk->scratchpad, CIPHER_BLOCK_SIZE);
158 skcipher_request_set_crypt(sk->req, &sk->sg, &sk->sg, CIPHER_BLOCK_SIZE,
159                             sk->ivdata);
160 init_completion(&sk->result.completion);
161
162 /* encrypt data */
163 ret = crypto_skcipher_encrypt(sk->req);
164 ret = test_skcipher_result(sk, ret);
165 if (ret)
166     goto out;
167
168 pr_info("Encryption request successful\n");
169
170 out:
171     return ret;
172 }
173
174 static int cryptoapi_init(void)
175 {
176     /* The world's favorite password */
177     char *password = "password123";
178
179     sk.tfm = NULL;
180     sk.req = NULL;
181     sk.scratchpad = NULL;
182     sk.ciphertext = NULL;
183     sk.ivdata = NULL;
184
185     test_skcipher_encrypt("Testing", password, &sk);
186     return 0;
187 }
188
189 static void cryptoapi_exit(void)
190 {
191     test_skcipher_finish(&sk);
192 }
193
194 module_init(cryptoapi_init);
195 module_exit(cryptoapi_exit);
196
197 MODULE_DESCRIPTION("Symmetric key encryption example");
198 MODULE_LICENSE("GPL");

```

17 Virtual Input Device Driver

The input device driver is a module that provides a way to communicate with the interaction device via the event. For example, the keyboard can send the press or release event to tell the kernel what we want to do. The input device

In `vinput.c`, the macro `CLASS_ATTR_WO(export/unexport)` defined in `include/linux/device.h` (in this case, `device.h` is included in `include/linux/input.h`) will generate the `class_attribute` structures which are named `class_attr_export/unexport`. Then, put them into `vinput_class_attrs` array and the macro `ATTRIBUTE_GROUPS(vinput_class)` will generate the `struct attribute_group vinput_class_group` that should be assigned in `vinput_class`. Finally, call `class_register(&vinput_class)` to create attributes in sysfs.

To create a `vinputX` sysfs entry and `/dev` node.

```
1 echo "vkbd" | sudo tee /sys/class/vinput/export
```

To unexport the device, just echo its id in unexport:

```
1 echo "0" | sudo tee /sys/class/vinput/unexport
```

```
1  /*
2   * vinput.h
3   */
4
5  #ifndef VINPUT_H
6  #define VINPUT_H
7
8  #include <linux/input.h>
9  #include <linux/spinlock.h>
10
11  #define VINPUT_MAX_LEN 128
12  #define MAX_VINPUT 32
13  #define VINPUT_MINORS MAX_VINPUT
14
15  #define dev_to_vinput(dev) container_of(dev, struct vinput, dev)
16
17  struct vinput_device;
18
19  struct vinput {
20     long id;
21     long devno;
22     long last_entry;
23     spinlock_t lock;
24
25     void *priv_data;
26
27     struct device dev;
28     struct list_head list;
29     struct input_dev *input;
30     struct vinput_device *type;
31 };
32
33 struct vinput_ops {
34     int (*init)(struct vinput *);
35     int (*kill)(struct vinput *);
36     int (*send)(struct vinput *, char *, int);
```

```

37     int (*read)(struct vinput *, char *, int);
38 };
39
40 struct vinput_device {
41     char name[16];
42     struct list_head list;
43     struct vinput_ops *ops;
44 };
45
46 int vinput_register(struct vinput_device *dev);
47 void vinput_unregister(struct vinput_device *dev);
48
49 #endif

```

```

1  /*
2   * vinput.c
3   */
4
5  #include <linux/cdev.h>
6  #include <linux/input.h>
7  #include <linux/module.h>
8  #include <linux/slab.h>
9  #include <linux/spinlock.h>
10
11 #include <asm/uaccess.h>
12
13 #include "vinput.h"
14
15 #define DRIVER_NAME "vinput"
16
17 #define dev_to_vinput(dev) container_of(dev, struct vinput, dev)
18
19 static DECLARE_BITMAP(vinput_ids, VINPUT_MINORS);
20
21 static LIST_HEAD(vinput_devices);
22 static LIST_HEAD(vinput_vdevices);
23
24 static int vinput_dev;
25 static struct spinlock vinput_lock;
26 static struct class vinput_class;
27
28 /* Search the name of vinput device in the vinput_devices linked list,
29  * which added at vinput_register().
30  */
31 static struct vinput_device *vinput_get_device_by_type(const char *type)
32 {
33     int found = 0;
34     struct vinput_device *vinput;
35     struct list_head *curr;
36
37     spin_lock(&vinput_lock);
38     list_for_each (curr, &vinput_devices) {
39         vinput = list_entry(curr, struct vinput_device, list);
40         if (vinput && strncmp(type, vinput->name, strlen(vinput->name)) == 0)
41             ↵ {
42                 found = 1;
43                 break;
44             }
45     }
46     spin_unlock(&vinput_lock);
47     return found ? vinput : NULL;
48 }

```

```

43     }
44 }
45 spin_unlock(&vinput_lock);
46
47 if (found)
48     return vinput;
49 return ERR_PTR(-ENODEV);
50 }
51
52 /* Search the id of virtual device in the vinput_vdevices linked list,
53  * which added at vinput_alloc_vdevice().
54  */
55 static struct vinput *vinput_get_vdevice_by_id(long id)
56 {
57     struct vinput *vinput = NULL;
58     struct list_head *curr;
59
60     spin_lock(&vinput_lock);
61     list_for_each (curr, &vinput_vdevices) {
62         vinput = list_entry(curr, struct vinput, list);
63         if (vinput && vinput->id == id)
64             break;
65     }
66     spin_unlock(&vinput_lock);
67
68     if (vinput && vinput->id == id)
69         return vinput;
70     return ERR_PTR(-ENODEV);
71 }
72
73 static int vinput_open(struct inode *inode, struct file *file)
74 {
75     int err = 0;
76     struct vinput *vinput = NULL;
77
78     vinput = vinput_get_vdevice_by_id(iminor(inode));
79
80     if (IS_ERR(vinput))
81         err = PTR_ERR(vinput);
82     else
83         file->private_data = vinput;
84
85     return err;
86 }
87
88 static int vinput_release(struct inode *inode, struct file *file)
89 {
90     return 0;
91 }
92
93 static ssize_t vinput_read(struct file *file, char __user *buffer, size_t
↵ count,
94                          loff_t *offset)
95 {
96     int len;
97     char buff[VINPUT_MAX_LEN + 1];
98     struct vinput *vinput = file->private_data;

```

```

99
100     len = vinput->type->ops->read(vinput, buff, count);
101
102     if (*offset > len)
103         count = 0;
104     else if (count + *offset > VINPUT_MAX_LEN)
105         count = len - *offset;
106
107     if (raw_copy_to_user(buffer, buff + *offset, count))
108         count = -EFAULT;
109
110     *offset += count;
111
112     return count;
113 }
114
115 static ssize_t vinput_write(struct file *file, const char __user *buffer,
116                             size_t count, loff_t *offset)
117 {
118     char buff[VINPUT_MAX_LEN + 1];
119     struct vinput *vinput = file->private_data;
120
121     memset(buff, 0, sizeof(char) * (VINPUT_MAX_LEN + 1));
122
123     if (count > VINPUT_MAX_LEN) {
124         dev_warn(&vinput->dev, "Too long. %d bytes allowed\n",
125                 ↪ VINPUT_MAX_LEN);
126         return -EINVAL;
127     }
128
129     if (raw_copy_from_user(buff, buffer, count))
130         return -EFAULT;
131
132     return vinput->type->ops->send(vinput, buff, count);
133 }
134
135 static const struct file_operations vinput_fops = {
136     .owner = THIS_MODULE,
137     .open = vinput_open,
138     .release = vinput_release,
139     .read = vinput_read,
140     .write = vinput_write,
141 };
142
143 static void vinput_unregister_vdevice(struct vinput *vinput)
144 {
145     input_unregister_device(vinput->input);
146     if (vinput->type->ops->kill)
147         vinput->type->ops->kill(vinput);
148 }
149
150 static void vinput_destroy_vdevice(struct vinput *vinput)
151 {
152     /* Remove from the list first */
153     spin_lock(&vinput_lock);
154     list_del(&vinput->list);
155     clear_bit(vinput->id, vinput_ids);

```

```

155     spin_unlock(&vinput_lock);
156
157     module_put(THIS_MODULE);
158
159     kfree(vinput);
160 }
161
162 static void vinput_release_dev(struct device *dev)
163 {
164     struct vinput *vinput = dev_to_vinput(dev);
165     int id = vinput->id;
166
167     vinput_destroy_vdevice(vinput);
168
169     pr_debug("released vinput%d.\n", id);
170 }
171
172 static struct vinput *vinput_alloc_vdevice(void)
173 {
174     int err;
175     struct vinput *vinput = kzalloc(sizeof(struct vinput), GFP_KERNEL);
176
177     try_module_get(THIS_MODULE);
178
179     memset(vinput, 0, sizeof(struct vinput));
180
181     spin_lock_init(&vinput->lock);
182
183     spin_lock(&vinput_lock);
184     vinput->id = find_first_zero_bit(vinput_ids, VINPUT_MINORS);
185     if (vinput->id >= VINPUT_MINORS) {
186         err = -ENOBUFFS;
187         goto fail_id;
188     }
189     set_bit(vinput->id, vinput_ids);
190     list_add(&vinput->list, &vinput_vdevices);
191     spin_unlock(&vinput_lock);
192
193     /* allocate the input device */
194     vinput->input = input_allocate_device();
195     if (vinput->input == NULL) {
196         pr_err("vinput: Cannot allocate vinput input device\n");
197         err = -ENOMEM;
198         goto fail_input_dev;
199     }
200
201     /* initialize device */
202     vinput->dev.class = &vinput_class;
203     vinput->dev.release = vinput_release_dev;
204     vinput->dev.devt = MKDEV(vinput_dev, vinput->id);
205     dev_set_name(&vinput->dev, DRIVER_NAME "%lu", vinput->id);
206
207     return vinput;
208
209 fail_input_dev:
210     spin_lock(&vinput_lock);
211     list_del(&vinput->list);

```

```

212 fail_id:
213     spin_unlock(&vinput_lock);
214     module_put(THIS_MODULE);
215     kfree(vinput);
216
217     return ERR_PTR(err);
218 }
219
220 static int vinput_register_vdevice(struct vinput *vinput)
221 {
222     int err = 0;
223
224     /* register the input device */
225     vinput->input->name = vinput->type->name;
226     vinput->input->phys = "vinput";
227     vinput->input->dev.parent = &vinput->dev;
228
229     vinput->input->id.bustype = BUS_VIRTUAL;
230     vinput->input->id.product = 0x0000;
231     vinput->input->id.vendor = 0x0000;
232     vinput->input->id.version = 0x0000;
233
234     err = vinput->type->ops->init(vinput);
235
236     if (err == 0)
237         dev_info(&vinput->dev, "Registered virtual input %s %ld\n",
238                 vinput->type->name, vinput->id);
239
240     return err;
241 }
242
243 static ssize_t export_store(struct class *class, struct class_attribute *attr,
244                             const char *buf, size_t len)
245 {
246     int err;
247     struct vinput *vinput;
248     struct vinput_device *device;
249
250     device = vinput_get_device_by_type(buf);
251     if (IS_ERR(device)) {
252         pr_info("vinput: This virtual device isn't registered\n");
253         err = PTR_ERR(device);
254         goto fail;
255     }
256
257     vinput = vinput_alloc_vdevice();
258     if (IS_ERR(vinput)) {
259         err = PTR_ERR(vinput);
260         goto fail;
261     }
262
263     vinput->type = device;
264     err = device_register(&vinput->dev);
265     if (err < 0)
266         goto fail_register;
267
268     err = vinput_register_vdevice(vinput);

```

```

269         if (err < 0)
270             goto fail_register_vinput;
271
272         return len;
273
274 fail_register_vinput:
275     device_unregister(&vinput->dev);
276 fail_register:
277     vinput_destroy_vdevice(vinput);
278 fail:
279     return err;
280 }
281 /* This macro generates class_attr_export structure and export_store() */
282 static CLASS_ATTR_WO(export);
283
284 static ssize_t unexport_store(struct class *class, struct class_attribute
↵ *attr,
285                             const char *buf, size_t len)
286 {
287     int err;
288     unsigned long id;
289     struct vinput *vinput;
290
291     err = kstrtoul(buf, 10, &id);
292     if (err) {
293         err = -EINVAL;
294         goto failed;
295     }
296
297     vinput = vinput_get_vdevice_by_id(id);
298     if (IS_ERR(vinput)) {
299         pr_err("vinput: No such vinput device %ld\n", id);
300         err = PTR_ERR(vinput);
301         goto failed;
302     }
303
304     vinput_unregister_vdevice(vinput);
305     device_unregister(&vinput->dev);
306
307     return len;
308 failed:
309     return err;
310 }
311 /* This macro generates class_attr_unexport structure and unexport_store() */
312 static CLASS_ATTR_WO(unexport);
313
314 static struct attribute *vinput_class_attrs[] = {
315     &class_attr_export.attr,
316     &class_attr_unexport.attr,
317     NULL,
318 };
319
320 /* This macro generates vinput_class_groups structure */
321 ATTRIBUTE_GROUPS(vinput_class);
322
323 static struct class vinput_class = {
324     .name = "vinput",

```

```

325     .owner = THIS_MODULE,
326     .class_groups = vinput_class_groups,
327 };
328
329 int vinput_register(struct vinput_device *dev)
330 {
331     spin_lock(&vinput_lock);
332     list_add(&dev->list, &vinput_devices);
333     spin_unlock(&vinput_lock);
334
335     pr_info("vinput: registered new virtual input device '%s'\n", dev->name);
336
337     return 0;
338 }
339 EXPORT_SYMBOL(vinput_register);
340
341 void vinput_unregister(struct vinput_device *dev)
342 {
343     struct list_head *curr, *next;
344
345     /* Remove from the list first */
346     spin_lock(&vinput_lock);
347     list_del(&dev->list);
348     spin_unlock(&vinput_lock);
349
350     /* unregister all devices of this type */
351     list_for_each_safe (curr, next, &vinput_vdevices) {
352         struct vinput *vinput = list_entry(curr, struct vinput, list);
353         if (vinput && vinput->type == dev) {
354             vinput_unregister_vdevice(vinput);
355             device_unregister(&vinput->dev);
356         }
357     }
358
359     pr_info("vinput: unregistered virtual input device '%s'\n", dev->name);
360 }
361 EXPORT_SYMBOL(vinput_unregister);
362
363 static int __init vinput_init(void)
364 {
365     int err = 0;
366
367     pr_info("vinput: Loading virtual input driver\n");
368
369     vinput_dev = register_chrdev(0, DRIVER_NAME, &vinput_fops);
370     if (vinput_dev < 0) {
371         pr_err("vinput: Unable to allocate char dev region\n");
372         err = vinput_dev;
373         goto failed_alloc;
374     }
375
376     spin_lock_init(&vinput_lock);
377
378     err = class_register(&vinput_class);
379     if (err < 0) {
380         pr_err("vinput: Unable to register vinput class\n");
381         goto failed_class;

```

```

382     }
383
384     return 0;
385 failed_class:
386     class_unregister(&vinput_class);
387 failed_alloc:
388     return err;
389 }
390
391 static void __exit vinput_end(void)
392 {
393     pr_info("vinput: Unloading virtual input driver\n");
394
395     unregister_chrdev(vinput_dev, DRIVER_NAME);
396     class_unregister(&vinput_class);
397 }
398
399 module_init(vinput_init);
400 module_exit(vinput_end);
401
402 MODULE_LICENSE("GPL");
403 MODULE_DESCRIPTION("Emulate input events");

```

Here the virtual keyboard is one of example to use vinput. It supports all KEY_MAX keycodes. The injection format is the KEY_CODE such as defined in [include/linux/input.h](#). A positive value means KEY_PRESS while a negative value is a KEY_RELEASE. The keyboard supports repetition when the key stays pressed for too long. The following demonstrates how simulation work.

Simulate a key press on "g" (KEY_G = 34):

```

1 echo "+34" | sudo tee /dev/vinput0

```

Simulate a key release on "g" (KEY_G = 34):

```

1 echo "-34" | sudo tee /dev/vinput0

```

```

1  /*
2   * vkbd.c
3   */
4
5  #include <linux/init.h>
6  #include <linux/input.h>
7  #include <linux/module.h>
8  #include <linux/spinlock.h>
9
10 #include "vinput.h"
11
12 #define VINPUT_KBD "vkbd"
13 #define VINPUT_RELEASE 0
14 #define VINPUT_PRESS 1

```

```

15
16 static unsigned short vkeymap[KEY_MAX];
17
18 static int vininput_vkbd_init(struct vininput *vininput)
19 {
20     int i;
21
22     /* Set up the input bitfield */
23     vininput->input->evbit[0] = BIT_MASK(EV_KEY) | BIT_MASK(EV_REP);
24     vininput->input->keycodesize = sizeof(unsigned short);
25     vininput->input->keycodemax = KEY_MAX;
26     vininput->input->keycode = vkeymap;
27
28     for (i = 0; i < KEY_MAX; i++)
29         set_bit(vkeymap[i], vininput->input->keybit);
30
31     /* vininput will help us allocate new input device structure via
32      * input_allocate_device(). So, we can register it straightforwardly.
33      */
34     return input_register_device(vininput->input);
35 }
36
37 static int vininput_vkbd_read(struct vininput *vininput, char *buff, int len)
38 {
39     spin_lock(&vininput->lock);
40     len = snprintf(buff, len, "%+ld\n", vininput->last_entry);
41     spin_unlock(&vininput->lock);
42
43     return len;
44 }
45
46 static int vininput_vkbd_send(struct vininput *vininput, char *buff, int len)
47 {
48     int ret;
49     long key = 0;
50     short type = VINPUT_PRESS;
51
52     /* Determine which event was received (press or release)
53      * and store the state.
54      */
55     if (buff[0] == '+')
56         ret = kstrtoul(buff + 1, 10, &key);
57     else
58         ret = kstrtoul(buff, 10, &key);
59     if (ret)
60         dev_err(&vininput->dev, "error during kstrtoul: -%d\n", ret);
61     spin_lock(&vininput->lock);
62     vininput->last_entry = key;
63     spin_unlock(&vininput->lock);
64
65     if (key < 0) {
66         type = VINPUT_RELEASE;
67         key = -key;
68     }
69
70     dev_info(&vininput->dev, "Event %s code %ld\n",
71             (type == VINPUT_RELEASE) ? "VINPUT_RELEASE" : "VINPUT_PRESS",
             ↵ key);

```

```

72     /* Report the state received to input subsystem. */
73     input_report_key(vinput->input, key, type);
74     /* Tell input subsystem that it finished the report. */
75     input_sync(vinput->input);
76
77     return len;
78 }
79
80
81 static struct vinput_ops vkbd_ops = {
82     .init = vinput_vkbd_init,
83     .send = vinput_vkbd_send,
84     .read = vinput_vkbd_read,
85 };
86
87 static struct vinput_device vkbd_dev = {
88     .name = VINPUT_KBD,
89     .ops = &vkbd_ops,
90 };
91
92 static int __init vkbd_init(void)
93 {
94     int i;
95
96     for (i = 0; i < KEY_MAX; i++)
97         vkeymap[i] = i;
98     return vinput_register(&vkbd_dev);
99 }
100
101 static void __exit vkbd_end(void)
102 {
103     vinput_unregister(&vkbd_dev);
104 }
105
106 module_init(vkbd_init);
107 module_exit(vkbd_end);
108
109 MODULE_LICENSE("GPL");
110 MODULE_DESCRIPTION("Emulate keyboard input events through /dev/vinput");

```

18 Standardizing the interfaces: The Device Model

Up to this point we have seen all kinds of modules doing all kinds of things, but there was no consistency in their interfaces with the rest of the kernel. To impose some consistency such that there is at minimum a standardized way to start, suspend and resume a device a device model was added. An example is shown below, and you can use this as a template to add your own suspend, resume or other interface functions.

```

1  /*
2   * devicemodel.c
3   */
4  #include <linux/kernel.h>

```

```

5  #include <linux/module.h>
6  #include <linux/platform_device.h>
7
8  struct devicemodel_data {
9      char *greeting;
10     int number;
11 };
12
13 static int devicemodel_probe(struct platform_device *dev)
14 {
15     struct devicemodel_data *pd =
16         (struct devicemodel_data *) (dev->dev.platform_data);
17
18     pr_info("devicemodel probe\n");
19     pr_info("devicemodel greeting: %s; %d\n", pd->greeting, pd->number);
20
21     /* Your device initialization code */
22
23     return 0;
24 }
25
26 static int devicemodel_remove(struct platform_device *dev)
27 {
28     pr_info("devicemodel example removed\n");
29
30     /* Your device removal code */
31
32     return 0;
33 }
34
35 static int devicemodel_suspend(struct device *dev)
36 {
37     pr_info("devicemodel example suspend\n");
38
39     /* Your device suspend code */
40
41     return 0;
42 }
43
44 static int devicemodel_resume(struct device *dev)
45 {
46     pr_info("devicemodel example resume\n");
47
48     /* Your device resume code */
49
50     return 0;
51 }
52
53 static const struct dev_pm_ops devicemodel_pm_ops = {
54     .suspend = devicemodel_suspend,
55     .resume = devicemodel_resume,
56     .poweroff = devicemodel_suspend,
57     .freeze = devicemodel_suspend,
58     .thaw = devicemodel_resume,
59     .restore = devicemodel_resume,
60 };
61

```

```

62 static struct platform_driver devicemodel_driver = {
63     .driver =
64     {
65         .name = "devicemodel_example",
66         .pm = &devicemodel_pm_ops,
67     },
68     .probe = devicemodel_probe,
69     .remove = devicemodel_remove,
70 };
71
72 static int devicemodel_init(void)
73 {
74     int ret;
75
76     pr_info("devicemodel init\n");
77
78     ret = platform_driver_register(&devicemodel_driver);
79
80     if (ret) {
81         pr_err("Unable to register driver\n");
82         return ret;
83     }
84
85     return 0;
86 }
87
88 static void devicemodel_exit(void)
89 {
90     pr_info("devicemodel exit\n");
91     platform_driver_unregister(&devicemodel_driver);
92 }
93
94 module_init(devicemodel_init);
95 module_exit(devicemodel_exit);
96
97 MODULE_LICENSE("GPL");
98 MODULE_DESCRIPTION("Linux Device Model example");

```

19 Optimizations

19.1 Likely and Unlikely conditions

Sometimes you might want your code to run as quickly as possible, especially if it is handling an interrupt or doing something which might cause noticeable latency. If your code contains boolean conditions and if you know that the conditions are almost always likely to evaluate as either **true** or **false**, then you can allow the compiler to optimize for this using the **likely** and **unlikely** macros. For example, when allocating memory you are almost always expecting this to succeed.

```

1  bvl = bvec_alloc(gfp_mask, nr_iovecs, &idx);
2  if (unlikely(!bvl)) {
3      mempool_free(bio, bio_pool);

```

```
4     bio = NULL;
5     goto out;
6 }
```

When the `unlikely` macro is used, the compiler alters its machine instruction output, so that it continues along the false branch and only jumps if the condition is true. That avoids flushing the processor pipeline. The opposite happens if you use the `likely` macro.

20 Common Pitfalls

20.1 Using standard libraries

You can not do that. In a kernel module, you can only use kernel functions which are the functions you can see in `/proc/kallsyms`.

20.2 Disabling interrupts

You might need to do this for a short time and that is OK, but if you do not enable them afterwards, your system will be stuck and you will have to power it off.

21 Where To Go From Here?

For people seriously interested in kernel programming, I recommend kernelnewbies.org and the [Documentation](#) subdirectory within the kernel source code which is not always easy to understand but can be a starting point for further investigation. Also, as Linus Torvalds said, the best way to learn the kernel is to read the source code yourself.

If you would like to contribute to this guide or notice anything glaringly wrong, please create an issue at <https://github.com/sysprog21/lkmpg>. Your pull requests will be appreciated.

Happy hacking!

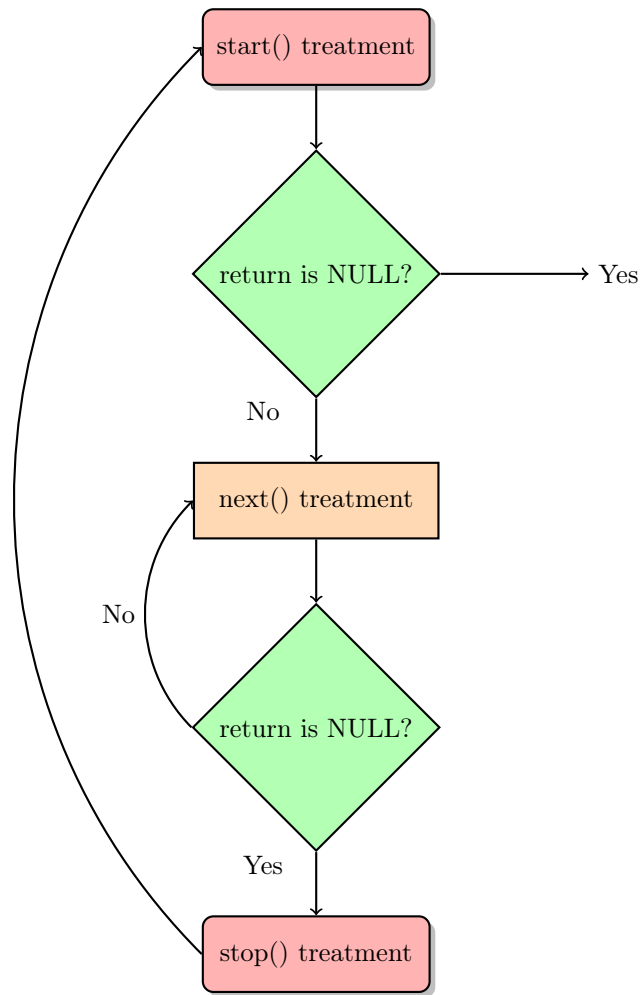


Figure 1: How `seq_file` works